

RISC-V Specification for CHERI Extensions

Authors: Thomas Aird, Hesham Almatary, Andres Amaya Garcia, John Baldwin, Paul Buxton, David Chisnall, Jessica Clarke, Brooks Davis, Nathaniel Wesley Filardo, Franz A. Fuchs, Timothy Hutt, Alexandre Joannou, Martin Kaiser, Tariq Kurd, Ben Laurie, Marno van der Maas, Maja Malenko, A. Theodore Markettos, David McKay, Jamie Melling, Stuart Menefy, Simon W. Moore, Peter G. Neumann, Robert Norton, Alexander Richardson, Michael Roe, Peter Rugg, Peter Sewell, Carl Shaw, Ricki Tura, Robert N. M. Watson, Toby Wenman, Jonathan Woodruff, Jason Zhijingcheng Yu, Robert Riglar, Florian Schmaus

Version v0.9.10, 20260929: Intermediate Release

Table of Contents

Preamble 1
Copyright and License Information 2
Contributors 3
1. Introduction 5
1.1. CHERI Concepts and Terminology 5
1.2. CHERI for RISC-V 5
1.2.1. Extensions and Specifications 5
Chapters for the Unprivileged Specification 7
2. RVY Base Capability Instruction Set, Version 1.0 8
2.1. CHERI Overview 8
2.2. CHERI Protection Model 8
2.3. Capability Registers and Format 9
2.3.1. Address 9
2.3.2. Capability Tag 9
2.3.3. Capability tags in Registers 10
2.3.4. Capability tags in Memory 11
2.3.5. Capability Bounds 11
2.3.6. Deriving New Bounds 12
2.3.7. Representability and Updating the Address 12
2.3.8. Memory Space 13
2.3.9. Capability Type (CT) 14
2.3.10. Architectural Permissions (AP) 15
2.3.10.1. Permission Transitions 17
2.3.11. Software-Defined Permissions (SDP) 17
2.3.12. Special Capabilities 18
2.3.12.1. Root Capabilities 18
2.3.12.2. NULL Capability 18
2.4. Capability-Encoding Formats 18
2.5. Integrity of Capabilities 20
2.6. Extended State 21
2.6.1. General-Purpose Registers 21
2.6.2. The Program Counter Capability (`pc`) 22
2.6.3. Added CSRs 22
2.6.3.1. User Thread Identifier Capability (utidc) 22
2.6.4. CSR Classes 23
2.7. Capability Checks 24
2.8. RVY Instruction Encodings 24
2.8.1. Instructions to Update The Capability Pointer 25
2.8.1.1. YADD, YADDI 26
2.8.1.2. YADDRW 28
2.8.2. Instructions to Manipulate Capabilities 29
2.8.2.1. YPERMC 30
2.8.2.2. YMV 32

2.8.2.3. YHIW 33
2.8.2.4. YBNDSW, YBNDSWI 34
2.8.2.5. YBNDSRW 37
2.8.2.6. YUNSEALS 38
2.8.2.7. YBLD 39
2.8.2.8. YSEALE 40
2.8.3. Instructions to Decode Capability Bounds 41
2.8.3.1. YBASER 42
2.8.3.2. YLENR 43
2.8.3.3. YTOPR 43
2.8.4. Instructions to Extract Capability Fields 45
2.8.4.1. YTAGR 46
2.8.4.2. YPERMR 47
2.8.4.3. YTYPER 48
2.8.4.4. YHIR 48
2.8.5. Miscellaneous Instructions to Handle Capability Data 50
2.8.5.1. YEQ 51
2.8.5.2. YSS 52
2.8.5.3. YAMASK 53
2.8.6. Instructions to Load and Store Capability Data 54
2.8.6.1. LY 55
2.8.6.2. SY 58
2.9. Changes to Existing RISC-V Base ISA Instructions 61
2.9.1. Changes to Load/Stores 61
2.9.2. Changes to PC 62
2.9.3. AUIPC (RVY) 63
2.9.4. JAL (RVY) 64
2.9.5. JALR (RVY) 65
2.10. RVY ISA Summary 66
2.10.1. RVY Added Instructions 66
2.10.2. RVI (RVY modified behavior) 66
3. The RV64LYA Capability Base for RV64Y 67
3.1. Capability-Encoding Format 67
3.1.1. Capability-Encoding Format Parameter Summary 68
3.1.2. Architectural Permissions (AP) Encoding 68
3.1.3. Software-Defined Permissions (SDP) Encoding 69
3.1.4. Capability Type (CT) Encoding 69
3.1.5. Bounds (EF, T, TE, B, BE, L_8) Encoding 69
3.1.5.1. Concept 69
3.1.5.2. Calculation of Variables 70
3.1.5.3. Format of the Bounds 71
3.1.5.4. Correction Factors 72
3.1.5.4.1. Correction Factor Calculations 73
3.1.5.4.2. Correction Factor Summary 74
3.1.5.5. Top Bound Address MSB Correction 74

3.1.5.6. Malformed Capability Bounds 75
3.2. Representable Range Check 75
3.2.1. Practical Information 75
3.3. Encoding of Special Capabilities 77
3.3.1. NULL Capability Encoding 77
3.3.2. Infinite Capability Encoding 78
4. Zca (RVY Added and Modified Instructions) 79
4.1. RV64 / RV64Y RVC Load/Store Mapping Summary 80
4.1.1. C.LY 81
4.1.2. C.LYSP 81
4.1.3. C.SY 82
4.1.4. C.SYSP 82
4.2. Zca (RVY modified behavior) 82
4.2.1. C.ADDI16SP (RVY) 84
4.2.2. C.ADDI4SPN (RVY) 85
4.2.3. C.YMV 86
4.2.4. C.JR (RVY) 87
4.2.5. C.JALR (RVY) 88
4.2.6. Zca HINT Encodings 88
5. Zalrsc (RVY added instructions) 89
5.1. LR.Y 90
5.2. SC.Y 92
6. Zaamo (RVY added instructions) 95
6.1. AMOSWAP.Y 96
7. Zba (RVY added instructions) 99
7.1. YSH1ADD, YSH2ADD, YSH3ADD, YSH4ADD (RV64Y) 100
7.2. YSH1ADD.UW, YSH2ADD.UW, YSH3ADD.UW, YSH4ADD.UW (RV64Y) 101
8. Zicbom (RVY modified behavior) 102
8.1. CBO.CLEAN (RVY) 103
8.2. CBO.FLUSH (RVY) 104
8.3. CBO.INVAL (RVY) 105
9. Zicboz (RVY modified behavior) 107
9.1. CBO.ZERO (RVY) 108
10. Zicbop (RVY modified behavior) 110
10.1. PREFETCH.I (RVY) 111
10.2. PREFETCH.R (RVY) 112
10.3. PREFETCH.W (RVY) 113
11. U-mode YLEN-bit CSR Summary 114
12. Vector "V" Extension (RVY) 115
13. "Zylg" Extension for CHERI 2-level (Local/Global) Information Flow Control 116
13.1. Added Metadata Bits (Permissions and Flags) 116
13.1.1. The Capability Global (GL) Flag 116
13.1.2. Store Local Permission (SL) 116
13.1.3. Load Global Permission (LG) 116
13.2. Interaction with Root Capabilities 117

13.3. Interaction with YPERMC and YPERMR ... 117
13.3.1. Additional YPERMC Permission Rules ... 117
13.4. Interaction with LY ... 117
13.5. Interaction with SY ... 118
13.6. Interaction with Capability Subsets ... 118
13.7. Rationale and Software Use Cases ... 119
13.7.1. Restricting Capability Flow Between Regions or Domains (e.g., Stack-to-Heap or Kernel-to-User) ... 119
13.7.2. Enforcing Shallow vs. Deep No-Capture Guarantees ... 119
14. "Zyhybrid" Extension for CHERI Pointer Modes ... 120
14.1. CHERI Pointer Modes ... 120
14.1.1. CHERI Pointer Mode Encoding ... 122
14.1.2. Changing CHERI Pointer Mode ... 122
14.1.3. Additional YPERMC Permission Rule ... 122
14.1.4. Observing the CHERI Pointer Mode ... 123
14.2. Added Instructions ... 124
14.2.1. YMODEW ... 125
14.2.2. YMODER ... 126
14.2.3. YMODESWY, YMODESWI ... 127
14.3. Added State ... 127
14.3.1. Default Data Capability CSR (ddc) ... 128
15. Zicsr (RVY modified behavior) ... 129
15.1. CSRRWI, CSRRS, CSRRSI, CSRRC, CSRRCI (RVY) ... 130
15.2. CSRRW (RVY) ... 132
16. Placeholder References to the Unprivileged Spec ... 133
Chapters for the Privileged Specification ... 134
17. "Smy" Extension for Machine-Level Architecture, Version 1.0 ... 135
17.1. Machine-Level CSRs widened by RVY ... 135
17.1.1. Machine Trap Vector Base Address Capability Register (mtvec) ... 135
17.1.2. Machine Scratch Capability Register (mscratch) ... 136
17.1.3. Machine Exception Program Counter Capability (mepc) ... 136
17.2. Machine-Level CSRs added by RVY ... 137
17.2.1. Machine Thread Identifier Capability (mtidc) ... 137
17.3. Machine-Level CSRs modified by RVY ... 137
17.3.1. Machine Environment Configuration (`menvcfg`) Register ... 137
17.3.2. Machine ISA (`misa`) Register ... 137
17.3.3. Machine Status Registers (mstatus) ... 138
17.3.4. Machine Cause Register (mcause) ... 139
17.3.5. Machine Trap Delegation Register (medeleg) ... 140
17.3.6. "Smstateen" Integration ... 140
17.4. CHERI Exception Handling ... 140
17.5. CHERI Exceptions and Speculative Execution ... 141
17.6. Physical Memory Attributes (PMA) ... 142
17.7. Modified MRET Behavior ... 142
17.8. M-mode YLEN-bit CSR Summary ... 142

18. "Ssy" Extension for Supervisor-Level Architecture, Version 1.0 ..... **144**
18.1. Supervisor-Level CSRs widened by RVY 144
18.1.1. Supervisor Trap Vector Base Address Capability Register (stvec) 144
18.1.2. Supervisor Status Registers (sstatus) 144
18.1.3. Supervisor Scratch Capability Register (sscratch) 144
18.1.4. Supervisor Exception Program Counter Capability (sepc) 144
18.2. Supervisor-Level CSRs added by RVY 145
18.2.1. Supervisor Thread Identifier Capability (stidc) 145
18.3. Supervisor-Level CSRs modified by RVY 145
18.3.1. Supervisor Environment Configuration Register (senvcfg) 145
18.3.2. Supervisor Cause Register (scause) 145
18.3.3. "Ssstateen" Integration 145
18.4. Modified SRET Behavior 145
18.4.1. SRET (RVY) 146
18.4.2. MRET (RVY) 146
18.5. S-mode YLEN-bit CSR Summary 147
19. "Sv39/Sv48/Sv57 (RVY)" Page-Based Virtual-Memory Extensions, Version 1.0 ..... **149**
19.1. Limiting Capability Propagation 149
19.2. The *pte.rvy* Field for Capability Flow Control 149
19.3. Invalid Virtual Address Handling 150
19.3.1. Updating CSRs 150
19.3.2. Branches and Jumps 150
19.3.3. Memory Accesses 151
20. "Smyhybrid and Ssyhybrid" Extensions, Version 1.0 ..... **152**
21. "Svyrg" Extension, Version 1.0 for RV64Y **154**
21.1. CHERI Load Capability Faults 155
21.2. Capability Dirty Tracking 156
21.3. Virtual Address Translation Process 158
Appendix A: RVY Instruction Encoding Overview **160**
A.1. Bytefield Encoding Overview 160
A.1.1. 3OP Instruction Encodings 160
A.1.2. 2OP Instruction Encodings 160
A.1.3. YADDI/LY/SY Instruction Encodings 160
A.1.4. AMO Instruction Encodings 161
A.1.5. MISC Instruction Encodings 161
A.2. Wavedrom Encoding Overview 161
A.2.1. 3OP Instruction Encodings 161
A.2.2. 2OP Instruction Encodings 161
A.2.3. YADDI/LY/SY Instruction Encodings 161
A.2.4. AMO Instruction Encodings 162
A.2.5. MISC Instruction Encodings 162
A.3. AsciiDoc Table Encoding Overview 162
A.3.1. 3OP Instruction Encodings 162
A.3.2. 2OP Instruction Encodings 163
A.3.3. YADDI/LY/SY Instruction Encodings 163

A.3.4. AMO Instruction Encodings 164
A.3.5. MISC Instruction Encodings 164
A.4. RVY-A Instruction Allocation 164
A.4.1. Funct3 Allocations 164
A.4.2. R-Type 3-Operand (funct3=000) 164
A.4.3. R-Type 1-Op/2-Op (funct3=000, funct7=1111000) 165
A.4.4. R-Type 1-Op/2-Op (funct3=000, funct7=1111010) 165
A.4.5. AMO Sub-opcode Allocations (funct3=011) 166
A.4.6. MISC Sub-opcode Allocations (funct3=101) 166
Appendix B: Hybrid Mode Usage Models 168
B.1. High-Level Overview of Compatibility 168
B.2. Example Usage Models 168
B.3. CSRs and Registers Involved in Mode Transitions 171
B.4. CHERI Enable Bits in Setup (When Changing Privilege Levels) 171
B.5. Transition Instructions and Frequency 172
Appendix C: CHERI System Implications 173
C.1. Small CHERI System Example 174
C.2. Large CHERI System Example 175
C.3. Large CHERI Pure-Capability System Example 177
Bibliography 178

Preamble



This document is in the Stable state

Assume anything could still change, but limited change should be expected.

Copyright and License Information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2026 by RISC-V International.

Contributors

This RISC-V specification has been contributed to directly or indirectly by:

- Thomas Aird <thomas.aird@codasip.com>
- Hesham Almatary <hesham.almatary@cl.cam.ac.uk>
- Andres Amaya Garcia <andres.amaya@codasip.com>
- John Baldwin <jhb61@cl.cam.ac.uk>
- Paul Buxton <paul.buxton@codasip.com>
- David Chisnall <david.chisnall@cl.cam.ac.uk>
- Jessica Clarke <jessica.clarke@cl.cam.ac.uk>
- Brooks Davis <brooks.davis@sri.com>
- Lawrence Esswood <lesswood@google.com>
- Nathaniel Wesley Filardo <nwf20@cam.ac.uk>
- Franz A. Fuchs <franz.fuchs@cl.cam.ac.uk>
- Timothy Hutt <timothy.hutt@codasip.com>
- Alexandre Joannou <alexandre.joannou@cl.cam.ac.uk>
- Martin Kaiser <martin.kaiser@codasip.com>
- Tariq Kurd <tariq.kurd@codasip.com>
- Ben Laurie <benl@google.com>
- Marno van der Maas <mvdmaas@lowrisc.org>
- Maja Malenko <maja.malenko@codasip.com>
- A. Theodore Markettos <theo.markettos@cl.cam.ac.uk>
- Alfredo Mazzinghi <alfredo.mazzinghi@cl.cam.ac.uk>
- Jan Matyas <jan.matyas@codasip.com>
- Dave McEwan <dave.mcewan@codasip.com>
- David McKay <david.mckay@codasip.com>
- Jamie Melling <jamie.melling@codasip.com>
- Stuart Menefy <stuart.menefy@codasip.com>
- Simon W. Moore <simon.moore@cl.cam.ac.uk>
- Prashanth Mundkur <prashanth@riscv.org>
- Peter G. Neumann <neumann@csl.sri.com>
- Robert Norton <robert.norton@cl.cam.ac.uk>
- Alexander Richardson <alexrichardson@google.com>
- Robert Riglar <robert.riglar@codasip.com>
- Michael Roe <mr101@cam.ac.uk>
- Peter Rugg <peter.rugg@cl.cam.ac.uk>
- Florian Schmaus <florian.schmaus@codasip.com>
- Peter Sewell <peter.sewell@cl.cam.ac.uk>

- Carl Shaw <carl.shaw@codasip.com>
- Ricki Tura <ricki.tura@codasip.com>
- Robert N. M. Watson <robert.watson@cl.cam.ac.uk>
- Toby Wenman <toby.wenman@codasip.com>
- Jay Williams <jay.williams@codasip.com>
- Adrian Wise <adrian.wise@codasip.com>
- Jonathan Woodruff <jonathan.woodruff@cl.cam.ac.uk>
- Jason Zhijingcheng Yu <yu.zhi@comp.nus.edu.sg>

Chapter 1. Introduction

1.1. CHERI Concepts and Terminology

Current CPU architectures (including RISC-V) allow memory access solely by specifying and dereferencing a memory address stored as an integer value in a register or in memory. Any accidental or malicious action that modifies such an integer value can result in unrestricted access to the memory that it addresses. Unfortunately, this weak memory protection model is the root cause of the majority of security vulnerabilities in system software today.

CHERI provides strong hardware mechanisms that software uses to implement fine-grained memory protection and scalable software compartmentalization. These mechanisms enable software to prevent and mitigate security vulnerabilities.

Design goals include incremental adoption from current ISAs and software stacks, low performance overhead for memory protection, significant performance improvements for software compartmentalization, formal grounding, and programmer-friendly underpinnings. CHERI has been designed to provide strong, non-probabilistic protection rather than depending on short random numbers or truncated cryptographic hashes that can be leaked and reinjected, or that could be brute-forced.

1.2. CHERI for RISC-V

This specification is based on publicly available documentation including CHERI v9 (Watson et al., 2023) and CHERI Concentrate (Woodruff et al., 2019).

RISC-V CHERI introduces several new bases that are collectively referred to as RVY (e.g., RV32Y, RV64Y). Any CHERI RISC-V implementation must support one of these CHERI bases. CHERI will define profiles that provide equivalent functionality to the standard RVA, RVB, and RVM profiles. These RVY-based profiles (RVYA, RVYB, RVYM) will list the compatible extensions.



Most existing extensions are compatible with RVY with no modifications. Incompatible extensions will be listed in the profile documents.



Zicfiss is currently incompatible as the relevant push/pop instructions would need modifying for CHERI. An RVY version has not yet been developed.

This section lists new extensions available to RVY, and also extensions where the behavior is modified for RVY.

Almost all existing RISC-V extensions can be added to an implementation, but in most cases they will have some behavioral differences and/or new instructions operating on capabilities.

1.2.1. Extensions and Specifications

Table 1. Unprivileged RVY base ISAs and extensions

Extension or Specification	Description
RV64Y	CHERI Base ISA and capability-encoding format for RV64
Zylg	Local-global capability flow control
Zyhybrid	Hybrid extension for RVI compatibility
Zca (RVY added instructions)	New 16-bit encodings added to Zca

Extension or Specification	Description
Zba (RVY added instructions)	New 32-bit encodings added to Zba
Zalrsc (RVY added instructions)	New 32-bit encodings added to Zalrsc
Zaamo (RVY added instructions)	New 32-bit encodings added to Zaamo

Table 2. Standard base ISAs and extensions with differences under RVY

Base ISA or Extension	Differences
I	Control flow instructions and load/stores use capability values as address operands. Load/stores require capability operands and check permissions and bounds.
Zca	Compressed control-flow, load/store, and stack-pointer adjustment instructions operate on capabilities. Load/stores require capability operands and check permissions and bounds.
V	Vector memory access instructions require capability operands and check permissions and bounds.
Zicbom	Cache-block management operations require capability operands and check permissions and bounds.
Zicbop	Cache-block prefetch operations require capability operands and check permissions and bounds.
Zicboz	Cache-block zero operations require capability operands and check permissions and bounds.
Zicsr	Access to privileged and select unprivileged CSRs is constrained by ASR-permission.

RVY defines the privileged extensions listed in Table 3. Except for `Svyrg`, which is optional, each privileged extension is mandatory whenever the corresponding privilege mode or feature is implemented.

Table 3. Privileged extensions for RVY

Extension	Required When	Description
Smy	RVY	Machine-level architecture for RVY (widens trap handling and vector CSRs such as `mtvec` and `mepc` to capabilities and adds capability exception causes)
Ssy	RVY + S-mode	Supervisor-level architecture for RVY (widens supervisor trap and vector CSRs such as `stvec` and `sepc` to capabilities and adds capability exception causes)
Smyhybrid	Zyhybrid	Machine-level support for Zyhybrid (adds the `Y` enable bit to `menvcfg` and makes `misa.Y` writable)
Ssyhybrid	Zyhybrid + S-mode	Supervisor-level support for Zyhybrid (adds the `Y` enable bit to `senvcfg`)
Sv39/Sv48/Sv57 (RVY)	Sv39/Sv48/Sv57 respectively + Ssy	Page-based virtual memory support for RVY (Sv39, Sv48, and Sv57 PTE formats reserve bits to allow/deny capability stores)
Svyrg	Optional (requires Sv39)	MMU-based capability data flow control and measurement

Chapters for the Unprivileged Specification

Chapter 2. RVY Base Capability Instruction Set, Version 1.0

This chapter introduces a new set of base ISAs called RVY that extend the RV32[IE]/RV64[IE] base ISAs with CHERI.

The RVY base ISA will also be compatible with other standard bases, such as a Zfinx base.

2.1. CHERI Overview

CHERI enhances the base ISA by adding hardware memory access control. It has an additional memory access mechanism that protects *references to code and data* (pointers), rather than the *location of code and data* (integer addresses). This mechanism is implemented by providing a new primitive, called a **capability**, that software components can use to implement strongly protected pointers within an address space. Capabilities are unforgeable and delegatable tokens of authority that grant software the ability to perform a specific set of operations. In CHERI, integer-based pointers are replaced by capabilities to provide memory access control.

Every RVY base ISA requires a *capability-encoding format* to complete the specification, such as RV64LYA. The *capability-encoding format* gives full details of how CHERI capabilities are represented in memory and in registers, and specifies the exact properties such as bounds precision and available combinations of permissions.

For example, in RV64LYA:

1. `XLEN=64`
2. `L` denotes a long base name.
3. When `I` or `E` is omitted, the default is 32 integer registers (`I`); `E` specifies 16 integer registers (e.g., `RV64LEYA`).
4. `Y` denotes a CHERI base.
5. `A` denotes the capability-encoding format; the capability-encoding format after `Y` is mandatory.



This naming scheme is based on the RISC-V "longer base name" proposal, which allows base ISA names longer than two characters. Making the capability-encoding format after `Y` mandatory simplifies future long base naming.



Future RV64 CHERI capability-encoding formats could be named RV64LYB, RV64LYC, and so on, or they may have more descriptive names after the RV64LY prefix.

Also see capability-encoding format.

2.2. CHERI Protection Model

The CHERI model is motivated by the *principle of least privilege*, which argues that greater security can be obtained by minimizing the privileges accessible to running software. A second guiding principle is the *principle of intentional use*, which argues that, where many privileges are available to a piece of software, the privilege to use should be explicitly named rather than implicitly selected. While CHERI does not prevent the expression of vulnerable software designs, it provides strong vulnerability mitigation: attackers have a more limited vocabulary for attacks, and should a vulnerability be successfully exploited, they gain fewer rights, and have reduced access to further attack surfaces.

Protection properties for capabilities include the ISA ensuring that capabilities are always derived via valid

manipulations of other capabilities (*provenance*), that corrupted[1] in-memory capabilities cannot be dereferenced (*integrity*), and that derived capabilities must not grant permissions or bounds that exceed those of their source capabilities (*monotonicity*). Tampering or modifying capabilities in an attempt to elevate their rights will yield an invalid capability. Attempting to dereference via an invalid capability will raise an exception.

[1] Not all possible corrupted states are detected, see Section 2.5, "Integrity of Capabilities".

CHERI capabilities may be held in registers or memory, and are loaded, stored, and dereferenced using CHERI-aware instructions that expect capability operands rather than integer addresses. Upon initialization, the execution environment provides initial capabilities to software via general-purpose registers. All other capabilities will be derived from these initial valid capabilities through valid capability transformations.

Developers can use CHERI to build fine-grained spatial and temporal memory protection into their system software and applications and significantly improve their security.

2.3. Capability Registers and Format

RVY widens all registers that can hold addresses (including the general-purpose registers, pc, and address-holding CSRs) from `XLEN` to `2*XLEN` bits (hereafter referred to as YLEN), adding metadata to protect their integrity, limit how they are manipulated, and control their use. In addition to widening to YLEN, each register also gains a one-bit capability tag which is defined below.

RVY specifies the minimum required fields that the capability-encoding format must support, and their semantics. This version of RVY defines only little-endian capability memory layouts. The memory representation of capabilities is always `2*XLEN` (`YLEN`) bits wide and holds the address in the lower `XLEN` bits of memory and the metadata in the upper `XLEN` bits.

RVY is designed to be highly extensible by allowing different capability-encoding formats to exist. Specific capability-encoding formats may support different ISA extensions and add new architectural permissions.

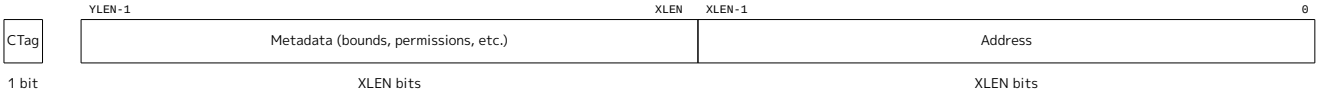


Figure 1. CHERI Capability structure

2.3.1. Address

The lower XLEN bits of a capability encode the address to which the capability points. This is also referred to as the integer part of the capability. When an instruction writes non-capability data to an extended register, the metadata fields and capability tag of the destination are set to 0 unless that instruction's definition explicitly states otherwise.

2.3.2. Capability Tag

The capability tag is a single additional bit associated with YLEN-bit memory locations and YLEN-bit registers. It is stored separately (i.e., is *out-of-band* or *hidden*), and is *hardware-managed*. It indicates whether a YLEN-bit register or naturally aligned YLEN-bit memory location contains a valid capability. If the capability tag is set to 1, the capability is valid and may authorize operations, subject to type, permission, bounds, and integrity checks.

All registers or memory locations able to hold a valid capability are YLEN bits wide and have an associated

capability tag.

Not all memory in a real system will support capability tags. The Execution Environment Interface (EEI) will define what regions of the address space support them.

The capability tag cannot be directly set by software; it is *not* a conventionally accessible bit of state. If the capability tag is set to 1 then it shows that the capability has been derived correctly according to the principles listed above (*provenance*, *integrity*, *monotonicity*). If the rules are followed then the capability tag will propagate through the instructions that derive, load, or store the capability value.

Therefore, whenever an instruction writes a capability with the capability tag set to 1 to a register or to memory:

- At least one capability tag must have been set to 1 in the input operands.
  - This is the *provenance* check.
- The requested operation must have been legal and not increase bounds or permissions.
  - This is the *monotonicity* check.
- The instruction may have checked that none of the input capabilities were corrupted.
  - This is the *integrity* check. Not all instructions check all input operands for *integrity*.

For capability loads and stores, provenance is preserved across memory:

- Storing a capability to memory preserves the capability tag only if the store is performed using a valid capability that authorizes storing capabilities to that memory location by checking that every stored byte is within capability bounds, and that suitable permissions are granted to write to memory.
- Loading a capability from memory preserves the capability tag in the destination register only if the load is performed using a valid capability that authorizes loading capabilities from that memory location by checking that every byte is within capability bounds, and that suitable permissions are granted to read from memory.

When a memory access fails a check, either due to software error or malicious intent, the operation raises an exception. For other instructions a failed check will typically set the resulting capability tag to zero, the exact behavior is defined by each instruction.

- Valid capabilities have the capability tag set to 1.
- Invalid capabilities have the capability tag set to 0.
  - The term *invalidating a capability* means deriving a new capability with identical metadata and address but the capability tag set to 0.

Using an invalid capability to dereference memory or authorize any operation raises an exception. All capabilities derived from invalid capabilities are themselves invalid.



Writing non-capability data into a register or memory location causes the capability tag to be set to 0. When the capability tag associated with the memory location is zero, the location contains non-capability data.

2.3.3. Capability tags in Registers

Every YLEN-bit register has a one-bit capability tag, indicating whether the capability in the register is valid to be dereferenced. This capability tag is set to 0 whenever an invalid capability operation is performed.

Examples of such invalid operations include writing only the integer portion (the address field) of the register or attempting to increase bounds or permissions.

2.3.4. Capability tags in Memory

Capability tags are tracked through the memory subsystem: every naturally aligned YLEN-bit memory region has a non-addressable one-bit capability tag, which the hardware manages atomically with the data. The implementation must set the capability tag to zero if any byte in the naturally aligned YLEN-bit memory region is ever written using an operation other than the successful store of a capability operand.

For example, the SY instruction sets the capability tag to one if all the instruction's prerequisites are met, including `rs2` being a valid capability, but SW will always set the stored capability tag to zero.

A failed SC will leave the capability tag in memory unmodified.

All system memory and caches that store capabilities must preserve this abstraction, handling the capability tags atomically with the data.

2.3.5. Capability Bounds

Capabilities encode memory bounds as a half-open byte range `[base, top)`, which is the range that may be accessed when the capability authorizes data memory access or instruction execution.

Checking is on a byte-by-byte basis, so that it is possible for a memory access to be fully in-bounds, partially out-of-bounds, or fully out-of-bounds.

A partially or fully out-of-bounds data access or instruction fetch raises an exception.

Every capability has two memory address bounds: *base* representing the lowest accessible byte, and *top* representing one byte above the highest accessible byte.

- The *base* is XLEN bits and is *inclusive*.
- The *top* is (XLEN+1) bits and is *exclusive*.

Inclusive top, with XLEN bits, was considered but rejected in favor of the exclusive top (with an additional bit to allow covering the entire address space). Using exclusive top allows representing a zero-length capability starting at address zero, which is not possible with inclusive top unless you introduce a special value for `-1`.

- The *length* is (XLEN+1) bits and is defined to be *top - base*.

Therefore, a memory location `A` in the range `base ≤ A < top` is within bounds, and so valid to access.

Checking every byte of every executed instruction and every byte of every data memory access is fundamental to the memory safety which CHERI provides. In a typical load/store unit, the expansion of the bounds from `rs1` and bounds checking is in parallel with the address calculation, the memory translation, and/or the PMA/PMP checking.

A compressed format is used to encode the bounds with a scheme similar to floating-point using an exponent and a mantissa. Therefore, small exponents can allow byte granularity on the bounds, but larger exponents give coarser granularity. The standard bounds encoding format for RV64LYA is based upon the compressed bounds encoding scheme (Woodruff et al., 2019) and is defined in Section 3.1.5.

Future CHERI bases may use capability-encoding formats with different bounds encoding

schemes (see capability-encoding format).

Software can query the bounds of a capability held in a general-purpose register using the following instructions:

- The *base* is returned by the YBASER instruction.
- The *length* is returned by the YLENR instruction.
- The *top* is returned by the YTOPR instruction.



The length and top are (XLEN+1)-bit values, so YLENR and YTOPR saturate their result to the maximum XLEN-bit value (2^{XLEN}-1) when it overflows an XLEN-bit register.

Infinite bounds

Capabilities which allow access to every byte in memory have *infinite bounds*.

2.3.6. Deriving New Bounds

Upon initialization, the execution environment defines how capabilities are made available to the execution context. All capabilities with smaller bounds are derived from these, or from capabilities in memory accessible by these.

The bounds are reduced using the YBNDSW, YBNDSWI, and YBNDSRW instructions.

These bounds-reducing instructions all copy `rs1` to the output `rd`, set the *base* bound address of `rd` to `rs1.address`, subject to encoding granularity constraints, and set the *length* of `rd` from `rs2` or the immediate field depending on the instruction.

- YBNDSW sets the *base* to `rs1.address`, and the *length* to `rs2`.
  - The capability tag is set to 0 if the bounds cannot be encoded exactly.
- YBNDSWI sets the *base* to `rs1.address`, and the *length* to the immediate value.
  - The capability tag is set to 0 if the bounds cannot be encoded exactly.
- YBNDSRW sets the *base* to `rs1.address`, and the *length* to `rs2`.
  - If the requested bounds cannot be encoded exactly, the result uses the smallest representable bounds that contain the requested range and remain within the initial bounds.
- YAMASK can be used to calculate the nearest precisely encodable *length* and *base* values for a given size.

The bounds are encoded relative to the address field, using some upper bits of the address combined with bits from the capability metadata. The exact method for encoding the bounds varies by capability-encoding format. See Section 3.1.5 for the method used for RV64LYA.

2.3.7. Representability and Updating the Address

Software sometimes uses out-of-bounds pointers, for example, to represent a one-past-the-end pointer to an array in C (`arr + size`). To support this pattern, CHERI must allow these out-of-bounds pointers to be held in capabilities while still protecting the bounds and integrity. Because the standard bounds encoding scheme uses upper bits of the address for the computation of the bounds, only a limited range of out-of-bounds pointers can be represented for any given set of bounds. This range is known as the **representable**

range.

Any address that is within the capability's bounds is always representable and preserves the capability tag. The relationship between the bounds and the representable range is illustrated in Figure 2.



The C standard permits forming a one-past-the-end pointer (but no more than one), as long as it is not dereferenced. Real-world software has been observed to temporarily create pointers that point multiple bytes past the end or even before the start. To maximize software compatibility, the bounds representation was designed to allow for out-of-bounds pointers.

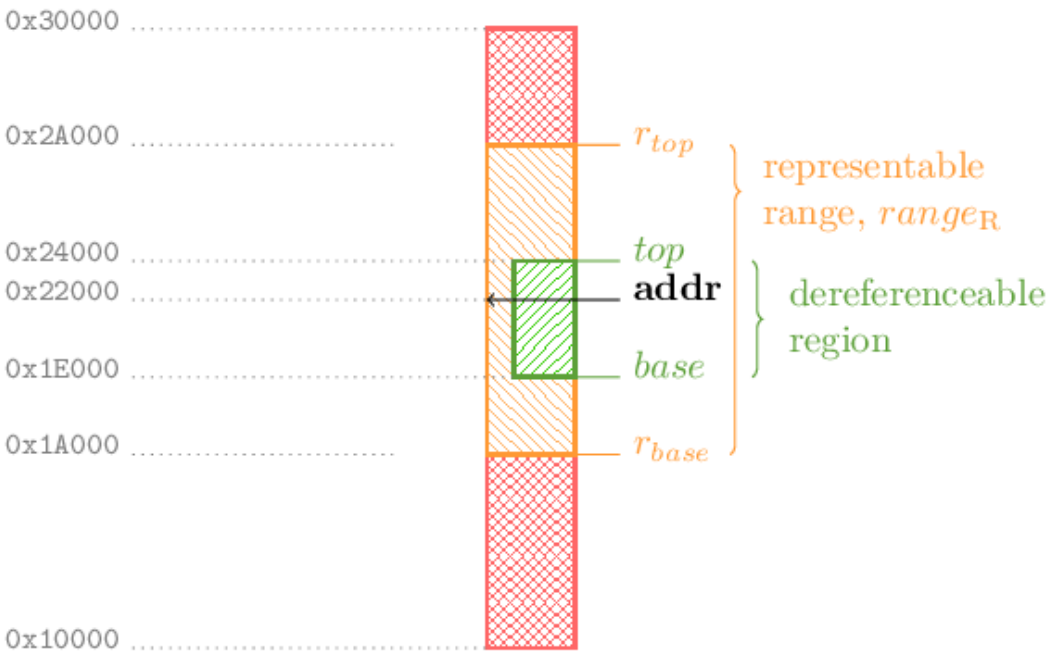


Figure 2. Memory address bounds and representable range encoded within a capability



The historic, uncompressed 64-bit CHERI-MIPS CPU had a 256-bit capability-encoding format with separate 64-bit values for the base and top memory bounds, and another for metadata fields. This scheme could represent all out-of-bounds pointers with a very high hardware cost.

Since the upper bits of the address are used to calculate the top and base bound addresses, deriving a new capability with a different address could change the resulting bounds. All instructions that derive a capability with a new address (such as YADDRW, YADD, and YADDI) must check that the new address does not change the decoding of the bounds. If an address calculation moves the address outside the representable range, the hardware detects that the bounds can no longer be represented correctly, and sets the capability tag to zero so that the derived capability becomes invalid.



RVY implementations that use a different capability-encoding scheme to RV64LYA (e.g., for accelerators or specific micro-controllers) may specify an alternative to the representable range check. Such implementations must still support one-past-the-end pointers as required by the C standard, but are not required to support a representable range larger than that. Therefore, the ISA specification uses the phrase **represented exactly** *for this check.*

2.3.8. Memory Space

A hart supporting RVY has a single byte-addressable address space of 2^{XLEN} bytes for all memory accesses. Each memory region capable of holding a capability also stores a capability tag for each naturally aligned YLEN bits (e.g., 16 bytes in RV64), so that capabilities with their capability tag set to 1 can only be stored in naturally aligned addresses. Capability tags must be atomically bound to the data they protect.

Address arithmetic is modulo 2^{XLEN}, so the byte at address 2^{XLEN} - 1 is adjacent to the byte at address zero. A capability's Representable Range is also circular, so address 0 is within the Representable Range of a capability where address 2^{XLEN} - 1 is within the bounds.

However, the decoded top bound address of a capability is XLEN + 1 bits wide and does **not** wrap. For example, a hypothetical capability with base 2^{XLEN} - 1 and top 2^{XLEN} + 1 authorizes access to the last byte of the address space only, and does not authorize access to the byte at address 0. A capability with architectural top > 2^{XLEN} cannot be generated by any valid capability derivation and is only possible due to a logic or memory fault.

Unlike malformed bounds, an architectural top > 2^{XLEN} is not special-cased in bounds decoding to return zero base and top; normal bounds check rules apply.

2.3.9. Capability Type (CT)

This metadata value indicates the type of the capability and determines which operations the capability authorizes. For example, the CT-field can be used to *seal* capabilities against modification, to provide control flow integrity, and to provide immutable software tokens.

Unsealed capabilities

When `CT=0`, the capability authorizes access to a region of memory as defined by the permissions and bounds.

Sealed capabilities

Capabilities with `CT≠0` are sealed against modification and cannot be dereferenced to access memory. Instructions that operate on capabilities will produce an invalid capability if the source capability is sealed and the operation would result in altered address, bounds, or permissions.

Sealing

Given a capability with `CT=0`, deriving a capability with `CT≠0` is termed *sealing*.
This is achieved using YSEALx instructions.

- The YSEALE instruction copies `rs1` to `rd` and seals it as a Sealed entry point capability.



Future extensions will add more YSEAL`x` instructions such as seal by type.

Unsealing

Given a capability with `CT≠0`, deriving a capability with `CT=0` is termed *unsealing*.
This is achieved using YUNSEALx instructions.

- The YUNSEALS instruction authorizes the unsealing of `rs2` into `rd` with a superset capability in `rs1` (the S suffix represents superset). NOTE: Future extensions will add more YUNSEAL`x` instructions such as unseal by type.

Sealed entry point capability

Sealed entry point capabilities provide **immutable** function pointers within a CHERI software system.



Such function pointers are sometimes known as "sentries", a contraction of "sealed entries".

Sentry capabilities can establish a form of control-flow integrity between mutually distrusting code. Because they are immutable, the jump target cannot be modified, ensuring that control flow can only enter at the specific address encoded in the capability and not elsewhere in a function.

A sealed entry point capability may be passed as the `rs1` argument to JALR (RVY); an unsealed version of

this capability value is written to the pc. If `rd` is not `x0`, the link capability written to `rd` is written as a Sealed entry point capability.



In addition to using sealed capabilities for sealed entry points, sealed capabilities can also be useful to software as secure software tokens. YUNSEALS can be used to convert such a token back to an unsealed capability.

The standard capability type mapping is shown in Table 4, extensions and capability-encoding formats may add more types.

Table 4. Standard capability type mapping

Type Name	Integer Value	Description
Unsealed	0	*Unsealed* capability granting access to memory
Sealed entry point capability	1	*Sealed* capability that can be used as a function entry point or function return.
All other values	Reserved	Reserved for future standard extensions or capability-encoding formats.



Future extensions and capability-encoding formats may declare new CT-field values, examples of which are:

- *Sealed types which cannot be used as a Sealed entry point capability*
- *Sealed entry point capability types which can be used only for function calls or only for function returns for improved CFI: specific CT-field values are required for each, in contrast with the current Sealed entry point capability which can be used for both.*

2.3.10. Architectural Permissions (AP)

This metadata field encodes architecturally defined permissions of the capability. Permissions grant access subject to the capability being valid, the capability being unsealed, and bounds checks passing.



Any memory operation is also contingent on requirements imposed by other RISC-V architectural features, such as virtual memory, PMP, and PMAs, even if the capability grants sufficient permissions.

The permissions defined in RVY are listed in Table 5. All capability-encoding formats must support at least this set of permissions. Permissions can be cleared when deriving a new capability value (using YPERMC) but they can never be added.

Table 5. AP-field summary

Permission	Type	Comment
R-permission	Data memory permission	Authorize data memory read access
W-permission	Data memory permission	Authorize data memory write access
X-permission	Instruction memory permission	Authorize instruction memory execute access
C-permission	Data memory permission	Authorize loading/storing of capability tags
LM-permission	Data memory permission	Used to restrict the permissions of loaded capabilities.

Permission	Type	Comment
ASR-permission	Privileged state permission	Authorize privileged instructions and CSR accesses.

Extensions building on top of RVY may define additional permissions.



An example of such an extension is Zylg.

Read Permission (R)

Allow reading data from memory.

Write Permission (W)

Allow writing data to memory.

Execute Permission (X)

Allow instruction execution.

Capability Permission (C)

Allow reading capability tags from memory if R-permission is also granted.

Allow writing capability tags to memory if W-permission is also granted.

If C-permission is missing then the capability tags for capability loads and stores are read and written as zero.

Load Mutable Permission (LM)

Allow preserving the W-permission of capabilities loaded from memory. If a capability grants R-permission and C-permission, but no LM-permission, then a capability loaded via this authorizing capability will have W-permission and LM-permission removed.

The permission stripping behavior *only* applies to valid loaded capabilities and that are not sealed after all other checks. This ensures that capability loads of non-capability data do not modify the loaded value, and that sealed capabilities are not modified.



Clearing a capability's LM-permission and W-permission allows sharing a read-only version of a data structure without making a copy. For example, if software removes LM-permission from a capability to a linked list, any pointers loaded through that capability will automatically lose their W-permission, preventing modification of the list content.

Access System Registers Permission (ASR)

Authorize the execution of privileged instructions and access to CSRs. This permission only has an effect when the capability is installed in the pc. ASR-permission acts as an additional constraint and does not bypass normal privilege mode checks: having ASR-permission in the pc does not grant an unprivileged mode access to privileged CSRs. Lacking ASR-permission in the pc prevents access to privileged CSRs and restricted instructions even if the current privilege mode would otherwise permit it.

Only the unprivileged CSRs listed below require ASR-permission; other unprivileged CSRs remain accessible as usual.

In the unprivileged RVY ISA, the following are affected by ASR-permission:

- **Thread ID CSRs:** Writing to utidc requires ASR-permission in the pc, but reading does not require ASR-permission.

- **Instructions:** CBO.INVAL (RVY) requires ASR-permission in the pc.

In the privileged RVY ISA, the following are affected by ASR-permission:

- **Instructions:** MRET (RVY), SRET (RVY) require ASR-permission in the pc.
- **CSRs:** The privileged `xtidc` CSRs require ASR-permission for writing, but not for reading (if access is permitted in the current privilege mode). All other privileged CSRs require ASR-permission in the pc for any access.

This permission is important in privileged execution environments. Removing this permission allows constraining privileged software to a sandbox that cannot be subverted by changing privileged state.

Extensions may add additional unprivileged CSRs that require ASR-permission.

2.3.10.1. Permission Transitions

Not all capability permissions are orthogonal as some permissions inherently depend on others. As such, using YPERMC to clear some permissions may have the effect of clearing others as well.

For the base set of permissions just defined, the rules in Table 6 apply.

Table 6. YPERMC base rules

YPERMC Rule	Permission	Valid only if
base-1	C-permission	R-permission or W-permission
base-2	LM-permission	C-permission and R-permission
base-3	ASR-permission	X-permission

These rules enforce sensible permission combinations.

The rules are referenced by name and extended by RV32Y capability-encoding formats which cannot represent all combinations of permissions.

Extensions that define new permission bits may also introduce new dependency constraints. Currently defined examples are:

1. Zyhybrid, see Table 51
2. Zylg, see Section 13.3.1

Capability-encoding formats may impose additional constraints to reduce the number of bits necessary to represent permissions.

2.3.11. Software-Defined Permissions (SDP)

The metadata also contains an encoding-dependent number of software-defined permission (SDP) bits. Software-defined permissions can be inspected by the operating system or application programs to enforce restrictions on API calls (e.g., permit/deny system calls, memory allocation, etc.). These permissions can be cleared by YPERMC but are not interpreted by the CPU otherwise.

While these bits are not used by the hardware as architectural permissions, modification follows the same rules: SDP bits can only be cleared and never set on valid capabilities.

This property is required to ensure restricted programs cannot forge capabilities that would pass the software-enforced checks.

Software is completely free to define the usage of these bits.

2.3.12. Special Capabilities

2.3.12.1. Root Capabilities

Root capabilities are the set of capability encodings for the maximum set of bounds, covering the whole 2^{XLEN} address space, and that grant all encodable permissions.

Root Executable Capability

An unsealed capability that has infinite bounds and grants at least all of X-permission, R-permission, C-permission, LM-permission, and ASR-permission.

Root Data Capability

An unsealed capability that has infinite bounds and grants at least all of R-permission, W-permission, C-permission, and LM-permission.

Extensions introducing new permissions may require these to be provided by Root Executable and/or Root Data capabilities.

Upon initialization, the execution environment defines how the initial set of capabilities are made available to software.

The definitions of Root Executable and Root Data are the maximum possible encodable values.

Implementations are not required to provide access to the entire address space to a single hart or even to provide all permissions. As such, where a root capability value is specified, harts may specify implementation-defined values instead with reduced bounds and/or permissions as required.

The hart-specific initial capabilities may be the maximum that the hart can ever have access to, but there could also be capabilities in memory with greater bounds or permissions.

Some capability-encoding formats like RV64LYA provide an Infinite capability which provides all the characteristics of the full set of root capabilities in a single encoding.

Future extensions may add additional root capabilities for purposes such as sealing and unsealing.

2.3.12.2. NULL Capability

An invalid capability with all-zero metadata and address is referred to as the NULL capability. This capability grants no permissions and any dereference raises an exception.

2.4. Capability-Encoding Formats

CHERI implementations make trade-offs in their "capability-encoding formats", such as the *precision* of bounds and the *expressible set* of combinations of permissions, that impact certain behaviors of the ISA. The definition of all CHERI base ISAs includes sufficient details of a capability-encoding format to precisely define the execution behavior of all base ISA instructions.

Whilst RV64Y has enough bits in the encoding format to prevent such trade-offs in general, the concepts are introduced here and RV32Y formats will make extensive use of them.

The name RVY refers generically to CHERI base ISAs, which share common semantics but may have different metadata encodings. The RVY name on its own does not describe the complete base ISAs, but defines all the *semantics* of a RVY machine, where certain behavioral parameters must be filled in by the capability-encoding format. For example, the YBNDSRW instruction yields a capability with new bounds that are rounded differently depending on the number of bits allocated to the bounds in the capability-encoding format. Systems with XLEN=64 can allocate more bits towards the mantissa of the bounds than XLEN=32 system and therefore have a larger Representable Range.



These observable differences in behavior are similar to floating-point numbers where the supported operations and their semantics are the same for all formats, but the exact result depends on the representation (e.g., 16/32/64-bit IEEE-754 or other floating-point formats).

It is possible to generate code for RVY that is compatible with all valid capability-encoding formats of the same address width. However, to allow compilers and/or software library authors to make assumptions about encoding-dependent properties such as bounds precisions, each capability-encoding format comes with a set of standard parameters.

Each new CHERI base ISA defines a capability-encoding format.

The documentation for each capability-encoding format specifies at least the parameters in Table 7 below where the default is not used.



The default settings all match RV64LYA.

Table 7. RVY capability-encoding format parameters

Parameter	Legal values	Default Value	Comment
Parameters used for the bounds encoding (see Section 3.1.5)			
MW	integer values	14	Mantissa width for the bounds encoding
EW	integer values	6	Exponent width for the bounds encoding
CAP_MAX_E	integer values	52	Maximum exponent value
enableL8	0/1	0	Whether the encoding format includes the L_8 bit
REP_RANGE	rc1/r0	rc1	Whether out-of-bounds addresses are representable
Parameters used for permissions encoding in the AP-field			
AP_MAX	integer values	all ones	Value of the AP-field giving maximum permissions
AP_ENC	full<suffix>/ part<suffix>	full1	Encoding scheme for the AP-field

Table 8. RVY base ISA parameters

Parameter	Legal values	Default Value	Comment
AUIPC_SHIFT	integer values up to 12	12	Shift distance of the immediate in AUIPC

Two parameters use non-numeric values:

AP_ENC
: Whether permissions in the AP-field are encoded using a *full* (`full`) representation allowing all valid combinations of permissions, or a *partial* (`part`) representation which does not represent all possible combinations and so uses fewer encoding bits.

REP_RANGE

Whether the capability-encoding format guarantees representability of out-of-bounds capability values: `rc1`, where the encoding uses one additional bit and a *centered* out-of-bounds region or `r0`, where no bits are used for out-of-bounds capabilities and there is no guaranteed representable range for out-of-bounds addresses.



For RV64Y, the number of spare bits in the capability-encoding format ensures sufficient future extensibility and therefore few custom formats are expected to exist. RV32Y, however, has insufficient bits available to enable features for all target domains, so multiple RV32Y-extending capability-encoding formats will exist.

2.5. Integrity of Capabilities

CHERI defines the following integrity check rules:

1. The bounds are not malformed (a capability-encoding-format-specific check; see Section 3.1.5.6 for RV64LYA).
2. The capability metadata does not have any bits, or fields, set to a value that is not valid for the implemented extensions.



An example of a value that is not valid for RVY is an illegal set of permissions such as ASR-permission without X-permission. Future extensions may allocate values that are reserved in RVY; capabilities using those values will fail integrity checks on harts that do not implement that extension.

If any of the above rules do not hold for a valid capability, it indicates either:

- State corruption due to memory errors or logic glitches, or
- The presence of incompatible or defective CHERI IP within the system.



These checks are much less rigorous than parity or ECC protection, and are only used to detect simple problems with the capability metadata.



While a capability whose decoded bounds exceed 2^{XLEN} also represents a loss of integrity resulting from state corruption or incompatible IP, detecting this condition requires full bounds decoding and is not included in the capability integrity checks due to implementation cost. As described in Section 2.3.8, hardware instead relies on fail-closed bounds checking and YBLD subset enforcement. Detecting such conditions is deferred to future reliability extensions.

Valid capabilities are guaranteed to satisfy all integrity rules unless state corruption or incompatible IP is present, as valid capability derivation preserves integrity. Integrity checks on operands that are expected to be valid are therefore *optional*: performing an integrity check on a valid capability never alters execution on a correctly functioning system. However, instructions that can produce a valid capability from an invalid input capability *must* perform an integrity check on that input, since an invalid capability can hold arbitrary bit patterns. Therefore, integrity checks are *optional* except for instructions that are designed to operate on potentially invalid capabilities.



In the RVY ISA, this mandatory integrity check only applies to the `rs2` input to YBLD.



Implementations may choose to enforce optional integrity checks for a number of reasons:

1. *Instruction-level verification is simpler as deterministic behavior is specified for all $2^{(YLEN+1)}$ input values (similar to how the D extension defines deterministic behavior for improperly*

NaN-boxed register values).

2. *To give more resilience in systems which mix CHERI IPs from different sources due to possible incompatibilities.*
3. *If the hart implements additional reliability features, it may wish to detect and report capabilities which lack integrity.*

Table 9 summarizes the possible states of a capability register or memory location, including the corrupted states a valid capability can only reach through memory errors, logic glitches, or incompatible IP.

Table 9. Summary of capability states

State	CTag	Legal?	Description
Valid capability	1	Yes	Passes all integrity checks and has `top` $\leq 2^{XLEN}$.
Invalid capability	0	Yes	Holds an arbitrary YLEN-bit payload and cannot authorize operations unless used as the `rs2` input to YBLD.
Corrupted valid capability (integrity failure)	1	No	Metadata has a value that is not valid for the implemented extensions or malformed bounds, which is detected by optional integrity checks.
Corrupted valid capability (`top` > 2^{XLEN})	1	No	Not detected by integrity checks, as decoded `top` > 2^{XLEN} is safely handled by existing bounds-checking logic.

2.6. Extended State

As stated above, unless noted otherwise, state which can hold addresses is extended from XLEN to YLEN bits.



The privileged `mconfigptr` *CSR is the only documented exception to the rule; see [mconfigptr] for details.*



A reminder that all YLEN state also has a hardware-managed capability tag.

2.6.1. General-Purpose Registers

The XLEN-bit integer registers (e.g., `x1`, `x2`) are all widened to YLEN bits, each gaining an associated capability tag, as shown in Figure 3.

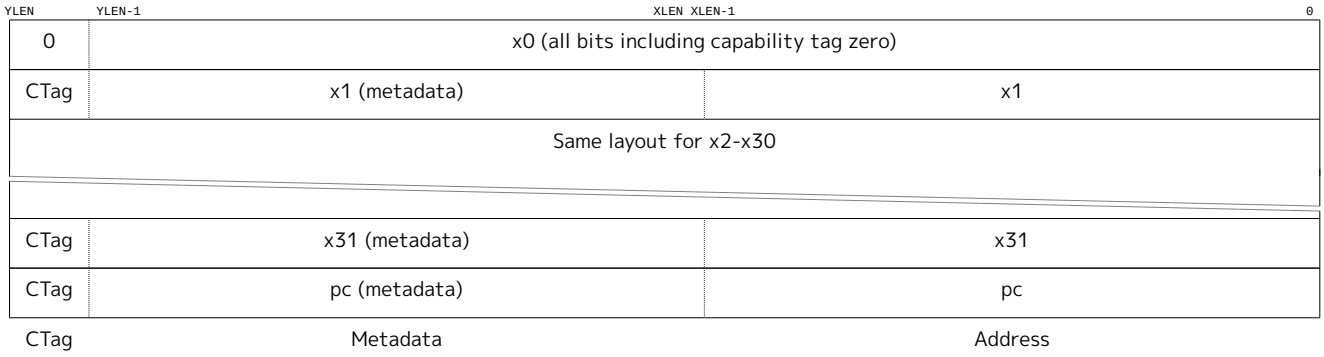


Figure 3. Extended registers in RVY

The zero register (`x0`) is extended with zero metadata and its capability tag set to 0: this is called the NULL capability.

2.6.2. The Program Counter Capability (pc)

The `pc` is widened to a capability. Widening the `pc` allows the range of branches, jumps, and linear execution for currently executing code to be restricted. The pc address field is the `pc` in the base RISC-V ISA so that the hardware automatically updates it as instructions are executed.

The hardware performs the following checks on pc for each instruction executed in addition to the checks already required by the base RISC-V ISA. A failing check raises a CHERI exception.

- The capability must be valid.
- The capability must not be sealed.
- The capability must grant X-permission.
- All bytes of the instruction must be in bounds.
- All integrity checks must have passed.

Upon initialization, the execution environment must set `pc` bounds and permissions such that the program can run successfully (e.g., by setting `pc` to a Root Executable capability to ensure *all* instructions are in bounds).



Future ISA extensions should respect these rules so that the checked bits do not need to be stored in all copies of the pc in the implementation.

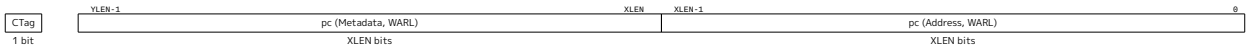


Figure 4. Program Counter Capability

2.6.3. Added CSRs

RVY adds the YLEN-bit CSR shown in Table 10.

Table 10. Unprivileged YLEN-bit CSRs added in RVY

YLEN-bit CSR	Permissions	Description
utidc	RW, ASR-permission required for writes, not reads	User Thread ID Capability

2.6.3.1. User Thread Identifier Capability (utidc)

The utidc register is an unprivileged YLEN-bit CSR used to identify the current software thread. Any operation that modifies utidc raises an exception unless the ASR-permission is set in the current pc.

The EEI sets the value of the capability tag of this CSR to zero, and the values of the metadata and address fields are UNSPECIFIED.

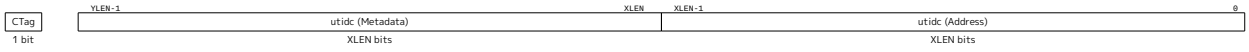


Figure 5. User thread identifier capability register



Each thread (as scheduled by the kernel) can have its own context stored in privileged data structures managed by the compartmentalization TCB, for example a stack of user-level compartment entries and exits. utidc allows compartment-switching code to determine the currently running thread without a call into the operating system. This is read-only without ASR-permission to prevent a malicious compartment from tricking the switching mechanism into accessing data corresponding to the wrong thread. While the RISC-V ABI includes a thread pointer (tp) register, it is not usable for the purpose of reliably identifying the current

software thread because the tp register is a general-purpose register and can be changed arbitrarily by untrusted code. Making utidc a capability allows a data structure to be shared (guarded by e.g., the subset-sealing mechanism), which allows for example efficient recovery of a trusted stack, without requiring additional indirection.

2.6.4. CSR Classes

Most CSRs that can hold pointers are widened to YLEN bits.

RVY has two classes of CSRs:

XLEN-bit CSRs
: These do not contain pointers (e.g., *fcsr* from the "F" extension).

YLEN-bit CSRs
: These hold capabilities. They are either CSRs that can hold pointers, widened from XLEN to YLEN bits (e.g., *xtvec* and *xepc* from the privileged specification), or CSRs added by RVY or its extensions (e.g., utidc).

  When a WARL CSR is widened the metadata and capability tag also become WARL. If the final written capability value cannot be written unmodified to the CSR due to WARL behavior then the capability value is considered to be *invalid* and so must be written with its capability tag set to 0.



Examples of WARL behavior for metadata are implementations requiring all fields to be in a valid (non-reserved) state, and also to fix specific permissions to certain values.

When accessing CSRs these rules are followed:

1. Accesses to XLEN-bit CSRs are as specified by Zicsr.
2. Accesses to YLEN-bit CSRs using CSRRW will:
   a. Read YLEN bits
   b. Write YLEN bits, and will invalidate the written capability if it fails any integrity checks or cannot be written unmodified due to WARL fields.
3. Accesses to YLEN-bit CSRs using instructions other than CSRRW will:
   a. Read YLEN bits.
   b. For CSRRS/CSRRC, compute the XLEN-bit address field value using the standard Zicsr bitwise set or clear operation on the old CSR address field and the XLEN-bit source operand.
   c. For immediate forms, compute the XLEN-bit address field value using the standard Zicsr operation with the zero-extended `uimm` operand.
   d. Write the computed XLEN-bit value to the address field, and use the semantics of the YADDRW instruction to determine the final written value.



YLEN-bit CSRs are summarized in Table 42, Table 58, and Table 61.



Any YLEN-bit CSRs may have additional rules defined to determine the final written value of the metadata and/or to write zero to the capability tag. These rules are shown in Table 43, Table 59, and Table 62.

The assembler pseudoinstruction to read a YLEN-bit CSR `csrr rd, csr`, is encoded as `csrrs rd, csr, x0`.

Table 11. YLEN-bit CSR access summary for RVY

Instruction	Read Width	Write Width
CSRRW rd==x0		YLEN
CSRRW rd!=x0	YLEN	YLEN
CSRR[C|S] rs1==x0	YLEN	
CSRR[C|S] rs1!=x0	YLEN	XLEN
CSRRWI rd==x0		XLEN
CSRRWI rd!=x0	YLEN	XLEN
CSRR[C|S]I uimm==0	YLEN	
CSRR[C|S]I uimm!=0	YLEN	XLEN

In Table 11, when there is no read or write width shown, the CSR access is *not* made and there are no side-effects following standard Zicsr rules.

2.7. Capability Checks

Every memory access must be authorized by a valid capability.

Instruction fetches and data memory accesses raise a CHERI exception if any of the following checks fail on the authorizing capability:

- The capability tag must be set to 1.
  - All memory access instruction encodings with `rs1=x0` are *reserved* (since dereferencing NULL in `x0` always faults due to its capability tag being zero).
- The authorizing capability must not be sealed.
- All integrity checks on the authorizing capability must pass.
- All bytes of the access must be in bounds.
- The virtual address in the authorizing capability must not be invalid if virtual memory is enabled.
- The authorizing capability must grant the required permissions:
  - All instruction fetches require X-permission in pc.
  - All load instructions require R-permission and are authorized by the capability in `rs1`.
  - All store instructions require W-permission and are authorized by the capability in `rs1`.
  - All AMO instructions require both R-permission and W-permission, and are authorized by the capability in `rs1`.

E.g., `lw t0, 16(sp)` loads a word from memory, getting the address, bounds, and permissions from the `sp` (capability stack pointer) register.

2.8. RVY Instruction Encodings

RVY includes all instructions defined by the RV32[IE]/RV64[IE] base ISAs and defines new instructions to operate on capabilities.

As shown in Table 12, three major opcodes are allocated for standard RVY instructions:

- RVY-A (`inst[6:2]=11110`).

- RVY-B (`inst[6:2]=10110`).
- RVY-C (`inst[6:2]=01010`).

The ***custom*** major opcode (`inst[6:2]=00010`) is available for custom instructions.

Table 12. RISC-V base opcode map for RVY, inst[1:0]=11

inst[4:2] / inst[6:5]	000	001	010	011	100	101	110	111 (>32b)
00	LOAD	LOAD-FP	*custom*	MISC-MEM	OP-IMM	AUIPC	OP-IMM-32	*reserved*
01	STORE	STORE-FP	**RVY-C**	AMO	OP	LUI	OP-32	*reserved*
10	MADD	MSUB	NMSUB	NMADD	OP-FP	OP-V	**RVY-B**	*reserved*
11	BRANCH	JALR	*reserved*	JAL	SYSTEM	OP-VE	**RVY-A**	*reserved*



In the RV32[IE]/RV64[IE] base opcode map, these four major opcodes correspond to custom-3, custom-2, custom-1, and custom-0, respectively. Standard RVY instructions use the former three, leaving a single ***custom*** *opcode space for vendor extensions. In the future parts of* RVY-A/B/C *may be allocated for custom use.*

2.8.1. Instructions to Update The Capability Pointer

Creating a new capability with a different address (i.e., updating the pointer) requires specific instructions instead of integer ADD/ADDI. These instructions all include a check that the resulting address can be represented exactly within the new capability.

Table 13. Instructions which update the address field summary in RVY

Mnemonic	Description
YADDI	Capability pointer increment by immediate
YADD	Capability pointer increment
YADDRW	Write capability address

2.8.1.1. YADD, YADDI

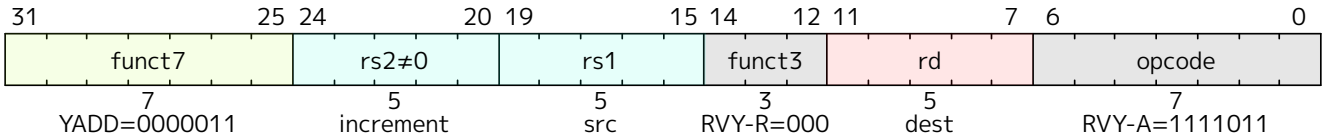
Synopsis

Capability pointer increment

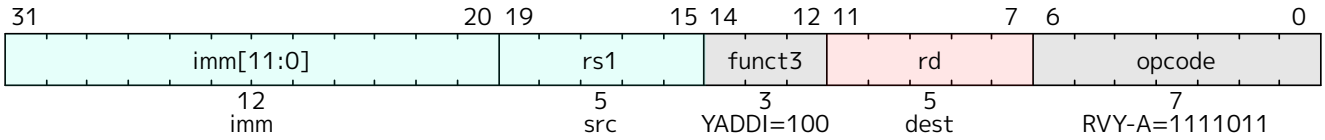
Mnemonics

```
YADD rd, rs1, rs2
YADDI rd, rs1, imm
```

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0



The instruction encoding is not final and is highly likely to change prior to v1.0

YMV uses the reserved encoding of YADD where `rs2=0`.

Description

Copy the capability in register `rs1` to register `rd`.

For YADD, increment `rd.address` by the value in `rs2[XLEN-1:0]`.
For YADDI, increment `rd.address` by the sign-extended immediate value `imm`.

Set `rd.tag=0` if `rs1` is sealed.

Set `rd.tag=0` if the resulting capability cannot be represented exactly.

Set `rd.tag=0` if `rs1` fails any integrity checks.

Operation for YADD

```
let cs1_val = C(cs1);
let rs2_val = X(rs2);

let newCap = incCapAddrChecked(cs1_val, rs2_val);

C(cd) = newCap;
RETIRE_SUCCESS
```

Operation for YADDI

```
let cs1_val = C(cs1);
let immBits : xlenbits = sign_extend(imm);
```

```
let newCap = incCapAddrChecked(cs1_val, immBits);

C(cd) = newCap;
RETIRE_SUCCESS
```

2.8.1.2. YADDRW

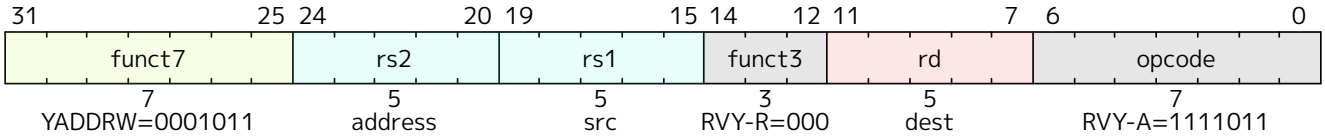
Synopsis

Write capability address

Mnemonic

```
YADDRW rd, rs1, rs2
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Copy the capability `rs1` to `rd`.

Set `rd.address` to `rs2[XLEN-1:0]`.

Set `rd.tag=0` if `rs1` is sealed.

Set `rd.tag=0` if the resulting capability cannot be represented exactly.

Set `rd.tag=0` if `rs1` fails any integrity checks.

Operation

```
C(cd) = setCapAddrChecked(C(cs1), X(rs2));
RETIRE_SUCCESS
```

2.8.2. Instructions to Manipulate Capabilities

For security, capabilities can only be modified in restricted ways. Special instructions are provided to copy capabilities or derive a new capability using manipulations such as *shrinking* the bounds (YBNDSW), *reducing* the permissions (YPERMC) or *authorizing* a capability with another one which has a superset (or identical) bounds and permissions (YBLD).

Capability subset

Some instructions use the concept of `cap2` being a *capability subset* of `cap1`. The definition is:

1. All permissions granted by the AP-field and the SDP-field of `cap2` are also granted by those of `cap1`, and
2. `cap2` 's top bound address is less than or equal to `cap1` 's top bound address, and
3. `cap2` 's base bound address is greater than or equal to `cap1` 's base bound address, and
4. `cap1` and `cap2` pass all integrity checks



A valid capability is assumed to satisfy the integrity checks, so this rule can only affect the result for an operand whose capability tag is set to 0. Among the instructions that use the subset relation, this is only material for the `rs2` operand of YBLD (which can set a capability tag on an arbitrary bit pattern) and for YSS (which can compare operands whose capability tags are set to 0).

Table 14. Summary of RVY instructions that create a modified capability

Mnemonic	Description
YPERMC	Clear capability permissions
YMV	Capability register copy
YHIW	Write capability metadata and clear capability tag
YBNDSWI	Write capability bounds by immediate
YBNDSW	Write capability bounds
YBNDSRW	Write capability bounds, rounding up if required
YUNSEALS	Unseal by superset reconstruction
YBLD	Build capability
YSEALE	Create a sealed entry point capability

2.8.2.1. YPERMC

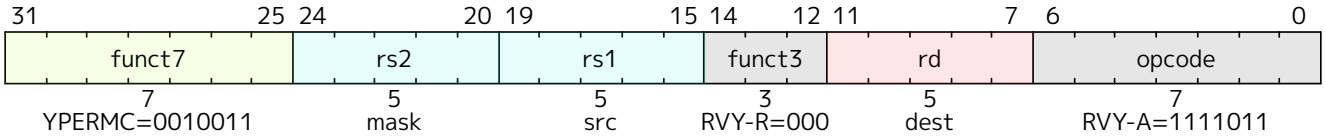
Synopsis

Clear capability permissions

Mnemonic

```
YPERMC rd, rs1, rs2
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0

Description

YPERMC performs the following operations:

1. Convert the AP-field and SDP-field fields of capability `rs1` into a bit field with the format shown in Figure 6.
2. The initial value in register `rs2[XLEN-1:0]` is treated as a bit mask that specifies bit positions to be cleared in the bit field. Any bit that is high in `rs2` will cause the corresponding bit to be cleared in the bit field.
    a. Permission bits which are hardwired to one are not cleared by set bits in `rs2`.
    This refers to bits shown as `Reserved 1` in Figure 6.
3. Encode the resulting architectural permissions as specified by the capability-encoding format in use. This involves applying the rules in Section 2.3.10.1, as well as any rules added by extensions or the capability-encoding format.
    a. If the capability-encoding format does not support the requested set of permissions then YPERMC will return a minimal set of permissions, which may be no permissions. Therefore, it is possible that requesting to clear a permission also clears others, but YPERMC will never *add* new permissions.
4. Copy `rs1` to `rd`, and update the AP-field and SDP-field fields with the newly calculated versions.
5. Set `rd.tag=0` if `rs1` is sealed and any bits in the AP-field or SDP-field were affected by YPERMC.
    a. Extensions such as Zylg add a field to the metadata which is reduced by YPERMC even when the capability is sealed.
6. Set `rd.tag=0`, `rd.AP=0`, `rd.SDP=0`, and `rd.P=0`[1] if `rs1` fails any integrity checks.

[1] If Zyhybrid is implemented.

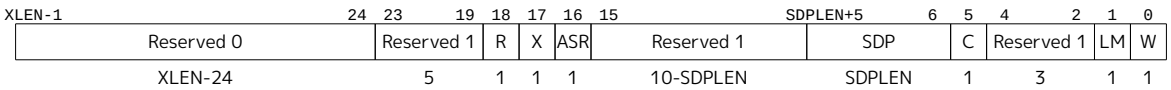


Figure 6. Capability permissions bit field



If a future extension adds a new permission that is dependent on an existing permission then clearing the original must also clear the new permission. This ensures software forward-compatibility: for example, a kernel that does not know about finer-grained ASR permissions must still be able to prevent all access to privileged instructions and CSRs simply by clearing ASR-permission. Such dependent permissions must be allocated to the bits that are currently

reported as 1 to ensure forward-compatibility.

Operation

This listing is manually maintained rather than extracted from the Sail model.

```
let cs1_val = C(cs1);
let rs2_val = X(rs2);

let cond = capIsSealed(cs1_val) | not(capReservedValid(cs1_val));
let inCap = clearTagIf(cs1_val, cond);

let old_perms = packPerms(getArchPermsLegalized(inCap), inCap.sd_perms).bits;

let new_perms = old_perms & not(rs2_val);

let (new_arch_perms, new_sd_perms) = unpackPerms(struct {bits = new_perms});
let newCap = { setArchPerms(inCap, new_arch_perms) with sd_perms = new_sd_perms
};

C(cd) = newCap;
RETIRE_SUCCESS
```

2.8.2.2. YMV

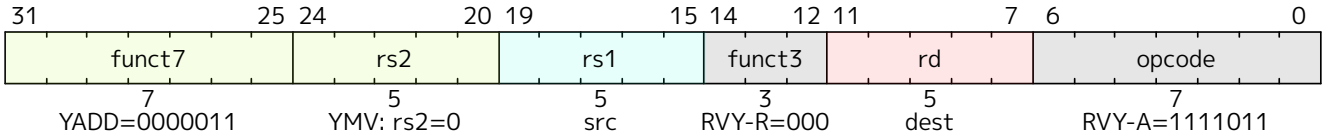
Synopsis

Capability register copy

Mnemonic

```
YMV rd, rs1
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0



YMV uses the reserved encoding of YADD where `rs2=0`.

Description

Copy `rs1` to `rd`.

This instruction can propagate valid capabilities which fail integrity checks.

Operation

```
C(cd) = C(cs1);
RETIRE_SUCCESS
```

2.8.2.3. YHIW

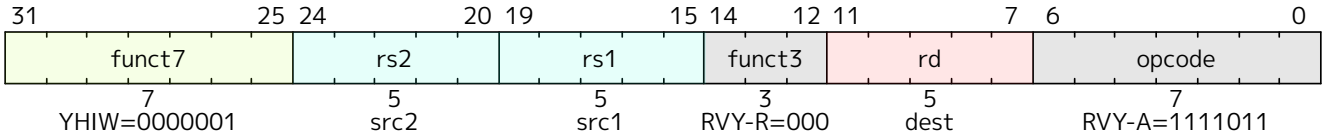
Synopsis

Write capability metadata and clear capability tag

Mnemonic

```
YHIW rd, rs1, rs2
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0

Description

The YHIW instruction copies `rs1[XLEN-1:0]` into `rd[XLEN-1:0]`, copies `rs2[XLEN-1:0]` into `rd[YLEN-1:XLEN]`, and sets `rd.tag=0`.

Operation

```
let capVal = C(cs1);
let intVal = X(rs2);
let newCap = bitsToCap(false, intVal @ capVal.address);
C(cd) = newCap;
RETIRE_SUCCESS
```

2.8.2.4. YBNDSW, YBNDSWI

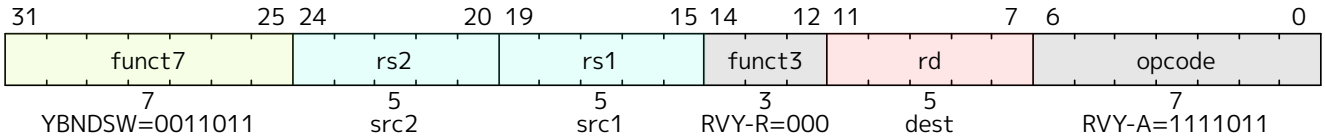
Synopsis

Write capability bounds

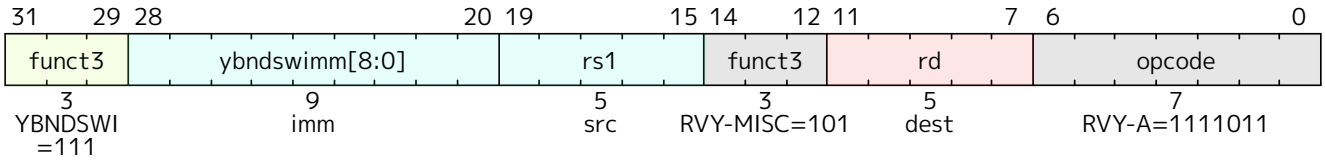
Mnemonics

```
YBNDSW rd, rs1, rs2
YBNDSWI rd, rs1, imm
```

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0



The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Copy the capability from register `rs1` to register `rd`. Set the capability base address to the value of `rs1.address` and set the length of its bounds to `rs2[XLEN-1:0]` for YBNDSW, or `imm` for YBNDSWI.

Set `rd.tag=0` if `rs1.tag=0`.

Set `rd.tag=0` if `rs1` is sealed.

Set `rd.tag=0` if `rd` 's bounds exceed `rs1` 's bounds.

Set `rd.tag=0` if the requested bounds cannot be encoded exactly.

Set `rd.tag=0` if `rs1` fails any integrity checks.

YBNDSWI decodes the 9-bit immediate value `imm[8:0]` to the requested length `result` as follows:

- If `imm[8:0] = 0`, `result` is `4096`.
- If `imm[8] = 0` and `imm[7:0] != 0`, `result` is `imm[7:0]`.
- If `imm[8] = 1` and `imm[7:5] = 0`, `result` is `256 | (imm[3:0] << 4) | (imm[4] << 3)`.
- Otherwise, `result` is `imm[7:0] << 4`.

The resulting logical encodable regions and step sizes are summarized in the table below:

Region	Decoded range (bytes)	Immediate decoding condition
Byte granular	`1, 2, …, 255`	`imm[8] = 0` and `imm[7:0] > 0`
8-byte granular	`256, 264, …, 504`	`imm[8:5] = 4'b1000`
16-byte granular	`512, 528, …, 4080`	`imm[8:5] > 4'b1000`

Region	Decoded range (bytes)	Immediate decoding condition
Page-sized	4096	imm[8:0] = 0

This immediate encoding scheme achieves coverage of almost all observed immediate values while decoding the 9-bit immediate field without any adders or variable shifts. Mapping `imm = 0` *to* `4096` *reclaims the redundant zero encoding (already covered by* `YBNDSW dst, src, x0`*) to support highly common page-sized objects.*



The hardware implementation is a simple bit-remapping:

```
result[12] = (imm[8:0] == 0);
result[11:9] = imm[8] ? imm[7:5] : 0;
result[8] = imm[8] ? (imm[7:5] == 0b000 || imm[4]) : 0;
result[7:4] = imm[8] ? imm[3:0] : imm[7:4];
result[3] = imm[8] ? (imm[7:4] == 0b0001) : imm[3];
result[2:0] = imm[8] ? 0 : imm[2:0];
```

Operation for YBNDSW

```
let cs1_val = C(cs1);
let length = X(rs2);
let newBase = cs1_val.address;
let newTop : CapLenBits = zero_extend(newBase) + zero_extend(length);
// inCapBoundsNoWrap returns false if the input bounds are malformed.
let inBounds = inCapBoundsNoWrap(cs1_val, newBase, unsigned(length));
let (exact, newCap) : (bool, Capability) = setCapBounds(cs1_val, newBase,
newTop);
let cond = not(inBounds & exact) |
           boundsMalformed(newCap) |
           not(capReservedValid(newCap)) |
           capIsSealed(newCap);
C(cd) = clearTagIf(newCap, cond);
RETIRE_SUCCESS
```

Operation for YBNDSWI



This listing is manually maintained rather than extracted from the Sail model.

```
let cs1_val = C(cs1);
// Decode the requested length from the 9-bit immediate.
let length : xlenbits =
  if imm == 0b000000000 then to_bits(xlen, 4096)
  else if imm[8] == bitzero then zero_extend(imm[7 .. 0])
  else if imm[7 .. 5] == 0b000 then zero_extend(0b1 @ imm[3 .. 0] @ imm[4 .. 4]
@ 0b000)
  else zero_extend(imm[7 .. 0] @ 0b0000);
let newBase = cs1_val.address;
let newTop : CapLenBits = zero_extend(newBase) + zero_extend(length);
// inCapBoundsNoWrap returns false if the input bounds are malformed.
let inBounds = inCapBoundsNoWrap(cs1_val, newBase, unsigned(length));
let (exact, newCap) : (bool, Capability) = setCapBounds(cs1_val, newBase,
newTop);
```

```
let cond = not(inBounds & exact) |
           boundsMalformed(newCap) |
           not(capReservedValid(newCap)) |
           capIsSealed(newCap);
C(cd) = clearTagIf(newCap, cond);
RETIRE_SUCCESS
```

2.8.2.5. YBNDSRW

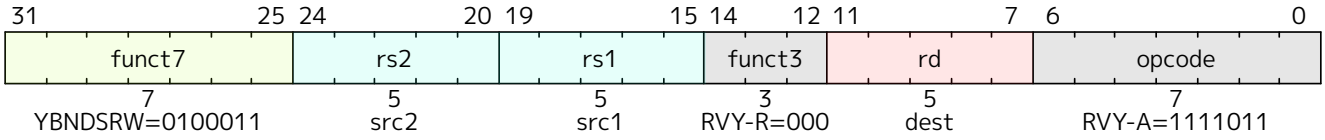
Synopsis

Write capability bounds, rounding up if required

Mnemonic

```
YBNDSRW rd, rs1, rs2
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Copy the capability from register `rs1` to register `rd`. Set the capability base address to the value of `rs1.address` and set the length of its bounds to `rs2[XLEN-1:0]`.

The base is rounded down and the top is rounded up by the smallest amounts needed to form a capability covering the requested base and top.

Set `rd.tag=0` if `rs1.tag=0`.

Set `rd.tag=0` if `rs1` is sealed.

Set `rd.tag=0` if `rd` 's bounds exceed `rs1` 's bounds.

Set `rd.tag=0` if `rs1` fails any integrity checks.

Operation

```
let cs1_val = C(cs1);
let length = X(rs2);
let newBase = cs1_val.address;
let newTop : CapLenBits = zero_extend(newBase) + zero_extend(length);
// inCapBoundsNoWrap returns false if the input bounds are malformed.
let inBounds = inCapBoundsNoWrap(cs1_val, newBase, unsigned(length));
let (_, newCap) : (bool, Capability) = setCapBounds(cs1_val, newBase,
newTop);
let cond = not(inBounds) |
           boundsMalformed(newCap) |
           not(capReservedValid(newCap)) |
           capIsSealed(newCap);
C(cd) = clearTagIf(newCap, cond);
RETIRE_SUCCESS
```



Checking only the requested region with `inCapBoundsNoWrap` *is sufficient, since the rounding performed by* `setCapBounds` *can never grow the bounds beyond those of* `rs1`.

2.8.2.6. YUNSEALS

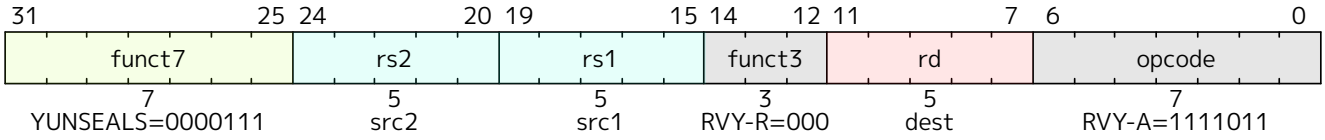
Synopsis

Unseal by superset reconstruction

Mnemonic

```
YUNSEALS rd, rs1, rs2
```

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Copy `rs2` to `rd`.

Set `rd.ct=0`. (That is, unseal `rd`.)

Set `rd.tag=1` if:

1. `rs1.tag=1`, and
2. `rs1` is not sealed, and
3. `rs2.tag=1`, and
4. `rs2` is sealed, and
5. `rs2` is a capability subset of `rs1`

Otherwise, set `rd.tag=0`

When `rs1` is `x0` YUNSEALS will copy `rs2` to `rd` and clear `rd.tag` and `rd.ct`. However future extensions may add additional behavior to update currently reserved fields, and so software should not assume `rs1=0` to be a pseudo-instruction for capability tag and type clearing.

Operation

This listing is manually maintained rather than extracted from the Sail model.

```
let cs1_val = C(cs1);
let cs2_val = C(cs2);

let tag = cs1_val.tag &
          not(capIsSealed(cs1_val)) &
          cs2_val.tag &
          capIsSealed(cs2_val) &
          /* subset checks for malformed bounds, perms, reserved bits */
          capIsSubset(cs2_val, cs1_val);

C(cd) = { unsealCap(cs2_val) with tag = tag };
```

```
RETIRE_SUCCESS
```

2.8.2.7. YBLD

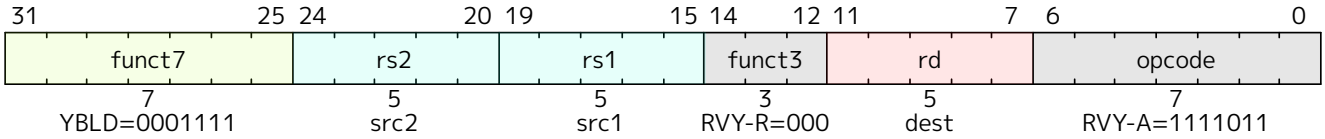
Synopsis

Build capability

Mnemonic

```
YBLD rd, rs1, rs2
```

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Copy `rs2` to `rd`.

Set `rd.tag=1` if:

1. `rs1.tag=1`, and
2. `rs1` is not sealed, and
3. `rs2` is a capability subset of `rs1`, and
4. `rs2` passes a mandatory integrity check.

Otherwise, set `rd.tag=0`.

The integrity check on `rs2` is mandatory to prevent authorizing a capability with a lack of integrity.

The integrity check, from the capability subset definition, on `rs1` is optional.

YBLD is typically used alongside YHIW to build capabilities from integer values.

YBLD can be used to speed up operations such as paging in memory after swap.

When `rs1` is `x0` YBLD will copy `rs2` to `rd` and clear `rd.tag`. However future extensions may add additional behavior to update currently reserved fields, and so software should not assume `rs1=0` to be a pseudoinstruction for capability tag clearing.

Operation

```
let cs1_val = C(cs1);
let cs2_val = C(cs2);

let tag = cs1_val.tag &
          not(capIsSealed(cs1_val)) &
          capIsSubset(cs2_val, cs1_val); /* Subset checks for malformed
```

```
bounds,
                                                    perms, and reserved bits */

  C(cd) = { cs2_val with tag = tag };
  RETIRE_SUCCESS
```

2.8.2.8. YSEALE

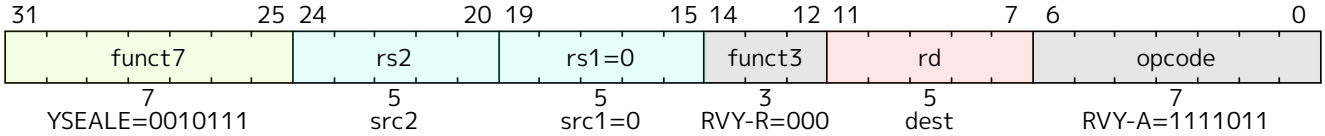
Synopsis

Create a sealed entry point capability

Mnemonic

`YSEALE rd, rs2`

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

`rs1≠x0` *is currently reserved to decode as a future YSEAL instruction.*

Description

Copy `rs2` to `rd`.

Seal the capability in `rd` by setting `rd.ct=1`.

Set `rd.tag=0` if `rs2` is sealed or fails any integrity checks.

A future extension will allow YSEAL, which has `rs1` *specifying an authorizing capability, to perform the sealing action and to specify the destination CT-field value.*

If the capability-encoding format does not support a CT-field of type 1, then this instruction is reserved. Future capability-encoding formats may redefine the behavior for CT-field values greater than 1.

Operation

This listing is manually maintained rather than extracted from the Sail model.

```
let cs2_val = C(cs2);
let inCap = clearTagIf(cs2_val, capIsSealed(cs2_val) |
not(capReservedValid(cs2_val)));
C(cd) = sealCap(inCap);
RETIRE_SUCCESS
```

2.8.3. Instructions to Decode Capability Bounds

The *bounds* describing the range of addresses the capability gives access to are stored in a compressed format. These instructions query the bounds and related information.

Table 15. Instructions which decode capability bounds summary in RVY

Mnemonic	Description
YBASER	Read capability base address
YLENR	Read capability length
YTOPR	Read capability top address

2.8.3.1. YBASER

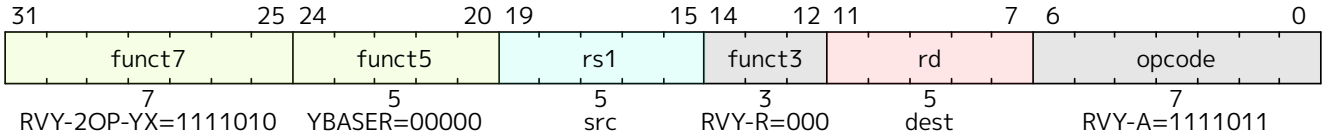
Synopsis

Read capability base address

Mnemonic

```
YBASER rd, rs1
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Decode the base integer address from `rs1` 's bounds and write the result to `rd`.

If `rs1` 's bounds cannot be decoded, or `rs1` fails any integrity checks, then return zero.



The value of `rs1.tag` does not affect the result.

Operation

```
let capVal = C(cs1);
X(rd) = match getCapBoundsBits(capVal) {
  None() => zeros(),
  Some(base, _) => base
};
RETIRE_SUCCESS
```

2.8.3.2. YLENR

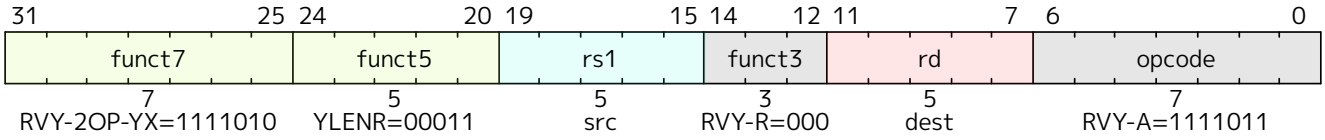
Synopsis

Read capability length

Mnemonic

```
YLENR rd, rs1
```

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Calculate the length of `rs1` 's bounds and write the result in `rd`.

The length is defined as the difference between the decoded top and base bound addresses, i.e., `top - base`.

Return the maximum length, 2^{XLEN}-1, if the length of `rs1` is 2^{XLEN}.

If `rs1` 's bounds cannot be decoded, or `rs1` fails any integrity checks, then return zero.

The value of `rs1.tag` does not affect the result.

Operation

```
let capVal = C(cs1);
// getCapLength returns 0 if the bounds are malformed
let len = getCapLength(capVal);
X(rd) = to_bits(xlen, if len > cap_max_addr then cap_max_addr else len);
RETIRE_SUCCESS
```

2.8.3.3. YTOPR

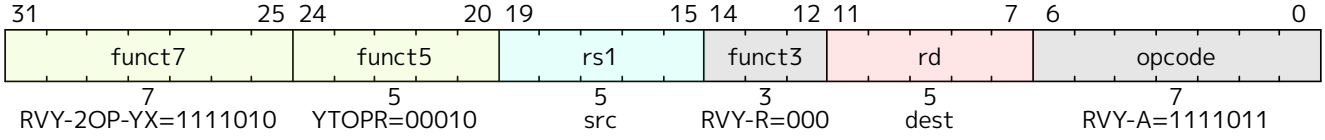
Synopsis

Read capability top address

Mnemonic

```
YTOPR rd, rs1
```

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Decode the top integer address from `rs1` 's bounds, saturated to 2^{XLEN}-1, and write the result to `rd`.

If `rs1` 's bounds cannot be decoded, or `rs1` fails any integrity checks, then return zero.



The value of `rs1.tag` does not affect the result.

Operation



This listing is manually maintained rather than extracted from the Sail model.

```
let capVal = C(cs1);
X(rd) = match getCapBoundsBits(capVal) {
  None() => zeros(),
  Some(_, top) => {
    let top_val = unsigned(top);
    to_bits(xlen, if top_val > cap_max_addr then cap_max_addr else top_val)
  }
};
RETIRE_SUCCESS
```

2.8.4. Instructions to Extract Capability Fields

These instructions either directly read bit fields from the metadata or capability tag, or only apply simple transformations on the metadata.

Table 16. Instructions which extract capability fields summary in RVY

Mnemonic	Description
YTAGR	Read capability tag
YPERMR	Read capability permissions
YTYPER	Read capability type
YHIR	Read capability metadata

2.8.4.1. YTAGR

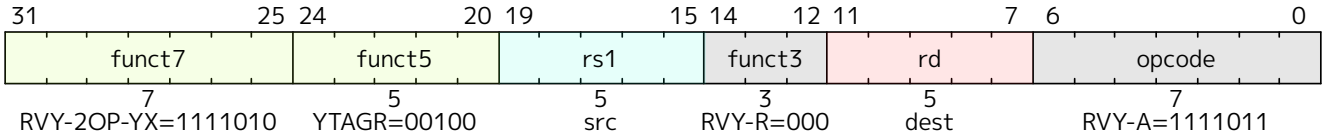
Synopsis

Read capability tag

Mnemonic

```
YTAGR rd, rs1
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Zero-extend the value of `rs1.tag` and write the result to `rd`.

Operation

```
let capVal = C(cs1);
X(rd) = zero_extend(bool_to_bits(capVal.tag));
RETIRE_SUCCESS
```

2.8.4.2. YPERMR

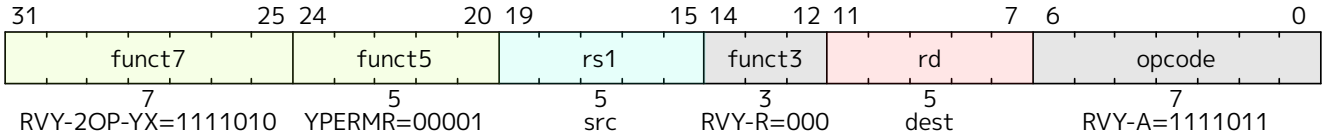
Synopsis

Read capability permissions

Mnemonic

```
YPERMR rd, rs1
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Convert the unpacked AP-field and SDP-field fields of capability `rs1` into a bit field, with the same format as used by YPERMC (see Figure 7), zero-extend, and write the result to `rd`.

All bits in the `[23:0]` range that are reserved or assigned to extensions that are not implemented by the current hart are hardwired to 1.

If `rs1` fails any integrity checks, all currently allocated permission bits in `rd` report 0.



The hardwired bits described above are unaffected and still report 1.

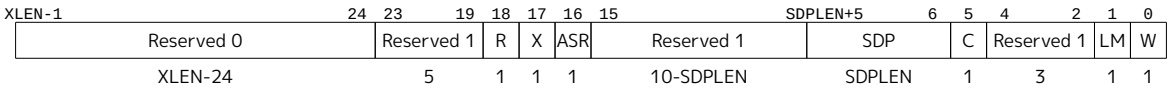


Figure 7. Capability permissions bit field



The value of `rs1.tag` does not affect the result.

Operation

```
let capVal = C(cs1);
X(rd) = packPerms(getArchPermsLegalized(capVal), capVal.sd_perms).bits;
RETIRE_SUCCESS
```

2.8.4.3. YTYPER

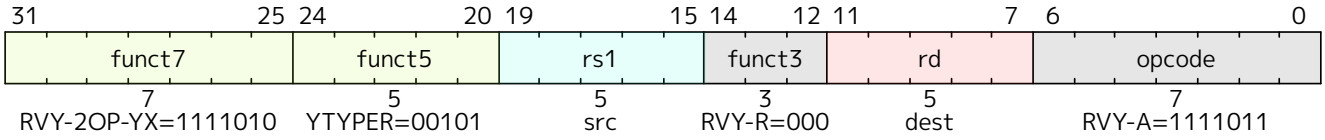
Synopsis

Read capability type

Mnemonic

```
YTYPER rd, rs1
```

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Decode the architectural capability type (CT-field) from `rs1`, zero-extend, and write the result to `rd`.

The value of `rs1.tag` does not affect the result.

Operation

```
let capVal = C(cs1);
X(rd) = zero_extend(bool_to_bits(capVal.sealed));
RETIRE_SUCCESS
```

The Sail model stores the 1-bit CT-field of RV64LYA as the boolean `sealed`. Capability-encoding formats with a wider CT-field must return the full field value.

2.8.4.4. YHIR

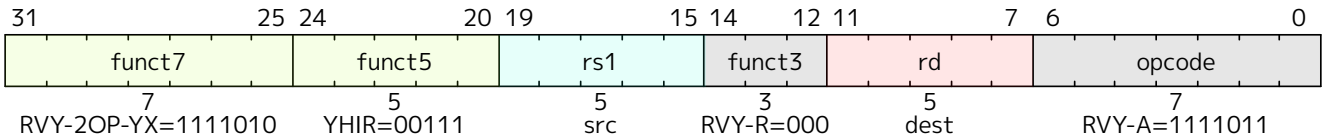
Synopsis

Read capability metadata

Mnemonic

```
YHIR rd, rs1
```

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Copy the metadata (bits [YLEN-1:XLEN]) of capability `rs1` into `rd`.

The value of `rs1.tag` does not affect the result.

Operation

```
let capVal = C(cs1);
X(rd) = capToMetadataBits(capVal).bits;
RETIRE_SUCCESS
```

2.8.5. Miscellaneous Instructions to Handle Capability Data

Table 17. Miscellaneous capability instruction summary in RVY

Mnemonic	Description
YEQ	Capability equality comparison including capability tag
YSS	Capability is a subset
YAMASK	Capability alignment mask

2.8.5.1. YEQ

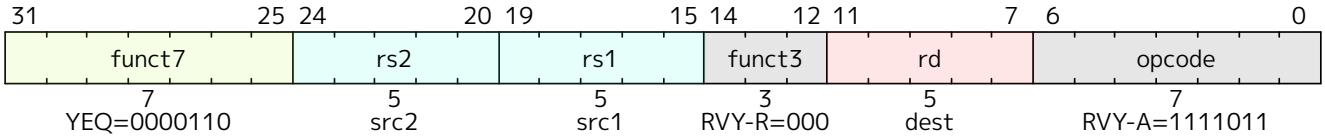
Synopsis

Capability equality comparison including capability tag

Mnemonic

```
YEQ rd, rs1, rs2
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Set `rd` to 1 if all bits (i.e., YLEN bits and the capability tag) of capabilities `rs1` and `rs2` are equal, otherwise set `rd` to 0.

Operation

```
let cs1_val = C(cs1);
let cs2_val = C(cs2);
X(rd) = zero_extend(bool_to_bits(cs1_val == cs2_val));
RETIRE_SUCCESS
```

2.8.5.2. YSS

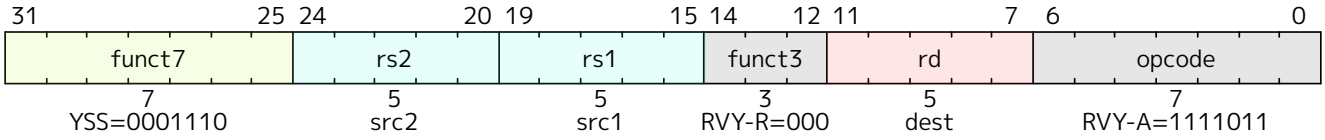
Synopsis

Capability is a subset

Mnemonic

```
YSS rd, rs1, rs2
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0

Description

`rd` is set to 1 if:

1. the capability tags of capabilities `rs1` and `rs2` are equal, and
2. `rs2` is a capability subset of `rs1`

Otherwise set `rd` to 0. Extensions may further impose constraints on when `rd` is set to 1.



The implementation of this instruction is similar to YBLD, although YSS does not include the sealed bit in the check.

Operation

```
  let cs1_val = C(cs1);
  let cs2_val = C(cs2);

  X(rd) = zero_extend(bool_bits(
    (cs1_val.tag == cs2_val.tag) &
    capIsSubset(cs2_val, cs1_val) /* capIsSubset returns false if either
input
                                     has malformed bounds, perms, or non-zero
                                     reserved bits */
  ));
  RETIRE_SUCCESS
```

2.8.5.3. YAMASK

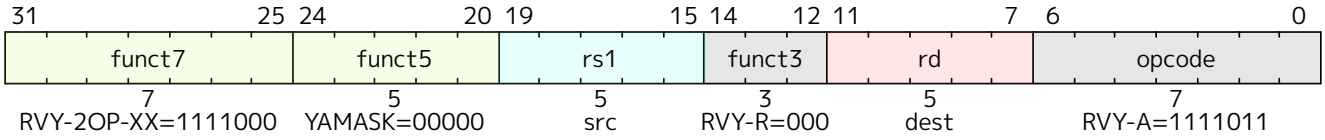
Synopsis

Capability alignment mask

Mnemonic

```
YAMASK rd, rs1
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0

Description

`rd[XLEN-1:0]` is set to a mask that can be used to round addresses down to a value that is sufficiently aligned to set exact bounds for the nearest representable length of `rs1[XLEN-1:0]`. The upper bits of `rd` are zero-extended. See Section 3.1.5 for the algorithm used to compute the next representable length.



This RVY instruction only handles XLEN-bit operands and produces an XLEN-bit result.

Operation

```
let len = X(rs1);
X(rd) = getRepresentableAlignmentMask(len);
RETIRE_SUCCESS
```

2.8.6. Instructions to Load and Store Capability Data

New loads and stores are introduced to handle capability data, LY and SY. These capability load and store instructions atomically access YLEN bits of data and the associated capability tag.

In addition to the common authorization checks for memory access instructions (see Section 2.7), capability memory accesses check for C-permission in the authorizing capability in `rs1`.

If C-permission is granted then:

- LY reads YLEN bits of data from memory, and returns the associated capability tag.
- SY writes YLEN bits of data to memory, and writes the associated capability tag.

If C-permission is not granted then:

- LY reads YLEN bits from memory, but does not return the associated capability tag, instead zero is returned.
- SY writes YLEN bits to memory, and writes zero to the associated capability tag.

All capability data memory access instructions require naturally aligned addresses, and will take an access fault exception if this requirement is not met. Misaligned capability accesses cannot be emulated.



An access fault is raised instead of a misaligned exception because these instructions cannot be emulated since there is one hidden capability tag per naturally aligned YLEN-bit memory region.

Under some circumstances LY will *modify* the data loaded from memory before writing it back to the destination register. See LY for details.

Table 18. Capability load/store instruction summary in RVY

Mnemonic	Description
LY	Load capability
SY	Store capability

2.8.6.1. LY

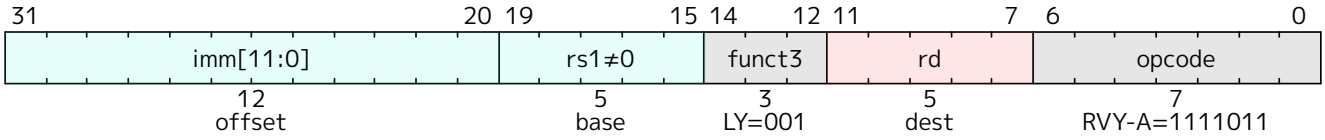
Synopsis

Load capability

Mnemonic

```
LY rd, offset(rs1)
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0



Encodings with `rs1`=`x0` are RESERVED for use by future extensions, as `x0` is defined to always hold a NULL capability.

Description

Calculate the effective address of the memory access by adding `rs1.address` to the sign-extended 12-bit offset.

Authorize the memory access with the capability in `rs1`.

Load a naturally aligned YLEN-bit data value and associated capability tag from memory.



The loaded capability may be invalidated under platform-specified conditions, such as by a hardware-based use-after-free prevention scheme. Privileged mechanisms may also set the loaded capability tag to zero such as PMA settings or virtual memory system page table entry configuration.



Extensions may also specify additional conditions which invalidate the resulting capability.

The final value of `rd` is determined as follows:

1. Write the loaded YLEN-bit data to `rd` and the loaded capability tag into `rd.tag`.
2. If `rs1` does not grant C-permission then set `rd.tag=0`.
3. If `rd.tag=1`, `rs1` does not grant LM-permission, and `rd` is not sealed, then an implicit YPERMC is performed to clear W-permission and LM-permission from `rd`.



The implicit YPERMC is not required to set `rd.tag=0` if the loaded data fails any integrity checks. A suggested implementation is to only check the AP-field for legal values, and set the whole AP-field to zero if the check fails.



Metadata modifications on loaded capabilities are on naturally aligned data. Therefore on the read path from a data cache, the modification typically happens in parallel with data alignment multiplexers.



Zylg adds to the YPERMC rules on loaded capability data.

When sending load data to a trace interface, implementations trace the final value written to `rd` which may not match the value in memory.

This instruction can propagate valid capabilities which fail integrity checks.

Exceptions

Raise a load access fault exception when the effective address is not naturally aligned.

Exceptions occur when the authorizing capability fails one of the checks listed below:

Kind	Reason
CHERI Load Access Fault	Authorizing capability tag is set to 0.
CHERI Load Access Fault	Authorizing capability is sealed.
CHERI Load Access Fault	Authorizing capability does not grant the necessary permissions. R-permission is required.
CHERI Load Access Fault	At least one byte accessed is outside the authorizing capability bounds, or the bounds could not be decoded.
CHERI Load Access Fault	Authorizing capability failed any integrity check.

Operation

```
  let offset : xlenbits = sign_extend(imm);
  let (auth_val, vaddr) = get_cheri_mode_cap_addr(rs1_cs1, offset);
  let aq : bool = false;
  let rl : bool = false;

  match check_and_handle_load_vaddr_for_triggers(vaddr, get_arch_pc()) {
    Some (ret) => return ret,
    None () => ()
  };
  if not(capTaggedAndReservedValid(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_TagViolation);
    RETIRE_FAIL
  } else if capIsSealed(auth_val) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_SealViolation);
    RETIRE_FAIL
  } else if not(canR(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_PermissionViolation);
    RETIRE_FAIL
  } else if not(validAddrRange(vaddr, cap_size) |
capBoundsInfinite(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_InvalidAddressViolation);
    RETIRE_FAIL
  } else if not(inCapBounds(auth_val, vaddr, cap_size)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_LengthViolation);
    RETIRE_FAIL
  } else if not(is_aligned_addr(vaddr, cap_size)) then {
    handle_mem_exception(vaddr, E_Load_Addr_Align());
    RETIRE_FAIL
  } else match translateAddr(vaddr, Read(Cap)) {
    TR_Failure(E_Extension(_)) => { internal_error(__FILE__, __LINE__,
"unexpected cheri exception for cap load") },
    TR_Failure(e) => { handle_mem_exception(vaddr, e); RETIRE_FAIL },
    TR_Address(addr, pbmt, ptw_info) => {
```

```
      let c = mem_read_cap(addr, pbmt, aq, aq & rl, false);
      match c {
        MemValue(v) => {
          let cr = clearTagIf(v, ptw_info.ptw_lc == PTW_LC_CLEAR |
not(canC(auth_val)));
          C(cd) = legalizeLM(cr, auth_val);
          RETIRE_SUCCESS
        },
        MemException(e) => {handle_mem_exception(vaddr, e); RETIRE_FAIL }
      }
    }
  }
```

2.8.6.2. SY

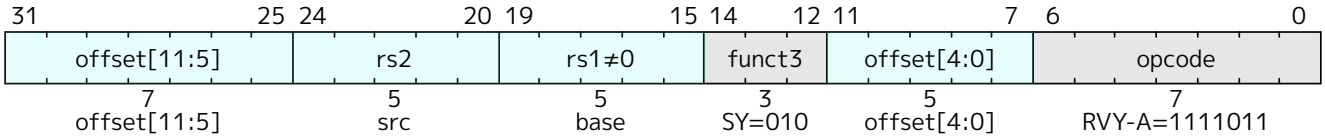
Synopsis

Store capability

Mnemonic

```
SY rs2, offset(rs1)
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0



Encodings with `rs1`=`x0` are RESERVED for use by future extensions, as `x0` is defined to always hold a NULL capability.

Description

Calculate the effective address of the memory access by adding `rs1.address` to the sign-extended 12-bit offset.

Authorize the memory access with the capability in `rs1`.

Store the YLEN-bit value in `rs2`, together with its capability tag, to the naturally aligned memory location.

This instruction can propagate valid capabilities which fail integrity checks.

Stored Capability Tag Value

Set the stored capability tag to zero if:

1. `rs2.tag=0`, or
2. `rs1` does not grant C-permission, or
3. Platform-specified conditions or privileged configurations (e.g., PMAs or PTEs) specify that it must be set to 0.



Extensions may define further circumstances under which stored capabilities may have their capability tags set to 0. Zylg is an example of this.

Exceptions

Raise a store/AMO access fault exception when the effective address is not naturally aligned.

Exceptions occur when the authorizing capability fails one of the checks listed below:

Kind	Reason
CHERI Store/AMO Access Fault	Authorizing capability tag is set to 0.
CHERI Store/AMO Access Fault	Authorizing capability is sealed.
CHERI Store/AMO Access Fault	Authorizing capability does not grant the necessary permissions. W-permission is required.

Kind	Reason
CHERI Store/AMO Access Fault	At least one byte accessed is outside the authorizing capability bounds, or the bounds could not be decoded.
CHERI Store/AMO Access Fault	Authorizing capability failed any integrity check.

Operation

```
  let offset : xlenbits = sign_extend(imm);
  let (auth_val, vaddr) = get_cheri_mode_cap_addr(rs1_cs1, offset);
  let cs2_val = C(cs2);
  let aq : bool = false;
  let rl : bool = false;
  let cs2_val = clearTagIf(cs2_val, not(canC(auth_val)));

  match check_and_handle_store_vaddr_for_triggers(vaddr, get_arch_pc()) {
    Some (ret) => return ret,
    None () => ()
  };
  if not(capTaggedAndReservedValid(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_TagViolation);
    RETIRE_FAIL
  } else if capIsSealed(auth_val) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_SealViolation);
    RETIRE_FAIL
  } else if not(canW(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_PermissionViolation);
    RETIRE_FAIL
  } else if not(validAddrRange(vaddr, cap_size) |
capBoundsInfinite(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_InvalidAddressViolation);
    RETIRE_FAIL
  } else if not(inCapBounds(auth_val, vaddr, cap_size)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_LengthViolation);
    RETIRE_FAIL
  } else if not(is_aligned_addr(vaddr, cap_size)) then {
    handle_mem_exception(vaddr, E_SAMO_Addr_Align());
    RETIRE_FAIL
  } else match translateAddr(vaddr, Write(if cs2_val.tag then Cap else Data))
{
    TR_Failure(e) => { handle_mem_exception(vaddr, e); RETIRE_FAIL },
    TR_Address(addr, pbmt, _) => {
      let eares : MemoryOpResult(unit) = mem_write_ea_cap(addr, aq & rl, rl,
false);
      match (eares) {
        MemException(e) => { handle_mem_exception(vaddr, e); RETIRE_FAIL },
        MemValue(_) => {
          let res : MemoryOpResult(bool) = mem_write_cap(addr, pbmt, cs2_val,
aq & rl, rl, false);
          match (res) {
            MemValue(true)  => RETIRE_SUCCESS,
            MemValue(false) => internal_error(__FILE__, __LINE__, "store got
```

```
false from mem_write_value"),
            MemException(e) => { handle_mem_exception(vaddr, e); RETIRE_FAIL
}
          }
        }
      }
    }
  }
```

2.9. Changes to Existing RISC-V Base ISA Instructions

RVY extends existing instructions that are used for handling addresses so that they manipulate a whole capability.

- Whenever an input operand is used as an address (e.g., the load/store base address), all capability bits are fed into the instruction instead of just XLEN bits.
- Any instruction that writes back an address (e.g., AUIPC (RVY) or CSRRW (RVY)) to the destination register writes a full capability register instead of just XLEN bits. For all other results the high bits of the register and the capability tag are set to 0.
- Whenever a capability with a new address is returned, the result is *always* created using the semantics of the YADDRW instruction.

`ADD` and `ADDI` are not affected by the rule above. Even though they *are* used for handling addresses, they also have other uses. New encodings are used for capability addition: YADD and YADDI. These capability add instructions must be used for all capability address incrementing.



Integer add (`ADD`) and capability add (YADD) have separate encodings. Using a single encoding for both is undesirable:

1. *Integer ADD is most commonly used for purposes other than address calculations.*
2. *For high performance implementations which can issue multiple ADDs, it means that the integer ADD units do not need the upper halves of the operands, and do not need the capability check logic on the result.*
3. *The compiler and/or programmer would have to execute another metadata clearing instruction after each ADD to ensure that compartments do not leak capabilities.*

The rules above apply to the base ISA instructions listed in the following subsections, but also apply to instructions added by other extensions. Any change to instruction semantics for RVY is called out in the chapter defining the extension.

2.9.1. Changes to Load/Stores

All load and store instructions behave as described in Load and Store Instructions with one fundamental difference:

- Any memory instruction that has `rs1` as a base address register reads the full capability register instead. The base address is unchanged, i.e., using the value from `rs1`. The metadata and capability tag are used to authorize the access.
- For a load instruction, the lower XLEN bits of the result written to the destination register are the same as in the RV32[IE]/RV64[IE] specification.

All load instructions, except for LY, always set the capability tag and the metadata of the result register to zero.

All store instructions, except for SY, always write zero to the capability tag or capability tags associated with the memory locations that are written to. Therefore, misaligned stores may clear up to two associated capability tags.

These rules affect the following base ISA instructions listed in Table 19, and also apply to instructions added by other extensions, e.g.,:

- Floating-point loads and stores.
- Vector load and stores.
- Atomic memory accesses, see Zaamo (RVY added instructions) and Zalrsc (RVY added instructions).

Table 19. Changed RISC-V base ISA load/store instructions summary in RVY

Mnemonic	Description
LD, LW[U], LH[U], LB[U]	Integer loads (authorized by the capability in `rs1`)
SD, SW, SH, SB	Integer stores (authorized by the capability in `rs1`)

2.9.2. Changes to PC

- Whenever the address field of the pc is modified, it is *always* updated using the semantics of the YADDRW instruction. This includes adding an offset to the pc from direct jumps and branches for both the target address and the link register. In this case, e.g., `new_pc = YADDRW(old_pc, offset)`
- JALR (RVY) copies `rs1` into the pc, and increments the address field with the offset. In this case, e.g., `new_pc = YADDRW(Unseal(rs1), offset)`



Using YADDRW to update the pc address can cause the target pc capability tag to be set to 0, that raises a CHERI Instruction Access Fault on the target instruction.



A future extension may raise an exception on branch and jump instructions if fetching a minimum-sized instruction at the target pc will raise a CHERI Instruction Access Fault.

These rules affect the following base ISA instructions listed in Table 20, and also apply to instructions added by other extensions.

Table 20. Changed RISC-V base ISA PC relative instructions summary in RVY

Mnemonic	Description
AUIPC (RVY)	Add upper immediate to pc
JAL (RVY)	Immediate offset jump, and link to capability register
JALR (RVY)	Jump to capability register, and link to capability register

2.9.3. AUIPC (RVY)

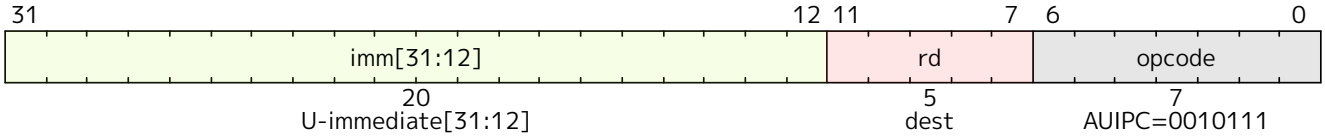
Synopsis

Add upper immediate to pc

Mnemonic

```
auipc rd, imm
```

Encoding



This instruction is modified from the version in RV32[IE]/RV64[IE].

Description

Form a 32-bit offset from the 20-bit immediate filling the lowest bits with zero; the number of places to shift is determined by the capability-encoding format's AUIPC_SHIFT parameter (defaulting to 12). Take the value of the AUIPC instruction's pc, increment its address by the 32-bit offset using the semantics of the YADDRW instruction and write the result to `rd`.

Set `rd.tag=0` if the resulting capability cannot be represented exactly.

Operation

```
let off : xlenbits = sign_extend(imm @ 0x000);
let (representable, newCap) = setCapAddr(PCC, PC + off);
C(cd) = clearTagIf(newCap, not(representable));
RETIRE_SUCCESS
```

2.9.4. JAL (RVY)

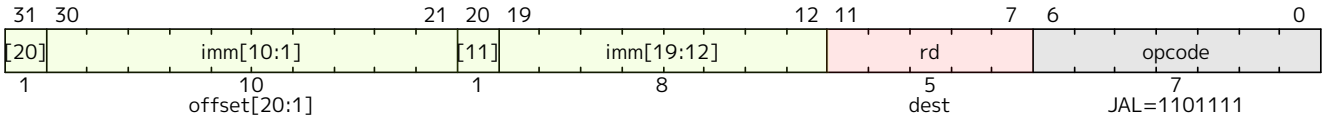
Synopsis

Immediate offset jump, and link to capability register

Mnemonic

```
jal rd, offset
```

Encoding



This instruction is modified from the version in RV32[IE]/RV64[IE].

Description

Direct jump with an address offset.

1. The target address is obtained by adding the sign-extended 20-bit J-immediate to `pc.address`.
2. The target pc is obtained by applying the target address to the pc of the JAL instruction, using the semantics of the YADDRW instruction.
3. The pc of the next instruction is sealed as a Sealed entry point capability and written to `rd`.
4. Jump to the target pc.

A future extension may raise an exception on the branch instruction itself if fetching a minimum-sized instruction at the target pc will raise a CHERI Instruction Access Fault. Performing the pc bounds check at the branch source instead of on instruction fetch is helpful for debugging and can simplify the implementation of CPUs with very short pipelines.

Operation

This listing is manually maintained rather than extracted from the Sail model.

```
let off : xlenbits = sign_extend(imm);
let newPC = PC + off;
/* nextPC accounts for compressed instructions */
let (_, linkCap) = setCapAddr(PCC, nextPC);
C(cd) = sealCap(linkCap);
set_next_pc(newPC);
RETIRE_SUCCESS
```

2.9.5. JALR (RVY)

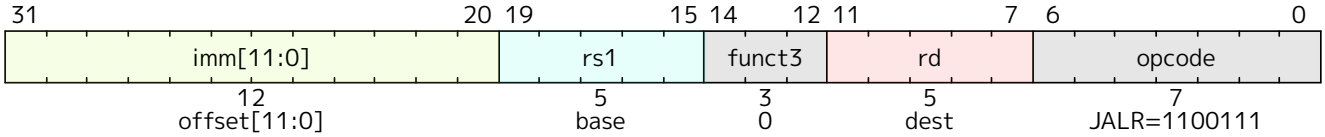
Synopsis

Jump to capability register, and link to capability register

Mnemonic

```
jalr rd, rs1, offset
```

Encoding



This instruction is modified from the version in RV32[IE]/RV64[IE].

Description

Indirect jump to the target capability in `rs1` with an address offset.

1. `rs1` is written to the target pc.
2. The target address is obtained by adding the sign-extended 12-bit I-immediate to `rs1.address`, then setting the least-significant bit of the result to zero.
3. Unseal the target pc if it is a Sealed entry point capability, `rs1.address[0]` is zero, and the I-immediate is zero.
4. Set the address of the target pc to the target address using the semantics of the YADDRW instruction.
5. The pc of the next instruction is sealed as a Sealed entry point capability and written to `rd`.
6. Jump to the target pc.

A future extension may raise an exception on the JALR (RVY) instruction itself if the target pc will raise a CHERI Instruction Access Fault at the target.

Operation

This listing is manually maintained rather than extracted from the Sail model.

```
let cs1_val = C(cs1);
let off : xlenbits = sign_extend(imm);
/* clear bit zero as for RISC-V JALR */
let newPC = [cs1_val.address + off with 0 = bitzero];
/* nextPC accounts for compressed instructions */
let (_, linkCap) = setCapAddr(PCC, nextPC);
C(cd) = sealCap(linkCap);
set_next_pc(newPC);
let (_, newPCC) = setCapAddr(cs1_val, newPC);
set_next_pcc(unsealCap(newPCC));
RETIRE_SUCCESS
```

2.10. RVY ISA Summary

2.10.1. RVY Added Instructions

Table 21. RVY added instructions

Mnemonic	Function
LY	Load capability
SY	Store capability
YADD	Capability pointer increment
YADDI	Capability pointer increment by immediate
YADDRW	Write capability address
YTAGR	Read capability tag
YPERMR	Read capability permissions
YMV	Capability register copy
YPERMC	Clear capability permissions
YHIR	Read capability metadata
YHIW	Write capability metadata and clear capability tag
YEQ	Capability equality comparison including capability tag
YSS	Capability is a subset
YBLD	Build capability
YUNSEALS	Unseal by superset reconstruction
YBNDSW	Write capability bounds
YBNDSWI	Write capability bounds by immediate
YBNDSRW	Write capability bounds, rounding up if required
YAMASK	Capability alignment mask
YBASER	Read capability base address
YLENR	Read capability length
YTYPER	Read capability type
YTOPR	Read capability top address

2.10.2. RVI (RVY modified behavior)

Table 22. RVI (RVY modified behavior) instructions

Mnemonic	Function
AUIPC (RVY)	Add upper immediate to pc
JAL (RVY)	Immediate offset jump, and link to capability register
JALR (RVY)	Jump to capability register, and link to capability register

Chapter 3. The RV64LYA Capability Base for RV64Y

This chapter describes the representation and properties of the RV64LYA capability-encoding format for RV64Y.



The format is closely modeled upon features from CHERI v9 (Watson et al., 2023), and the bounds encoding scheme is based upon CHERI Concentrate (Woodruff et al., 2019).

3.1. Capability-Encoding Format

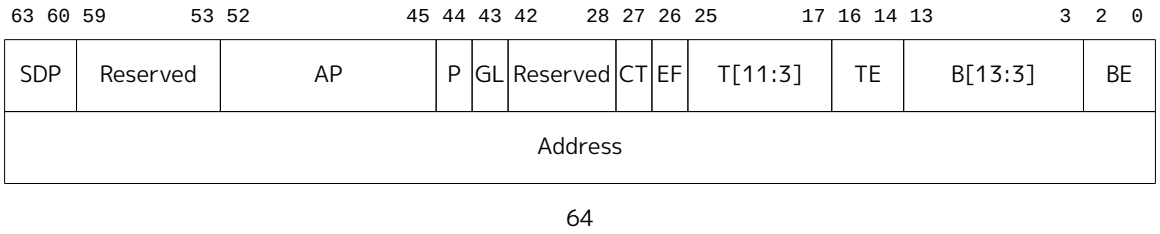


Figure 8. Capability-encoding format for RV64LYA



Reserved bits must be 0 in valid capabilities and are available for future extensions to RVY.

Figure 8 includes some fields which depend upon the presence of extensions:

Zyhybrid

When Zyhybrid is implemented:

- The P-bit is implemented, otherwise *reserved zero*.

Zylg

When Zylg is implemented:

- The GL flag is implemented, otherwise *reserved zero*.
- The AP-field includes LG-permission and SL-permission, otherwise both are *reserved zero*.

The bit ranges in Table 23 are relative to the XLEN-bit metadata, i.e., the upper XLEN bits of the capability.

Table 23. RV64LYA capability-encoding format metadata fields

Field	Bit range	Comment
SDP	63:60	Software-Defined Permissions.
AP	52:45	Architectural Permissions, see Table 25.
P	44	Pointer Mode (Zyhybrid only).
GL	43	Global (GL) flag (Zylg only).
CT	27	Capability Type field.
EF	26	Exponent format for the capability bounds decoding, see Table 26.
T[11:3]	25:17	Top bound address field for the capability bounds decoding, see Table 26.
TE	16:14	Top bound address or exponent field for the capability bounds decoding, see Table 26.
B[13:3]	13:3	Base bound address field for the capability bounds decoding, see Table 26.
BE	2:0	Base bound address or exponent field for the capability bounds decoding, see Table 26.

Field	Bit range	Comment
All other bits are *reserved*		



T[13:12], T[2:0], and B[2:0] are not specified by the metadata fields and are calculated as shown in Section 3.1.5.2.

3.1.1. Capability-Encoding Format Parameter Summary

Table 24. RV64LYA capability-encoding format parameters

Parameter	Default Value	Comment
MW	14	Mantissa width for the bounds encoding
EW	6	Exponent width for the bounds encoding
CAP_MAX_E	52	Maximum exponent value
enableL8	0	Whether the encoding format includes the L_8 bit
AP_MAX	All ones	Value of the AP-field giving maximum permissions
AP_ENC	full1	Encoding scheme for the AP-field - all combinations are representable
REP_RANGE	rc1	Whether out-of-bounds addresses are representable



All values in Table 24 match the default values specified in Table 7.

3.1.2. Architectural Permissions (AP) Encoding

The permissions field is 8 bits wide and is encoded using one bit per architectural permission as shown in Table 25. Some bit patterns are *reserved* because they encode illegal combinations of permissions (see Section 2.3.10.1).

Table 25. Encoding of architectural permissions for RV64LYA

Bit	Encoded permission
0	C-permission
1	W-permission
2	R-permission
3	X-permission
4	ASR-permission
5	LM-permission
6	LG-permission (Zylg)
7	SL-permission (Zylg)



Future extensions which define new permissions must augment Table 25.



There are 9 bits in total allocated to the AP-field and the P-bit.

- *The total valid combinations of C, W, R, LM, LG, and SL are 18.*
- *The total valid combinations of X, ASR, and P are 5.*
- *The two groups are fully orthogonal.*

Therefore, there are 90 valid combinations encoded in 9 bits.

> *Future extensions or capability-encoding formats may leverage these reserved patterns to implement a more compact joint AP-field and P-bit encoding that is fully compatible for all software to increase the number of reserved bits.*

3.1.3. Software-Defined Permissions (SDP) Encoding

The SDP-field is 4 bits wide. The value of the SDP-field bits of the YPERMR result maps 1:1 to the SDP-field in the capability.

3.1.4. Capability Type (CT) Encoding

The CT-field is as specified in Table 4.

The capabilities of this chapter define a 1-bit field for CT-field values; this field directly encodes the values `0` and `1`:

- The encoded value `0` indicates an *unsealed* type.
- The encoded value `1` indicates a Sealed entry point capability.

3.1.5. Bounds (EF, T, TE, B, BE, L_8) Encoding

3.1.5.1. Concept

The bounds encode the *base* and *top* addresses that constrain memory accesses. The capability can be used to access any memory location A in the range base ≤ A < top. The bounds are encoded in a compressed format, so it is not possible to encode any arbitrary combination of *base* and *top* addresses. An invalid capability is produced when attempting to construct a capability that is not *representable* because its bounds cannot be correctly encoded.

The variables related to the bounds decoding are shown in Table 26.

Table 26. RV64LYA capability bounds encoding fields

Variable	Description
T	Value substituted into the capability's address to decode the *top* bound address.
TE	Value either substituted into the exponent or the *top* bound address depending on EF.
B	Value substituted into the capability's address to decode the *base* bound address.
BE	Value either substituted into the exponent or the *base* bound address depending on EF.
E	Exponent that determines the position at which T and B are substituted into the capability's address.
L_8[1]	Value used to form either the MSB of the exponent or the MSB of the *top* bound address.
EF	Exponent format flag indicating the encoding for T, B, and E: EF=0: The exponent is calculated from TE and BE, and conditionally L_8, so it is 'internal'. EF=1: The exponent is zero; TE, BE, and L_8 form part of the address bounds.

[1] Only valid if `enableL8=1`.

The variables in Table 26 are either taken directly from the capability metadata (see Table 23) or, in the case of the upper and lower bits of `T` and `B`, are calculated from it. These variables are used as shown in the pseudocode below.

The bit widths of `T` and `B` are defined in terms of the mantissa width (`MW`), which is set depending on the capability-encoding format as shown in Table 24.

The exponent `E` indicates the position of `T` and `B` within the capability's address as described in Section 3.1.5.3. The bit width of the exponent (`EW`) is set depending on the encoding. The maximum value of the exponent is calculated as follows:

```
CAP_MAX_E = XLEN - MW + 2
```

The values of `EW` and `CAP_MAX_E` are shown in Table 24.

The address and bounds must be representable in valid capabilities (see Section 3.1.5.6).

3.1.5.2. Calculation of Variables

The metadata uses a floating-point representation to encode the compressed bounds relative to the capability address. The following subsections describe how the *base* and *top* bounds are decoded from the metadata.

The pseudocode below is normative. In this notation, `/` means "integer division", `[]` are the bit-select operators, `{}` is bit concatenation, `?:` is the conditional operator, and arithmetic is signed.

The pseudocode is intentionally simpler than the equivalent definitions in the SAIL model.

In the pseudocode below:

- `EF`, `T`, `TE`, `B`, and `BE` are all encoded in the capability metadata, see Table 23.
  - L_8 is also a capability metadata field, but only encoded if `enableL8=1`.
- `EW`, `MW`, `CAP_MAX_E`, and `enableL8` are all capability-encoding format parameters, see Table 24.
- `LCout` and `LMSB` are intermediate values used to reconstitute the top two bits of `T`, see below.

```
If EF = 1:
    E               = 0
    T[EW / 2 - 1:0] = TE
    B[EW / 2 - 1:0] = BE
    LCout           = (T[MW - 3:0] < B[MW - 3:0]) ? 1 : 0
    LMSB            = enableL8 ? L8 : 0
else:
    E               = CAP_MAX_E - ( enableL8 ? { L8, TE, BE } : { TE, BE } )
    T[EW / 2 - 1:0] = 0
    B[EW / 2 - 1:0] = 0
    LCout           = (T[MW - 3:EW / 2] < B[MW - 3:EW / 2]) ? 1 : 0
    LMSB            = 1
```

T and B are both MW bits wide, and are formed of different bit ranges:

- `T[MW - 1:MW - 2]` is specified below.

- `T[MW - 3:EW / 2]` is taken directly from the capability metadata.
- `T[EW / 2 - 1:0]` is as specified in the pseudocode above.
- `B[MW - 1:EW / 2]` is taken directly from the capability metadata.
- `B[EW / 2 - 1:0]` is as specified in the pseudocode above.

The design rationale for the two `EF` cases is as follows:

- `EF = 1`: The exponent is 0. When `enableL8=1`, L_8 encodes the MSB of the length, which can be used to derive `T[MW-1:MW-2]`, forming a full MW-bit `T` field.
- `EF = 0`: The exponent is *internal* with `E` stored in the lower bits of `T` and `B`, with L_8 used for the MSB of `E` when `enableL8=1`. `E` is chosen so that the most significant non-zero bit of the length of the region aligns with `T[MW - 2]` such that this bit is implied by `E`.

To save space in the encoding metadata, the top two bits of `T` are not stored directly but reconstituted as follows:

```
T[MW - 1:MW - 2] = B[MW - 1:MW - 2] + LCout + LMSB
```

This derivation relies on the equality `T = B + L`, where `L` is the encoded length of the capability. `LMSB` is the value of `L[MW - 2]`, which is known from the values of `EF` and `E` (as well as L_8 when `enableL8=1`). `LCout` is the carry out from the lower bits, which is implied by `T[MW - 3:0] < B[MW - 3:0]` since it is guaranteed that the *top* bound address is greater than or equal to the *base* bound address.

3.1.5.3. Format of the Bounds

The *base* and *top* bound addresses are derived from the capability address `a` by substituting `a[E + MW - 1:E]` with `B` and `T` respectively and clearing the lower `E` bits, then applying the Correction Factors c_b and c_t which are described below.

The bounds are formed of three sections as shown in Figure 9 and Figure 10.



XLEN and MW are constant parameters, but E is decoded from the capability metadata, and so the bit positions of the boundaries between the sections vary.



The top bound address has an additional bit so that the bounds can include the topmost byte of memory.

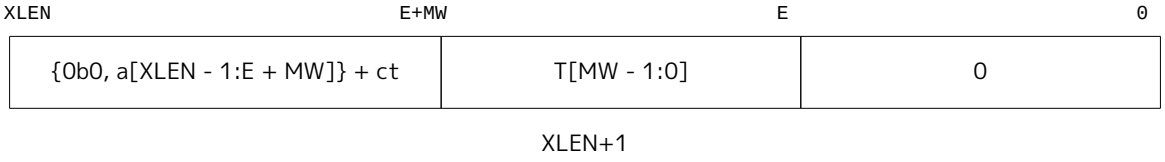


Figure 9. Decoding of the (XLEN+1)-bit top bound address

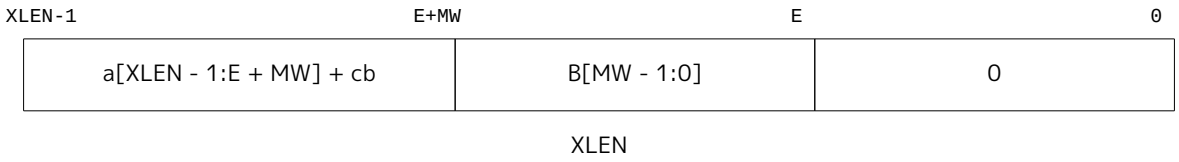


Figure 10. Decoding of the XLEN-bit base bound address

Figure 9 and Figure 10 include ranges which may not be present when the bounds are decoded:

- If `E = 0` the lower section does not exist.
- If `E+MW=XLEN` then the top section is only the least significant bit of c_t for the *top* bound address, and the top section does not exist for the *base* bound address.
- If `E+MW>XLEN` then neither top section exists, and so the bounds are calculated with no dependency on the address field *a*.

The compressed bounds encoding allows the address to roam over the *representable range* ($range_R$ in Figure 2 and Figure 11) while maintaining the original bounds. The overall effect is that the address can roam at least $2^{E+MW}/4$ bytes below the *base* bound address and at least $2^{E+MW}/4$ bytes above the *top* bound address while still allowing the bounds to be correctly decoded.

3.1.5.4. Correction Factors

Substituting `B`/`T` into the capability address, as described in Section 3.1.5.3 above, only decodes to the correct *base* or *top* bound address if `B`/`T` and the capability address both lie in the *same* $s=2^{E+MW}$-aligned block. In this case no correction is required.

If `B`/`T` and the capability address lie in *different* *s*-aligned blocks then *correction factors* are applied to move the computed bounds between the *s*-aligned blocks.

The *correction factors* are c_b for the *base* bound address and c_t for the *top* bound address.

In the example shown in Figure 11, the *top* bound address and the capability address lie in the same *s* -aligned block, but the *base* bound address does not; it is in the block below. Therefore, c_t is zero, but c_b is set to -1 to move the *base* bound address down into the *s*-aligned block below (i.e., from $space_U$ to $space_L$).

Therefore, adding the *correction factor* moves the computed bound into the correct *s*-aligned block. Each *correction factor* can have values of -1, 0, or +1, so that it is possible to move the computed bound up or down by one *s*-aligned block.

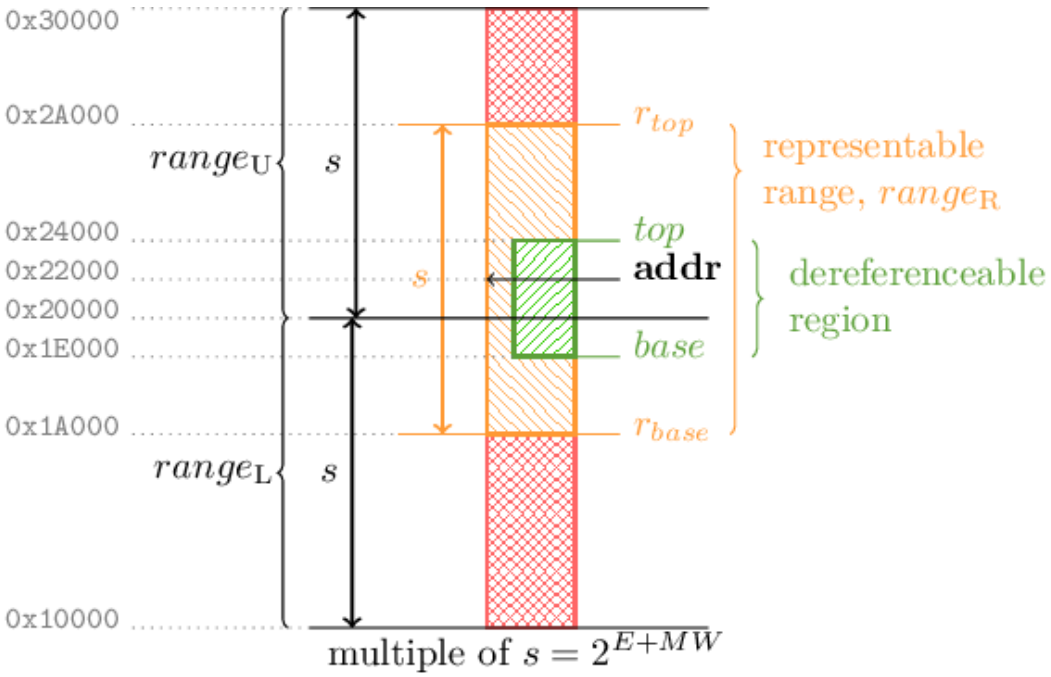


Figure 11. Example capability bounds that use the correction. The base bound address is an s-aligned block below the address, meaning the upper bits (above the mantissa) of the base bound address are smaller than the upper bits of the address. Therefore, c_b=-1. The address and top are in the same region, so c_t=0.

3.1.5.4.1. Correction Factor Calculations

`R` marks the lower boundary of the representable range (shown as r_{base} in Figure 11).

If `B`, `T`, or `A` is less than `R`, it is inferred that they lie in the space above the *s*-alignment boundary straddled by the representable region, labeled $space_U$ in Figure 11.

`A` and `R` are calculated as follows.

```
A[MW-1:0] = a[E + MW - 1:E]; // offset into s-aligned region from capability
address field
R[MW-1:0] = B - 2^MW-2;        // r-base which is 1/4 of the 2^MW aligned region
below B
```

The comparisons in Table 27 are unsigned.

The correction factors c_b and c_t are calculated as follows.

Mathematically, `B < R` *requires* `B[MW-1:MW-2] = 0`.

Table 27. Calculation of the top and base bound address corrections

A < R	T < R	B < R	Correction factors
false	false	false	c_t=0, c_b=0
	false	true	Impossible[1]
	true	false	c_t=+1, c_b=0
	true	true	c_t=+1, c_b=+1

A < R	T < R	B < R	Correction factors
true	false	false	c_t=-1, c_b=-1
	false	true	Impossible[1]
	true	false	c_t=0, c_b=-1
	true	true	c_t=0, c_b=0

[1] Since `B[MW-1:MW-2] = 0` in these cases, `R[MW-1:MW-2] = 3` as the value has wrapped.
As a result, `T[MW-1:MW-2]` is at most 2 if LMSB and LCout are both 1 (according to this formula).
Therefore, `T < R` always holds when `B < R`, ruling out c_t=-1, c_b=0 and c_t=0, c_b=+1.

Figure 12 shows the six reachable combinations of c_t and c_b that arise from this table.

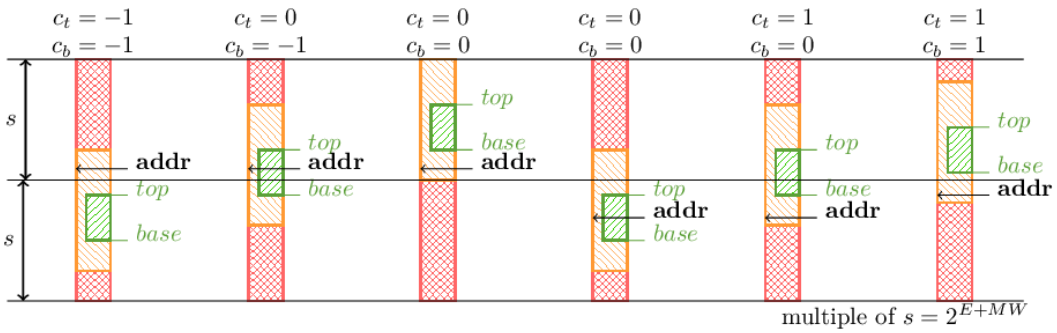


Figure 12. The reachable combinations of c_t and c_b.

3.1.5.4.2. Correction Factor Summary

In summary, for either bound:

- If the bound is below the *s*-aligned boundary and the capability address is above the *s*-aligned boundary, then set the correction factor to -1 to correct the bound downwards.
- If the bound and the capability address are within the same *s*-aligned boundary, then set the correction factor to 0 so the bound is unchanged.
- If the bound is above the *s*-aligned boundary and the capability address is below the *s*-aligned boundary, then set the correction factor to +1 to correct the bound upwards.

3.1.5.5. Top Bound Address MSB Correction

A capability has infinite bounds if its bounds cover the entire address space such that *base*=0 and *top*≥2^{XLEN}, i.e., *top* is an (XLEN+1)-bit value. However, *base* is an XLEN-bit value and this width mismatch introduces additional complications when decoding, so the following condition is required to correct *top* for capabilities whose Representable Range wraps the edge of the address space:

```
if ( (E < (CAP_MAX_E - 1)) && (top[XLEN: XLEN - 1] - {0b0, base[XLEN - 1]}) >=
0b10)
    top[XLEN] = !top[XLEN]
```

The comparison is unsigned.

That is, invert the most significant bit of *top* if the decoded *top* and *base* are one address space (2^{XLEN} bytes) or more apart, and E is too small to allow this.



A capability has infinite bounds if `E=CAP_MAX_E` *and it is not malformed (see Section 3.1.5.6); this check is equivalent to base=0 and top≥2^{XLEN}.*

3.1.5.6. Malformed Capability Bounds

A capability is *malformed* if its bounds cannot be correctly decoded. The following check indicates whether a capability is malformed. If `enableL8=1`, the L_8 bit is available in the capability-encoding format for extra precision when `EF=1`.

```
malformedMSB =  (E == CAP_MAX_E     && B         != 0)
             || (E == CAP_MAX_E - 1 && B[MW - 1] != 0)
malformedLSB =  (E  < 0) || (E == 0 && enableL8)
malformed    =  !EF && (malformedMSB || malformedLSB)
```



The above includes a special case for encodings where `enableL8=1`*. In this case, an* `EF=0` *encoding that decodes to* `E=0` *(with its implied* `LMSB` *of one) expresses the same bounds as the* `EF=1` *format with the L_8 bit set to 1. The* `EF=0, E=0` *encodings are therefore redundant, so they are declared malformed when* `enableL8=1`*, leaving* `EF=1` *as the canonical encoding.*

Capabilities with malformed bounds:

- Return both *base* and *top* bound addresses as zero, which affects instructions like YBASER.
- Cause certain manipulation instructions like YADDI to always set the capability tag of the result to zero.

3.2. Representable Range Check

The concept of the *representability check* was introduced in Section 2.3.7.

The definition of the check is:

- A source capability with address *a*, metadata *m* that decodes to give the bounds *base* and *top*.
- A derived capability with arbitrary address *a'* with the same metadata *m* that decodes to give the bounds *base'* and *top'*.

The address *a'* is within the source capability's *representable range* if `base == base' && top == top'`.

If the address *a'* is outside the *representable range*, then the derived capability is invalidated.

3.2.1. Practical Information

An artifact of the bounds encoding is that if the new address causes `top != top'`, then it is also the case that `base != base'`.

The converse is also true: if `base != base'`, then `top != top'`.

Therefore, for representable range checking, it is acceptable to either check `top == top'` or `base == base'`.

The *top* and *base* capability bounds are formed of two or three sections:

- Upper bits from the address

- This is only if the other sections do not fill the available bits (`E + MW < XLEN`)

- Middle bits from `T` and `B` decoded from the metadata
- Lower bits are set to 0
  - This is only if there is an internal exponent (`EF=0`)

Table 28. Composition of the decoded top bound address

Configuration	Upper Section (if E + MW < XLEN)	Middle Section	Lower Section
EF=0	address[XLEN-1:E + MW] + ct	T[MW - 1:0]	zeros(E)
EF=1, i.e., E=0	address[XLEN-1:MW] + ct	T[MW - 1:0]	

The *representable range* defines the range of addresses which do not corrupt the bounds encoding. The encoding was first introduced in Section 3.1.5, and is repeated in a different form in Table 28 to aid this description.

For the address to be valid for the current bounds encoding, the value in the *Upper Section* of Table 28 *must not change* as this will change the meaning of the bounds. This is because `T`, `B` and `E` will be unchanged for the source and destination capabilities. Therefore, the Middle and Lower sections of the bounds calculation are also unchanged for source and destination capabilities.

When E > CAP_MAX_E - 2, the calculation of the *top* bound address is entirely derived from `T` and `E`, which will be identical for both the source and destination capabilities, thus guaranteeing that `top == top'`. Likewise, with such values of E, the *base* bound address is entirely derived from `B` and `E` and therefore `base == base'`.

The calculation of the MSB of the *top* bound address may be inverted as specified in Section 3.1.5.5. Assuming `(E < (CAP_MAX_E - 1))`, the truth table for this inversion is as follows:

Table 29. Top bound address MSB inversion truth table

input_top[XLEN:XLEN-1]	base[XLEN-1]	output_top[XLEN:XLEN-1]
00	0	00
01	0	01
10	0	00
11	0	01
00	**1**	**10**
01	1	01
10	**1**	**10**
11	1	01

Inspection of Table 29 shows that `output_top[XLEN]` does not depend on `input_top[XLEN]` as:

- `output_top[XLEN] = {input_top[XLEN-1], base[XLEN-1]} == 0b01`.

This leads to the conclusions:

- If `top[XLEN-1] == top'[XLEN-1]` and `base[XLEN-1] == base'[XLEN-1]`, then it is guaranteed that `top[XLEN] == top'[XLEN]`.
- If `top[XLEN-1] != top'[XLEN-1]` or `base[XLEN-1] != base'[XLEN-1]`, then the representable check will fail regardless of checking `top[XLEN] == top'[XLEN]`.

Therefore, for the purpose of representable range checking, it is not required to check that `top[XLEN]==top'[XLEN]`.

Given that `top[XLEN]` is not part of the representable range check:

- When `E == CAP_MAX_E - 2`, `top[XLEN-1:E] == T[MW-1:0]` and `base[XLEN-1:E] == B[MW-1:0]`.

Since `T` and `B` are both derived from the capability's metadata and are constant, the representable range check always passes in this case too.

As a result:

- If E > CAP_MAX_E - 3, then the representability check always passes, *even though the bounds are only infinite if E = CAP_MAX_E*.

This gives a range of $s=2^{E+MW}$, as shown in Figure 2.

The gap between the object bounds and the bound of the representable range is always guaranteed to be at least 1/4 of `s`. This is represented by $R = B - 2^{MW-2}$ in Section 3.1.5. This gives useful guarantees, such that if an executed instruction is in pc bounds, then it is also guaranteed that the next linear instruction is *representable*.

3.3. Encoding of Special Capabilities

3.3.1. NULL Capability Encoding

The NULL capability is represented with 0 in all fields. This implies that it has no permissions and its exponent E is CAP_MAX_E (52). The exponent value causes its bounds to cover the entire address space, but no permissions are granted and so it cannot successfully be used to grant authorization.

Table 30. Field values of the NULL capability

Field	Value	Comment
Capability Tag	zero	Capability is not valid
SDP	zeros	Grants no permissions
AP	zeros	Grants no permissions
CT	zero	Unsealed
EF	zero	Internal exponent format
L_8[1]	zero	Top bound address reconstruction bit
T	zeros	Top bound address bits
TE	zeros	Exponent bits
B	zeros	Base bound address bits
BE	zeros	Exponent bits
Address	zeros	Capability address
Reserved	zeros	All reserved fields

[1] Only present if `enableL8=1` as defined by the capability-encoding format.

Permissions added by extensions (such as those of Zylg) are presumed absent in NULL capabilities.

3.3.2. Infinite Capability Encoding

This encoding is for an *Infinite Capability* value, which grants all permissions, has infinite bounds, and is both a Root Executable and a Root Data capability.

Table 31. Field values of the Infinite capability

Field	Value	Comment
Capability Tag	one	Capability is valid
SDP	ones	Grants all permissions
AP	AP_MAX	Grants all permissions
CT	zero	Unsealed
EF	zero	Internal exponent format
L_8[1]	zero	Top bound address reconstruction bit
T	zeros	Top bound address bits
TE	zeros	Exponent bits
B	zeros	Base bound address bits
BE	zeros	Exponent bits
Address	any[2]	Capability address
Reserved	zeros	All reserved fields

[1] Only present if `enableL8=1` as defined by the capability-encoding format.

[2] If an infinite capability is used as a constant in either hardware or software, then the address field will typically be set to 0. If the address field is non-zero, then it is still referred to as the *Infinite Capability*, and it still authorizes all memory accesses.



Permissions added by extensions (such as those of Zylg) are presumed present in the Infinite Capability, which may involve redefining the AP_MAX value.

Chapter 4. Zca (RVY Added and Modified Instructions)

Zcd (RV64) is incompatible with the RVY base ISA, as its encodings are reused for the capability loads and stores below. On harts that also implement Zyhybrid, these encodings revert to Zcd when executing in *Integral Pointer Mode* (see CHERI Pointer Mode).

Table 32. Zca (RVY added instructions) instruction extension

Mnemonic	Function
C.LYSP	Load capability stack pointer relative, 16-bit encoding
C.SYSP	Store capability stack pointer relative, 16-bit encoding
C.LY	Load capability, 16-bit encoding
C.SY	Store capability, 16-bit encoding

4.1. RV64 / RV64Y RVC Load/Store Mapping Summary

Table 33. 16-bit load/store instruction mapping in RV64I

Encoding		Supported Extensions		
[15:13]	[1:0]	Zca	Zcd	Zcmp/ Zcmt
111	00	C.SD	N/A	N/A
011	00	C.LD	N/A	N/A
111	10	C.SDSP	N/A	N/A
011	10	C.LDSP	N/A	N/A
101	00	N/A	C.FSD	reserved
001	00	N/A	C.FLD	reserved
101	10	N/A	C.FSDSP	Zcmp/ Zcmt
001	10	N/A	C.FLDSP	reserved

Table 34. 16-bit load/store instruction mapping in RV64Y

Encoding		Supported Extensions
[15:13]	[1:0]	Zca
111	00	C.SD
011	00	C.LD
111	10	C.SDSP
011	10	C.LDSP
101	00	C.SY
001	00	C.LY
101	10	C.SYSP
001	10	C.LYSP



Zcd, Zcmp and Zcmt are incompatible with RV64Y.

4.1.1. C.LY

See C.LYSP.

4.1.2. C.LYSP

Synopsis

Capability loads (C.LY, C.LYSP), 16-bit encodings

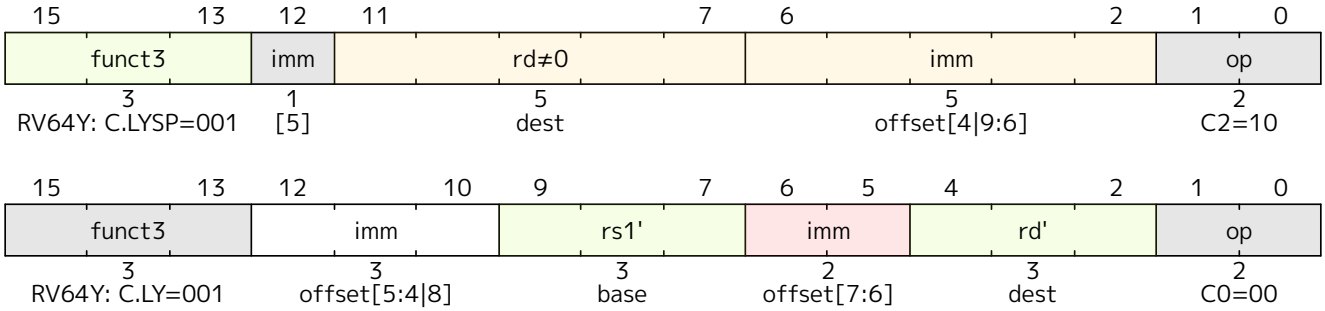
Mnemonics

```
C.LY rd', offset(rs1')
C.LYSP rd, offset(sp)
```

Expansions

```
LY rd', offset(rs1')
LY rd, offset(sp)
```

Encoding



Description

For C.LY the authorizing capability is `rs1'`.

For C.LYSP, the authorizing capability is `sp`.

The effective address of the memory access is obtained by adding the address of the authorizing capability to the zero-extended offset.

Load data from memory in the same manner as LY.

Prerequisites

Zca, RVY

Operation (after expansion to 32-bit encodings)

See LY.

4.1.3. C.SY

See C.SYSP.

4.1.4. C.SYSP

Synopsis

Capability stores (C.SY, C.SYSP), 16-bit encodings

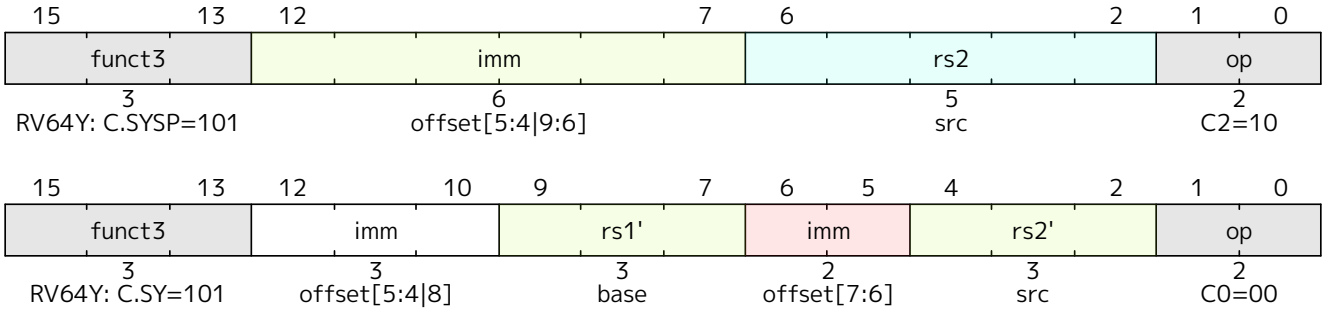
Mnemonics

```
C.SY rs2', offset(rs1')
C.SYSP rs2, offset(sp)
```

Expansions

```
SY rs2', offset(rs1')
SY rs2, offset(sp)
```

Encoding



Description

For C.SY the authorizing capability is `rs1'`.

For C.SYSP, the authorizing capability is `sp`.

The store data is in `rs2'`.

The effective address of the memory access is obtained by adding the address of the authorizing capability to the zero-extended offset.

Store data to the memory in the same manner as SY.

Prerequisites

Zca, RVY

Operation (after expansion to 32-bit encodings)

See SY.

4.2. Zca (RVY modified behavior)

An RVY core that supports Zca also supports Zca (RVY modified behavior) which *modifies* the behavior of some instructions.



C.YMV is renamed from `C.MV` to avoid ambiguity in disassembly.

Table 35. Zca (RVY modified behavior) instruction extension

Mnemonic	Function
C.ADDI16SP (RVY)	Stack pointer increment in blocks of 16, 16-bit encoding
C.ADDI4SPN (RVY)	Stack pointer increment in blocks of 4, 16-bit encoding
C.YMV	Capability register copy, 16-bit encoding
C.JALR (RVY)	Jump to capability register, and link to capability register, 16-bit encoding
C.JR (RVY)	Jump to capability register, 16-bit encoding

4.2.1. C.ADDI16SP (RVY)

Synopsis

Stack pointer increment in blocks of 16, 16-bit encoding

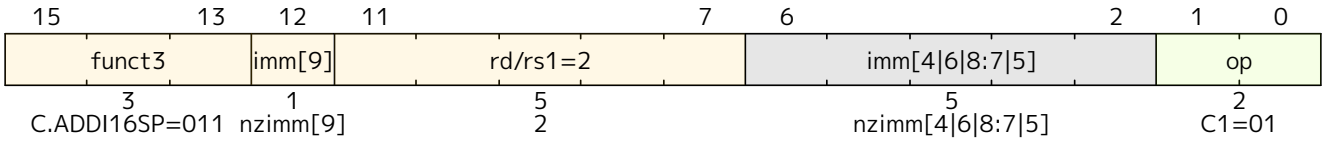
Mnemonic

```
C.ADDI16SP nzimm
```

Expansion

```
YADDI sp, sp, nzimm
```

Encoding



This instruction is modified from the version in RV32[IE]/RV64[IE].

Description

Add the non-zero sign-extended 6-bit immediate to `sp.address` (`sp=x2`), where the immediate is scaled to represent multiples of 16 in the range (-512,496).

All other details, including the conditions for clearing `sp.tag`, are as defined by YADDI for the expansion shown above.

Prerequisites

Zca, RVY

Operation

```
execute(CADDI(sp, sp, sign_extend(nzimm)))
```

4.2.2. C.ADDI4SPN (RVY)

Synopsis

Stack pointer increment in blocks of 4, 16-bit encoding

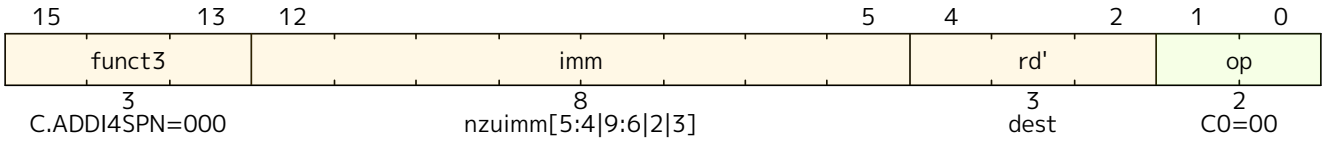
Mnemonic

```
C.ADDI4SPN rd', nzuimm
```

Expansion

```
YADDI rd', sp, nzuimm
```

Encoding



This instruction is modified from the version in RV32[IE]/RV64[IE].

Description

Copy `sp` (`sp=x2`) to `rd'`. Add a zero-extended non-zero immediate, scaled by 4, to `rd'.address`.

All other details, including the conditions for clearing `rd'.tag`, are as defined by YADDI for the expansion shown above.

Prerequisites

Zca, RVY

Operation

```
let cd = creg2reg_idx(cdc) in
execute(CADDI(cd, sp, zero_extend(nzuimm)))
```

4.2.3. C.YMV

Synopsis

Capability register copy, 16-bit encoding

Mnemonic

`C.YMV rd, rs2`

Expansion

`YMV rd, rs1`

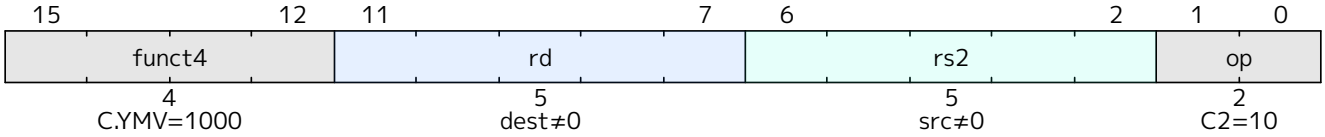


`rs2` in the 16-bit encoding is mapped to `rs1` in the expansion due to the different encoding formats. This follows the same pattern as `C.MV` expanding to `MV` (`ADDI rd, rs1, 0`).



`C.MV` is remapped to copy YLEN-bit registers for RVY. The mnemonic is changed to avoid ambiguity about whether the copy is XLEN or YLEN bits.

Encoding





This instruction is modified from the version in RV32[IE]/RV64[IE].

Description

Copy `rs2` to `rd`.

This instruction can propagate valid capabilities which fail integrity checks.

Prerequisites

Zca, RVY

Operation (after expansion to 32-bit encoding)

See YMV.

4.2.4. C.JR (RVY)

Synopsis

Jump to capability register, 16-bit encoding

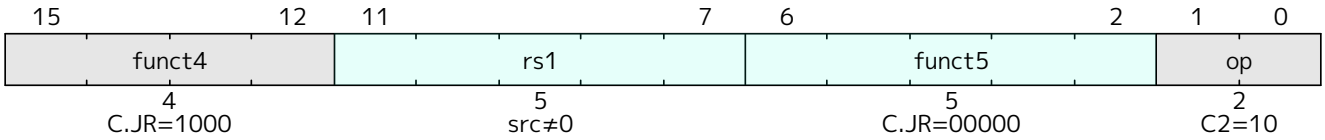
Mnemonic

```
C.JR rs1
```

Expansion

```
JALR x0, 0(rs1)
```

Encoding



This instruction is modified from the version in RV32[IE]/RV64[IE].

Description

See JALR (RVY) for execution of the expanded instruction as shown above. Note that the `offset` is zero in the expansion.

Prerequisites

Zca, RVY

Operation (after expansion to 32-bit encodings)

See JALR (RVY).

4.2.5. C.JALR (RVY)

Synopsis

Jump to capability register, and link to capability register, 16-bit encoding

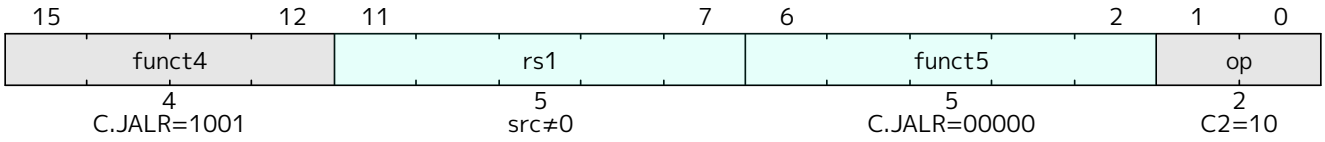
Mnemonic

```
C.JALR rs1
```

Expansion

```
JALR x1, 0(rs1)
```

Encoding



This instruction is modified from the version in RV32[IE]/RV64[IE].

Description

See JALR (RVY) for execution of the expanded instruction as shown above. Note that the `offset` is zero in the expansion.

Exceptions

See JALR (RVY).

Prerequisites

Zca, RVY

Operation (after expansion to 32-bit encodings)

See JALR (RVY).

4.2.6. Zca HINT Encodings

In RVI/RVE, Zca arithmetic encodings with *rd*≠`x0` and a zero immediate or shift amount are defined as HINTs because they leave the integer value in *rd* unchanged. In RVY, executing an integer arithmetic instruction that writes *rd*≠`x0` sets the destination register's metadata and capability tag to zero, so the following encodings are instead *reserved* (leaving only Zca HINT encodings with *rd*=`x0` as HINTs):

- `C.ADDI` with *rd*≠`x0` and *imm*=0
- `C.SLLI` with *rd*≠`x0` and *shamt*=0
- `C.SRLI` with *shamt*=0
- `C.SRAI` with *shamt*=0

Marking these encodings as reserved allows future extensions to allocate these code points consistently across RVI/RVE and RVY.

Chapter 5. Zalrsc (RVY added instructions)

Specifying RVY and Zalrsc adds atomic capability load and store instructions.

Table 36. Zalrsc (RVY added instructions) instruction extension

Mnemonic	Function
LR.Y	Load Reserved capability
SC.Y	Store Conditional capability

5.1. LR.Y

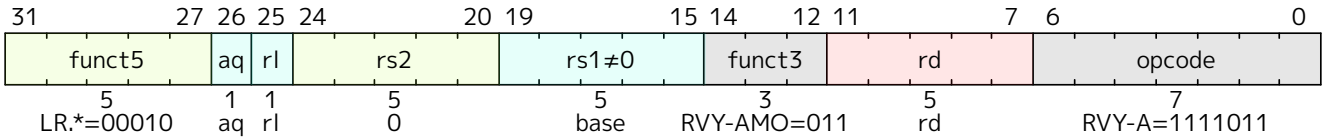
Synopsis

Load Reserved capability

Mnemonic

```
LR.Y rd, 0(rs1)
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0



Encodings with `rs1`=`x0` are RESERVED for use by future extensions, as `x0` is defined to always hold a NULL capability.

Description

`rs1.address` is the effective address of the memory access.

Authorize the memory access with the capability in `rs1`.

Load a naturally aligned YLEN-bit data value from memory.

Set the reservation as for LR.W/D.

Use the YLEN-bit data and the capability tag to determine the value of `rd` as specified by the LY instruction.

This instruction can propagate valid capabilities which fail integrity checks.

Exceptions

Raise a load access fault exception when the effective address is not naturally aligned.

Exceptions occur when the authorizing capability fails one of the checks listed below:

Kind	Reason
CHERI Load Access Fault	Authorizing capability tag is set to 0.
CHERI Load Access Fault	Authorizing capability is sealed.
CHERI Load Access Fault	Authorizing capability does not grant the necessary permissions. R-permission is required.
CHERI Load Access Fault	At least one byte accessed is outside the authorizing capability bounds, or the bounds could not be decoded.
CHERI Load Access Fault	Authorizing capability failed any integrity check.

Prerequisites

Zalrsc, RVY

Operation

```
  let (auth_val, vaddr) = get_cheri_mode_cap_addr(rs1_cs1, zeros());

  match check_and_handle_load_vaddr_for_triggers(vaddr, get_arch_pc()) {
    Some (ret) => return ret,
    None () => ()
  };
  if not(capTaggedAndReservedValid(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_TagViolation);
    RETIRE_FAIL
  } else if capIsSealed(auth_val) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_SealViolation);
    RETIRE_FAIL
  } else if not(canR(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_PermissionViolation);
    RETIRE_FAIL
  } else if not(validAddrRange(vaddr, cap_size) |
capBoundsInfinite(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_InvalidAddressViolation);
    RETIRE_FAIL
  } else if not(inCapBounds(auth_val, vaddr, cap_size)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_LengthViolation);
    RETIRE_FAIL
  } else if not(is_aligned_addr(vaddr, cap_size)) then {
    handle_mem_exception(vaddr, E_Load_Addr_Align());
    RETIRE_FAIL
  } else match translateAddr(vaddr, Read(Cap)) {
    TR_Failure(E_Extension(_)) => { internal_error(__FILE__, __LINE__,
"unexpected cheri exception for cap load") },
    TR_Failure(e) => { handle_mem_exception(vaddr, e); RETIRE_FAIL },
    TR_Address(addr, pbmt, ptw_info) => {
      let c = mem_read_cap(addr, pbmt, aq, aq & rl, false);
      match c {
        MemValue(v) => {
          let cr = if ptw_info.ptw_lc == PTW_LC_CLEAR
                   then clearTag(v) /* strip the tag */
                   else {
                     /* the Sail model currently reserves physical addresses
*/
                     load_reservation(addr);
                     clearTagIf(v, not(canC(auth_val)))
                   };
          C(cd) = legalizeLM(cr, auth_val);
          RETIRE_SUCCESS
        },
        MemException(e) => {handle_mem_exception(vaddr, e); RETIRE_FAIL }
      }
    }
  }
```

5.2. SC.Y

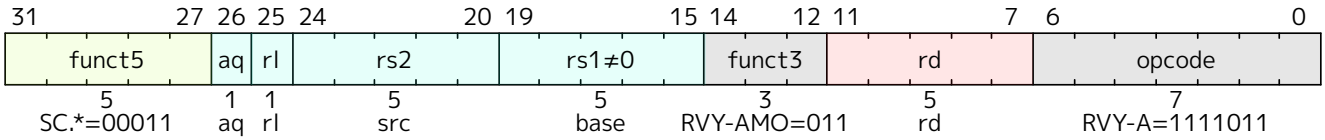
Synopsis

Store Conditional capability

Mnemonic

```
SC.Y rd, rs2, 0(rs1)
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0



Encodings with `rs1`=`x0` *are RESERVED for use by future extensions, as* `x0` *is defined to always hold a NULL capability.*

Description

`rs1.address` is the effective address of the memory access.

Authorize the memory access with the capability in `rs1`.

Conditionally store the YLEN-bit value in `rs2`, together with its capability tag, to the naturally aligned memory location, following the same rules as SC.W.

Set `rd` to zero for success or non-zero for failure.

The written capability tag may be cleared following the same modification rules as SY.

This instruction can propagate valid capabilities which fail integrity checks.

Exceptions

Raise a store/AMO access fault exception when the effective address is not naturally aligned.

Exceptions occur when the authorizing capability fails one of the checks listed below:

Kind	Reason
CHERI Store/AMO Access Fault	Authorizing capability tag is set to 0.
CHERI Store/AMO Access Fault	Authorizing capability is sealed.
CHERI Store/AMO Access Fault	Authorizing capability does not grant the necessary permissions. W-permission is required.
CHERI Store/AMO Access Fault	At least one byte accessed is outside the authorizing capability bounds, or the bounds could not be decoded.
CHERI Store/AMO Access Fault	Authorizing capability failed any integrity check.

Prerequisites

Zalrsc, RVY

Operation

```
  let (auth_val, vaddr) = get_cheri_mode_cap_addr(rs1_cs1, zeros());
  let cs2_val = C(cs2);
  let cs2_val = clearTagIf(cs2_val, not(canC(auth_val)));

  match check_and_handle_store_vaddr_for_triggers(vaddr, get_arch_pc()) {
    Some (ret) => return ret,
    None () => ()
  };
  if speculate_conditional () == false then {
    /* should only happen in rmem
     * rmem: allow SC to fail very early
     */
    X(rd) = zero_extend(0b1);
    return RETIRE_SUCCESS
  };
  if not(capTaggedAndReservedValid(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_TagViolation);
    RETIRE_FAIL
  } else if capIsSealed(auth_val) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_SealViolation);
    RETIRE_FAIL
  } else if not(canW(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_PermissionViolation);
    RETIRE_FAIL
  } else if not(validAddrRange(vaddr, cap_size) |
capBoundsInfinite(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_InvalidAddressViolation);
    RETIRE_FAIL
  } else if not(inCapBounds(auth_val, vaddr, cap_size)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_LengthViolation);
    RETIRE_FAIL
  } else if not(is_aligned_addr(vaddr, cap_size)) then {
    handle_mem_exception(vaddr, E_SAMO_Addr_Align());
    RETIRE_FAIL
  } else {
    match translateAddr(vaddr, Write(if cs2_val.tag then Cap else Data)) {
      TR_Failure(e) => { handle_mem_exception(vaddr, e); RETIRE_FAIL },
      TR_Address(addr, pbmt, _) => {
        if not(match_reservation(addr)) then {
          /* cannot happen in rmem */
          X(rd) = zero_extend(0b1);
          cancel_reservation();
          RETIRE_SUCCESS
        } else {
          let eares : MemoryOpResult(unit) = mem_write_ea_cap(addr, aq & rl,
rl, false);
          match (eares) {
            MemException(e) => { handle_mem_exception(vaddr, e); RETIRE_FAIL
},
            MemValue(_) => {
              let res : MemoryOpResult(bool) = mem_write_cap(addr, pbmt,
```

```
cs2_val, aq & rl, rl, false);
              match (res) {
                MemValue(true)  => {
                  X(rd) = zero_extend(0b0);
                  cancel_reservation();
                  RETIRE_SUCCESS
                },
                MemValue(false) => {
                  X(rd) = zero_extend(0b1);
                  cancel_reservation();
                  RETIRE_SUCCESS
                },
                MemException(e) => {
                  handle_mem_exception(vaddr, e);
                  RETIRE_FAIL
                }
              }
            }
          }
        }
      }
    }
  }
```

Chapter 6. Zaamo (RVY added instructions)

Specifying RVY and Zaamo gives Zaamo (RVY added instructions) functionality, which adds atomic capability swap.

Table 37. Zaamo (RVY added instructions) instruction extension

Mnemonic	Function
AMOSWAP.Y	Atomic swap of capabilities

6.1. AMOSWAP.Y

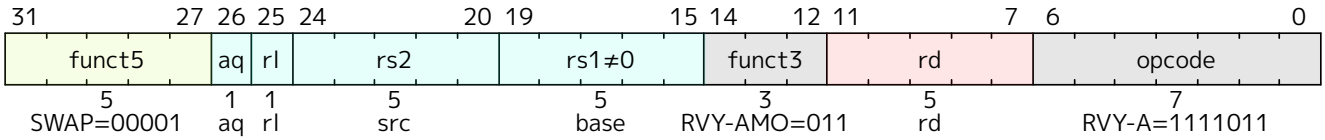
Synopsis

Atomic swap of capabilities

Mnemonic

```
AMOSWAP.Y rd, rs2, 0(rs1)
```

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

Encodings with `rs1`=`x0` *are RESERVED for use by future extensions, as* `x0` *is defined to always hold a NULL capability.*

Description

Atomic swap of capabilities, authorized by the capability in `rs1`.

The operation is equivalent to an atomically executed sequence of:

```
LY rd, 0(rs1)
```

```
SY rs2, 0(rs1)
```

`rd` is only updated if no exceptions are raised. The value stored to memory is the original value of `rs2`, even if `rd=rs1` or `rd=rs2`.

Permissions

Requires R-permission and W-permission in the authorizing capability. If C-permission is not granted, then the capability tag will be stored as zero.

Exceptions

Raise a store/AMO access fault exception when the effective address is not naturally aligned.

Exceptions occur when the authorizing capability fails one of the checks listed below:

Kind	Reason
CHERI Store/AMO Access Fault	Authorizing capability tag is set to 0.
CHERI Store/AMO Access Fault	Authorizing capability is sealed.
CHERI Store/AMO Access Fault	Authorizing capability does not grant the necessary permissions. W-permission and R-permission are both required.
CHERI Store/AMO Access Fault	At least one byte accessed is outside the authorizing capability bounds, or the bounds could not be decoded.
CHERI Store/AMO Access Fault	Authorizing capability failed any integrity check.

Prerequisites

Zaamo, RVY

Operation

```
  let (auth_val, vaddr) = get_cheri_mode_cap_addr(rs1_cs1, zeros());
  let cs2_val = C(cs2);
  let cs2_val = clearTagIf(cs2_val, not(canC(auth_val)));

  match check_and_handle_amo_vaddr_for_triggers(vaddr, get_arch_pc()) {
    Some (ret) => return ret,
    None () => ()
  };
  if not(capTaggedAndReservedValid(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_TagViolation);
    RETIRE_FAIL
  } else if capIsSealed(auth_val) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_SealViolation);
    RETIRE_FAIL
  } else if not(canR(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_PermissionViolation);
    RETIRE_FAIL
  } else if not(canW(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_PermissionViolation);
    RETIRE_FAIL
  } else if not(validAddrRange(vaddr, cap_size) |
capBoundsInfinite(auth_val)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_InvalidAddressViolation);
    RETIRE_FAIL
  } else if not(inCapBounds(auth_val, vaddr, cap_size)) then {
    handle_cheri_exception(CapCheckType_Data, CapEx_LengthViolation);
    RETIRE_FAIL
  } else if not(is_aligned_addr(vaddr, cap_size)) then {
    handle_mem_exception(vaddr, E_SAMO_Addr_Align());
    RETIRE_FAIL
  } else {
    match translateAddr(vaddr, ReadWrite(Cap, if cs2_val.tag then Cap else
Data)) {
      TR_Failure(e) => { handle_mem_exception(vaddr, e); RETIRE_FAIL },
      TR_Address(addr, pbmt, ptw_info) => {
        let eares : MemoryOpResult(unit) = mem_write_ea_cap(addr, aq & rl,
rl, false);
        match (eares) {
          MemException(e) => { handle_mem_exception(vaddr, e); RETIRE_FAIL },
          MemValue(_) => {
            let c = mem_read_cap(addr, pbmt, aq, aq & rl, false);
            match c {
              MemValue(v) => {
                let wres : MemoryOpResult(bool) = mem_write_cap(addr, pbmt,
cs2_val, aq & rl, rl, false);
                match wres {
                  MemValue(_) => {
                    let cr = clearTagIf(v, ptw_info.ptw_lc == PTW_LC_CLEAR |
not(canC(auth_val)));
```

```
                    C(cd) = legalizeLM(cr, auth_val);
                    RETIRE_SUCCESS
                  },
                  MemException(e) => {
                    handle_mem_exception(vaddr, e);
                    RETIRE_FAIL
                  }
                }
              },
              MemException(e) => { handle_mem_exception(vaddr, e);
RETIRE_FAIL }
            }
          }
        }
      }
    }
  }
```

Chapter 7. Zba (RVY added instructions)

Specifying RVY and Zba gives Zba (RVY added instructions) functionality, which adds more instructions.

Table 38. Zba (RVY added instructions) instruction extension

Mnemonic	Function
YSH1ADD	shift and add, representability check
YSH2ADD	shift and add, representability check
YSH3ADD	shift and add, representability check
YSH4ADD	shift and add, representability check
YSH1ADD.UW	shift and add unsigned word, representability check
YSH2ADD.UW	shift and add unsigned word, representability check
YSH3ADD.UW	shift and add unsigned word, representability check
YSH4ADD.UW	shift and add unsigned word, representability check



There is currently no RVY equivalent for `add.uw`*. A future extension may add such an instruction.*

7.1. YSH1ADD, YSH2ADD, YSH3ADD, YSH4ADD (RV64Y)

Synopsis

Shift by *n* and add for address generation (YSH1ADD, YSH2ADD, YSH3ADD, YSH4ADD)

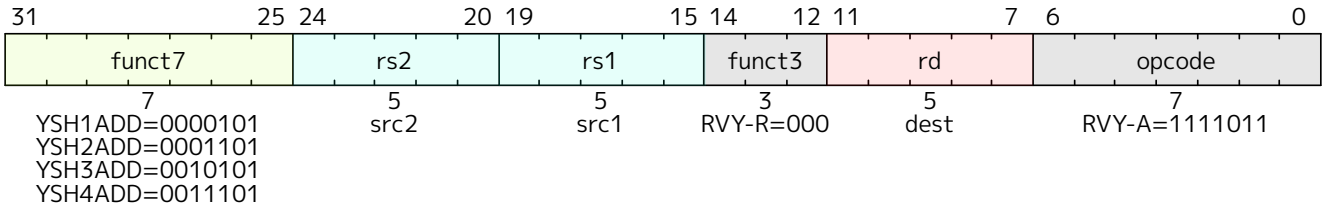
Mnemonics (RVY)

```
YSH1ADD rd, rs1, rs2
YSH2ADD rd, rs1, rs2
YSH3ADD rd, rs1, rs2
```

Mnemonics (RV64Y)

```
YSH4ADD rd, rs1, rs2
```

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Copy the capability in `rs2` to `rd`.

Increment `rd.address` by `rs1` shifted left by *n* bit positions.

Set `rd.tag=0` if `rs2` is sealed.

Set `rd.tag=0` if the resulting capability cannot be represented exactly.

Set `rd.tag=0` if `rs2` fails any integrity checks.

Operation

This listing is manually maintained rather than extracted from the Sail model.

```
let rs1_val = X(rs1);
let cs2_val = C(cs2);
let shamt : range(1,4) = match op {
  RISCV_SH1ADD => 1,
  RISCV_SH2ADD => 2,
  RISCV_SH3ADD => 3,
  RISCV_SH4ADD => 4,
};
let result = incCapAddrChecked(cs2_val, rs1_val << shamt);
C(cd) = result;
RETIRE_SUCCESS
```

7.2. YSH1ADD.UW, YSH2ADD.UW, YSH3ADD.UW, YSH4ADD.UW (RV64Y)

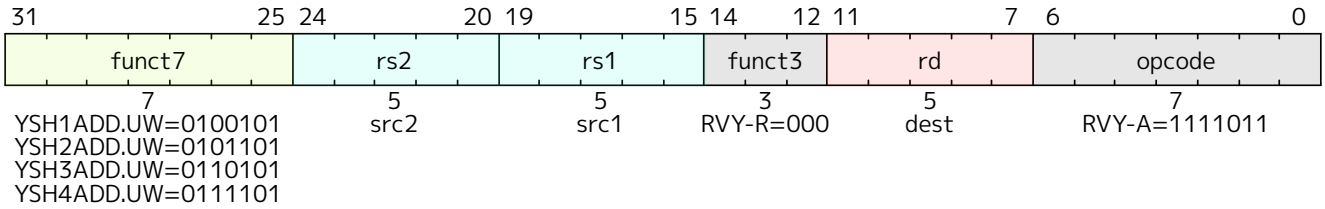
Synopsis

Shift by *n* and add unsigned words for address generation (YSH1ADD.UW, YSH2ADD.UW, YSH3ADD.UW, YSH4ADD.UW)

Mnemonics (RV64Y)

```
YSH1ADD.UW rd, rs1, rs2
YSH2ADD.UW rd, rs1, rs2
YSH3ADD.UW rd, rs1, rs2
YSH4ADD.UW rd, rs1, rs2
```

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Copy the capability in `rs2` to `rd`.

Increment `rd.address` by the unsigned word `rs1` shifted left by *n* bit positions.

Set `rd.tag=0` if `rs2` is sealed.

Set `rd.tag=0` if the resulting capability cannot be represented exactly.

Set `rd.tag=0` if `rs2` fails any integrity checks.

Operation

This listing is manually maintained rather than extracted from the Sail model.

```
let rs1_val = X(rs1);
let cs2_val = C(cs2);
let shamt : range(1,4) = match op {
  RISCV_SH1ADDUW => 1,
  RISCV_SH2ADDUW => 2,
  RISCV_SH3ADDUW => 3,
  RISCV_SH4ADDUW => 4,
};
let result = incCapAddrChecked(cs2_val, zero_extend(rs1_val[31..0]) << shamt);
C(cd) = result;
RETIRE_SUCCESS
```

Chapter 8. Zicbom (RVY modified behavior)

Specifying RVY and Zicbom gives Zicbom (RVY modified behavior) functionality, which extends the checking.

Table 39. Zicbom (RVY modified behavior) instruction extension

Mnemonic	Function
CBO.INVAL (RVY)	Cache block invalidate (implemented as clean)
CBO.CLEAN (RVY)	Cache block clean
CBO.FLUSH (RVY)	Cache block flush

8.1. CBO.CLEAN (RVY)

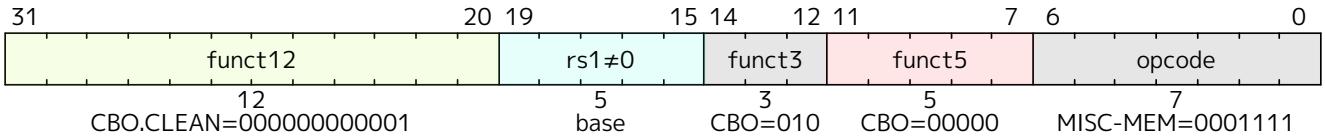
Synopsis

Perform a clean operation on a cache block

Mnemonic

```
CBO.CLEAN 0(rs1)
```

Encoding



Description

A CBO.CLEAN instruction performs a clean operation on the cache block whose effective address is the base address specified in `rs1`. The authorizing capability for this operation is `rs1`.

Exceptions

Kind	Reason
CHERI Store/AMO Access Fault	Authorizing capability tag is set to 0.
CHERI Store/AMO Access Fault	Authorizing capability is sealed.
CHERI Store/AMO Access Fault	Authorizing capability does not grant the necessary permissions. W-permission and R-permission are both required.
CHERI Store/AMO Access Fault	None of the bytes accessed are within the bounds, or the bounds could not be decoded.
CHERI Store/AMO Access Fault	Authorizing capability failed any integrity check.



CBO.CLEAN and CBO.FLUSH only require one byte of the cache block to be within the authorizing capability's bounds because these operations do not reveal or modify any data, so performing them on a partially covered cache block is harmless. CBO.INVAL and CBO.ZERO destroy or overwrite the contents of the entire cache block and therefore require every byte of the cache block to be within bounds.

Prerequisites

Zicbom, RVY

Operation



This listing is manually maintained rather than extracted from the Sail model.

```
let cache_block_size = 2 ^ plat_cache_block_size_exp();
let negative_offset = (X(cs1) &
~(zero_extend(ones(plat_cache_block_size_exp()))) - X(cs1);
match ext_data_get_addr(cs1, negative_offset, Cache(CleanFlush),
cache_block_size) {
  Ext_DataAddr_Error(e) => { ext_handle_data_check_error(e); RETIRE_FAIL },
  Ext_DataAddr_OK(_)    => RETIRE_SUCCESS
}
```

8.2. CBO.FLUSH (RVY)

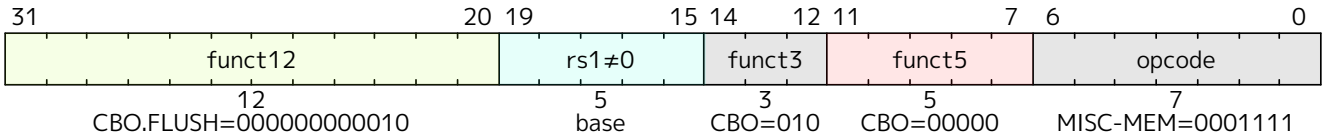
Synopsis

Perform a flush operation on a cache block

Mnemonic

```
CBO.FLUSH 0(rs1)
```

Encoding



Description

A CBO.FLUSH instruction performs a flush operation on the cache block whose effective address is the base address specified in `rs1`. The authorizing capability for this operation is `rs1`.

Exceptions

Kind	Reason
CHERI Store/AMO Access Fault	Authorizing capability tag is set to 0.
CHERI Store/AMO Access Fault	Authorizing capability is sealed.
CHERI Store/AMO Access Fault	Authorizing capability does not grant the necessary permissions. W-permission and R-permission are both required.
CHERI Store/AMO Access Fault	None of the bytes accessed are within the bounds, or the bounds could not be decoded.
CHERI Store/AMO Access Fault	Authorizing capability failed any integrity check.

CBO.CLEAN and CBO.FLUSH only require one byte of the cache block to be within the authorizing capability's bounds because these operations do not reveal or modify any data, so performing them on a partially covered cache block is harmless. CBO.INVAL and CBO.ZERO destroy or overwrite the contents of the entire cache block and therefore require every byte of the cache block to be within bounds.

Prerequisites

Zicbom, RVY

Operation

This listing is manually maintained rather than extracted from the Sail model.

```
let cache_block_size = 2 ^ plat_cache_block_size_exp();
let negative_offset = (X(cs1) &
~(zero_extend(ones(plat_cache_block_size_exp()))) - X(cs1);
match ext_data_get_addr(cs1, negative_offset, Cache(CleanFlush),
cache_block_size) {
  Ext_DataAddr_Error(e) => { ext_handle_data_check_error(e); RETIRE_FAIL },
  Ext_DataAddr_OK(_)    => RETIRE_SUCCESS
}
```

8.3. CBO.INVAL (RVY)

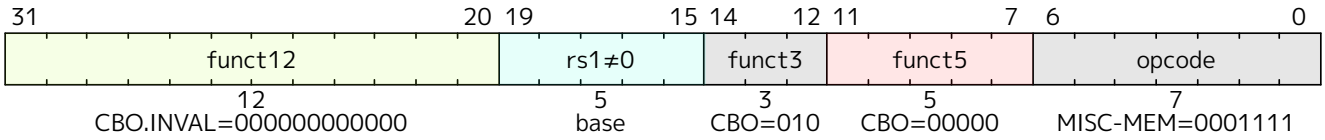
Synopsis

Perform an invalidate operation on a cache block

Mnemonic

```
CBO.INVAL 0(rs1)
```

Encoding



Description

A CBO.INVAL instruction performs an invalidate operation on the cache block whose effective address is the base address specified in `rs1`. The authorizing capability for this instruction is `rs1`.

Executing CBO.INVAL requires ASR-permission in pc, even when CSR state causes it to perform a flush operation instead of an invalidation.

An illegal instruction exception is raised if ASR-permission is not granted, matching the behavior of `xRET` and CSR accesses that require ASR-permission.

Exceptions

Kind	Reason
Illegal instruction	pc does not grant ASR-permission.
CHERI Store/AMO Access Fault	Authorizing capability tag is set to 0.
CHERI Store/AMO Access Fault	Authorizing capability is sealed.
CHERI Store/AMO Access Fault	Authorizing capability does not grant the necessary permissions. W-permission, R-permission are both required.
CHERI Store/AMO Access Fault	At least one byte accessed is outside the authorizing capability bounds, or the bounds could not be decoded.
CHERI Store/AMO Access Fault	Authorizing capability failed any integrity check.

CSR state controls whether CBO.INVAL performs cache block flushes instead of invalidations for less privileged modes.

Invalidating a cache block can re-expose capabilities previously stored to it after the most recent flush, not just secret values. As such, CBO.INVAL has stricter checks on its use than CBO.FLUSH, and should only be made available to, and used by, sufficiently-trusted software.

Untrusted software should use CBO.FLUSH instead as a minimum, and a sensible implementation choice for CHERI systems is to always execute CBO.INVAL as CBO.FLUSH.

Prerequisites

Zicbom, RVY

Operation

This listing is manually maintained rather than extracted from the Sail model.

```
let cache_block_size = 2 ^ plat_cache_block_size_exp();
let negative_offset = (X(cs1) &
~(zero_extend(ones(plat_cache_block_size_exp()))) - X(cs1);
match ext_data_get_addr(cs1, negative_offset, Cache(Inval), cache_block_size) {
  Ext_DataAddr_Error(e) => { ext_handle_data_check_error(e); RETIRE_FAIL },
  Ext_DataAddr_OK(_)    => RETIRE_SUCCESS
}
```

Chapter 9. Zicboz (RVY modified behavior)

Specifying RVY and Zicboz gives Zicboz (RVY modified behavior) functionality, which extends the checking.

Table 40. Zicboz (RVY modified behavior) instruction extension

Mnemonic	Function
CBO.ZERO (RVY)	Cache block zero

9.1. CBO.ZERO (RVY)

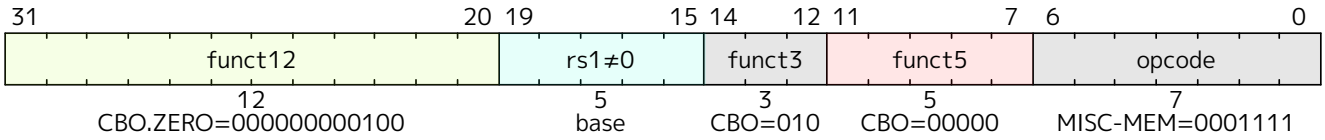
Synopsis

Store zeros to the full set of bytes corresponding to a cache block

Mnemonic

```
CBO.ZERO 0(rs1)
```

Encoding



Description

A `cbo.zero` instruction performs stores of zeros to the full set of bytes corresponding to the cache block whose effective address is the base address specified in `rs1`. An implementation may or may not update the entire set of bytes atomically, although each individual write must atomically set the capability tag of the corresponding naturally aligned YLEN-bit memory location to zero. The authorizing capability for this instruction is `rs1`.

Exceptions

Exceptions occur when the authorizing capability fails one of the checks listed below:

Kind	Reason
CHERI Store/AMO Access Fault	Authorizing capability tag is set to 0.
CHERI Store/AMO Access Fault	Authorizing capability is sealed.
CHERI Store/AMO Access Fault	Authorizing capability does not grant the necessary permissions. W-permission is required.
CHERI Store/AMO Access Fault	At least one byte accessed is outside the authorizing capability bounds, or the bounds could not be decoded.
CHERI Store/AMO Access Fault	Authorizing capability failed any integrity check.

Prerequisites

Zicboz, RVY

Operation

This listing is manually maintained rather than extracted from the Sail model.

```
let cache_block_size = 2 ^ plat_cache_block_size_exp();
let negative_offset = (X(cs1) &
~(zero_extend(ones(plat_cache_block_size_exp()))) - X(cs1);
match ext_data_get_addr(cs1, negative_offset, Cache(Zero), cache_block_size) {
  Ext_DataAddr_Error(e) => { ext_handle_data_check_error(e); RETIRE_FAIL },
  Ext_DataAddr_OK(vaddr) => match translateAddr(vaddr, Write(Data)) {
    TR_Failure(e, ext_ptw) => { handle_translate_exception(vaddr -
negative_offset, e, ext_ptw); RETIRE_FAIL },
    TR_Address(paddr, pbmt, _) => {
      let ep = effectivePrivilege(Cache(Zero), mstatus, cur_privilege());
```

```
      let _ = mem_write_value_priv_meta(pbmt, paddr, cache_block_size, zeros(),
                                        Cache(Zero), ep, default_meta, false,
false, false);
      RETIRE_SUCCESS
    }
  }
}
```

Chapter 10. Zicbop (RVY modified behavior)

Specifying RVY and Zicbop gives Zicbop (RVY modified behavior) functionality, which extends the checking.

Table 41. Zicbop (RVY modified behavior) instruction extension

Mnemonic	Function
PREFETCH.I (RVY)	Cache Block Prefetch for Instruction Fetch
PREFETCH.R (RVY)	Cache Block Prefetch for Data Read
PREFETCH.W (RVY)	Cache block Prefetch for Data Write

10.1. PREFETCH.I (RVY)

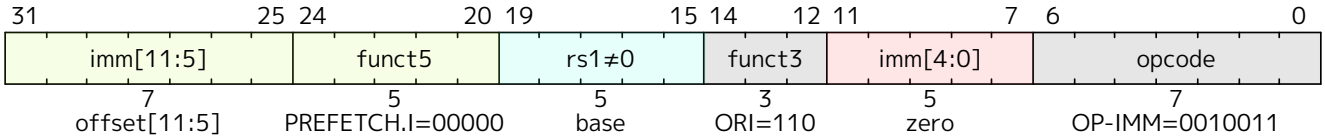
Synopsis

Provide a HINT to hardware that a cache block is likely to be accessed by an instruction fetch in the near future

Mnemonic

```
PREFETCH.I offset(rs1)
```

Encoding



Description

A PREFETCH.I instruction indicates to hardware that the cache block whose effective address is the sum of the base address specified in `rs1` and the sign-extended offset encoded in imm[11:0], where imm[4:0] equals 0b00000, is likely to be accessed by an instruction fetch in the near future.

The authorizing capability is `rs1`. This instruction does not raise any exceptions.

The following authorization checks are performed:

- `rs1.tag=1`
- `rs1` is not sealed
- At least one byte of the requested cache block is within `rs1` 's bounds
- `rs1` grants X-permission
- `rs1` passes all integrity checks

If any of these checks fail, the instruction executes as a `nop` and no prefetch is performed.

Only one byte of the cache block is required to be within bounds because a hardware prefetcher is permitted to prefetch the same cache block in the background.

If the checks above pass, an implementation may opt to cache a copy of the cache block in a cache accessed by an instruction fetch in order to improve memory access latency, but this behavior is not required.

If `rs1=x0` *this instruction executes as a* `nop`.

Prerequisites

Zicbop, RVY

Operation

```
// PREFETCH.I is a hint: it performs no architectural operation
RETIRE_SUCCESS
```

10.2. PREFETCH.R (RVY)

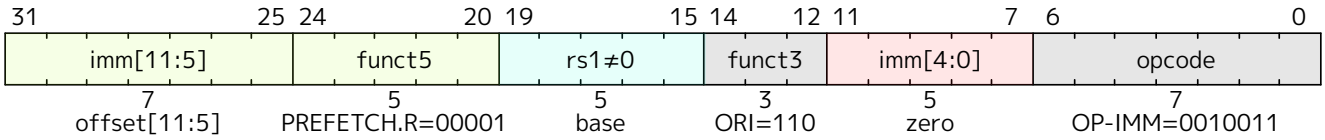
Synopsis

Provide a HINT to hardware that a cache block is likely to be accessed by a data read in the near future

Mnemonic

```
PREFETCH.R offset(rs1)
```

Encoding



Description

A PREFETCH.R instruction indicates to hardware that the cache block whose effective address is the sum of the base address specified in `rs1` and the sign-extended offset encoded in imm[11:0], where imm[4:0] equals 0b00000, is likely to be accessed by a data read (i.e., load) in the near future.

The authorizing capability is `rs1`. This instruction does not raise any exceptions.

The following authorization checks are performed:

- `rs1.tag=1`
- `rs1` is not sealed
- At least one byte of the requested cache block is within `rs1` 's bounds
- `rs1` grants R-permission
- `rs1` passes all integrity checks

If any of these checks fail, the instruction executes as a `nop` and no prefetch is performed.

Only one byte of the cache block is required to be within bounds because a hardware prefetcher is permitted to prefetch the same cache block in the background.

If the checks above pass, an implementation may opt to cache a copy of the cache block in a cache accessed by a data read in order to improve memory access latency, but this behavior is not required.

If `rs1=x0` this instruction executes as a `nop`.

Prerequisites

Zicbop, RVY

Operation

```
// PREFETCH.R is a hint: it performs no architectural operation
RETIRE_SUCCESS
```

10.3. PREFETCH.W (RVY)

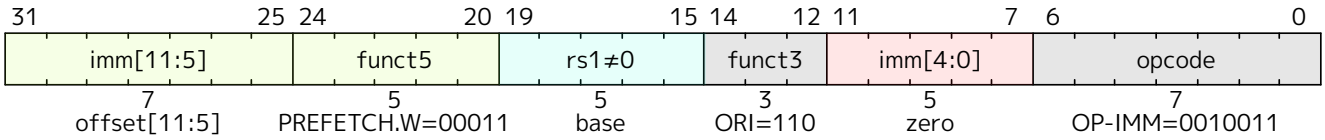
Synopsis

Provide a HINT to hardware that a cache block is likely to be accessed by a data write in the near future

Mnemonic

```
PREFETCH.W offset(rs1)
```

Encoding



Description

A PREFETCH.W instruction indicates to hardware that the cache block whose effective address is the sum of the base address specified in `rs1` and the sign-extended offset encoded in imm[11:0], where imm[4:0] equals 0b00000, is likely to be accessed by a data write (i.e., store) in the near future.

The authorizing capability is `rs1`. This instruction does not raise any exceptions.

The following authorization checks are performed:

- `rs1.tag=1`
- `rs1` is not sealed
- At least one byte of the requested cache block is within `rs1` 's bounds
- `rs1` grants W-permission
- `rs1` passes all integrity checks

If any of these checks fail, the instruction executes as a `nop` and no prefetch is performed.

Only one byte of the cache block is required to be within bounds because a hardware prefetcher is permitted to prefetch the same cache block in the background.

If the checks above pass, an implementation may opt to cache a copy of the cache block in a cache accessed by a data write in order to improve memory access latency, but this behavior is not required.

If `rs1=x0` *this instruction executes as a* `nop`.

Prerequisites

Zicbop, RVY

Operation

```
// PREFETCH.W is a hint: it performs no architectural operation
RETIRE_SUCCESS
```

Chapter 11. U-mode YLEN-bit CSR Summary

Table 42 shows all YLEN-bit U-mode CSRs added or modified by RVY or its extensions.

Table 42. All YLEN-bit U-mode CSRs.

RVY CSR	Extension	Address	Permissions	Initial Value	Description
ddc	Zyhybrid	0x03F	URW	Root Data	Default Data Capability
utidc	RVY	0x07f	Read: U, Write: U, ASR-permission	tag=0, otherwise UNSPECIFIED	User Thread ID



Where initial values are specified as a Root Executable or Root Data they are the maximum possible value. The EEI may initialize those CSRs to cover a smaller memory range, or to grant fewer permissions.

Table 43. Action taken on writing to YLEN-bit U-mode CSRs

RVY CSR	Extension	Action on XLEN write	Action on YLEN write
ddc	Zyhybrid	Apply Invalid address conversion. Always update the CSR with YADDRW even if the address did not change.	Apply Invalid address conversion and update the CSR with the result if the address changed, direct write if address did not change
utidc	RVY	Update the CSR using YADDRW.	Direct write

Code pointers and data pointers in Table 44 are WARL CSRs that do not need to store full 64-bit addresses on RV64, and so may be subject to Invalid address conversion on writing.

Table 44. YLEN-bit U-mode CSRs storing code pointers or data pointers

RVY CSR	Extension	Code Pointer	Data Pointer	Unseal On Execution
ddc	Zyhybrid		✔	

Chapter 12. Vector "V" Extension (RVY)

The Vector extension for RVY does not support capability tags in the vector registers. Consequently, vector stores will always set the associated capability tags in memory to zero.



A future extension may allow capability tags to be stored in vector registers. Until that time, vector load and store instructions must not be used to implement generic memory copying in software, such as the `memcpy()` *standard C library function, that requires capabilities to be preserved, because the vector registers do not hold capabilities, so the capability tags of any copied capabilities will be set to 0 in the destination memory.*

Under RVY, vector loads and stores follow the standard rules for *active* elements:

- Only *active* elements are subject to CHERI exception checks.
- CHERI exceptions are only raised on fault-only-first loads if element 0 is both *active* and fails any exception checks.

Additionally, the standard RVY rule that all loads and stores where the base register is `x0` are reserved applies to all vector memory access instructions.



The approach of using indexed loads with the base register set to the value zero and XLEN-bit offsets does not work well with CHERI as the authorizing capability must cover all of memory. If the authorizing capability is specified as `x0` *then the instruction encoding is reserved.*

Chapter 13. "Zylg" Extension for CHERI 2-level (Local/Global) Information Flow Control

Zylg adds an **Information Flow Control** (IFC) mechanism to CHERI.

It adds:

- One new "level" metadata bit (the GL flag) to classify capabilities as either **global** or **local**.
- Checks to memory operations to control how capabilities can be shared.
- Two new level-related permissions, Store Local (SL) and Load Global (LG).



The behavior is an extension as it may be replaced by a future scheme with more than two levels.

13.1. Added Metadata Bits (Permissions and Flags)

Bit	Type	Comment
GL flag	Information flow label	Used to classify capabilities as global or local.
SL-permission	Data memory permission	Used to filter the capability tags of stored capabilities.
LG-permission	Data memory permission	Used to filter the permissions of loaded capabilities.

13.1.1. The Capability Global (GL) Flag

The Global (GL) flag is a metadata bit used to track and control the propagation of capabilities. It works in conjunction with LG-permission and SL-permission to enforce information flow boundaries.

Global Capability
: A global capability has GL flag=1.

Local Capability
: A local capability has GL flag=0.

Like permissions, the GL flag can be cleared when creating a new capability from an existing one, but it can never be set to 1 unless derived from a superset capability which has the GL flag set to 1.

13.1.2. Store Local Permission (SL)

Store Local Permission (SL) controls whether a local capability can be stored to memory.

If the authorizing capability lacks SL-permission, then any stored local capability is invalidated.

SL-permission is dependent on both C-permission and W-permission as shown in Table 45.

13.1.3. Load Global Permission (LG)

Load Global Permission (LG) controls whether global capabilities can be loaded from memory. Loading a capability using an authorizing capability that lacks LG-permission clears the GL flag bit of the loaded

capability, making it into a local capability. If the loaded capability is unsealed, its LG-permission is also cleared.



This behavior is similar to how LM-permission affects loaded capabilities (but note the difference in interaction with seals).

LG-permission is dependent on both C-permission and R-permission as shown in Table 45.

13.2. Interaction with Root Capabilities

The Root capability definitions are extended:

- A Root Executable capability is required to grant LG-permission and to be a global capability.
- A Root Data capability is required to grant both LG-permission and SL-permission and to be a global capability.

Therefore, in summary, an Infinite capability is required to grant LG-permission, SL-permission, and to be a global capability.

13.3. Interaction with YPERMC and YPERMR

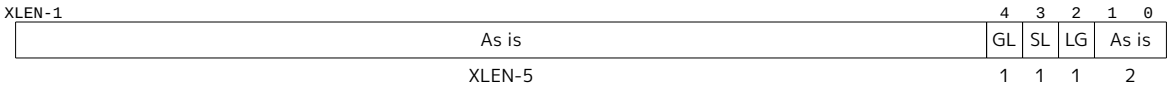


Figure 13. Extended capability permissions bit field (see Figure 6)

The GL flag, SL-permission, and LG-permission fields are mapped into the capability permissions bitfield as used by YPERMC and YPERMR, as shown in Figure 13.

The GL flag is treated as a permission by YPERMC and YPERMR.

Unlike actual permissions it is not subject to the sealed check, and so YPERMC can produce a new capability with its GL flag cleared, even if the source capability is sealed.

13.3.1. Additional YPERMC Permission Rules

The following YPERMC permission rules are added:

Table 45. YPERMC permission rules for Zylg.

YPERMC Rule	Permission	Valid only if
Zylg-1	LG-permission	C-permission and R-permission
Zylg-2	SL-permission	C-permission and W-permission

13.4. Interaction with LY

Zylg introduces two new constraints on capabilities loaded from memory. These constraints apply to LY and all other instructions that can load valid capabilities.

The permissions of the loaded capability may be reduced:

- If the authorizing capability lacks LG-permission and the loaded capability is unsealed, both GL flag

and LG-permission are cleared in the result.

- If the authorizing capability lacks LG-permission and the loaded capability is sealed, GL flag is cleared in the result.



The absence of LG-permission affects the GL flag even of sealed capabilities because the latter is an information flow label rather than an architectural permission.

Table 46. LG-permission effects for loading capabilities

Auth cap field			Data cap field		
R	C	LG	CTag	Sealed	Action
1	1	0	1	Yes	Load data capability with its GL flag cleared
				No	Load data capability with both its GL flag and LG-permission cleared
		All other cases			Load data capability with both its GL flag and LG-permission unmodified

13.5. Interaction with SY

Zylg adds a constraint to SY and all instructions which can store valid capabilities.

The valid capability is stored if:

- The capability being stored is a global capability, or
- The authorizing capability grants SL-permission.

Otherwise, the capability is invalidated before being stored.



While a lack of LG-permission restricts flow by reducing permissions (labeling the loaded capability as local and stripping permissions), SL-permission restricts flow by invalidating capabilities (by setting the capability tag to zero).

Table 47. SL-permission effects for stored capabilities

Auth cap field			Data cap field	
W	C	SL	GL	Notes
1	1	1	X	Store unmodified data capability
		0	1	Store unmodified data capability
			0	Store invalidated data capability

13.6. Interaction with Capability Subsets

The definition of capability subset has an additional rule:

- The GL flag of `cap2` is zero, or the GL flag of `cap1` is one.



This affects the behavior of YSS, YBLD and YUNSEALS.

13.7. Rationale and Software Use Cases

The primary goal of Zylg is to prevent storing pointers to specific memory regions in order to enforce security invariants. There are many uses of this mechanism, but two examples are listed below.

13.7.1. Restricting Capability Flow Between Regions or Domains (e.g., Stack-to-Heap or Kernel-to-User)

Software can use these features to prevent pointers from leaking between different memory regions or privilege domains that have different security properties. The general pattern is to mark sensitive capabilities as local (by clearing their GL flag) and ensuring that destination memory regions are accessed through authorities that lack the SL-permission.

- **Intra-program flow (e.g., Stack-to-Heap):**

  To prevent pointers to short-lived memory (like the stack) from leaking into long-lived memory (like the heap), software marks all stack pointers as local. If the authorities for the heap lack the SL-permission, any attempt to store a stack pointer into the heap results in the hardware invalidating the stored capability. This ensures stack pointers are strictly confined to registers and the stack itself.

- **Cross-domain flow (e.g., Kernel-to-User):**

  These features also apply across privilege boundaries. For example, a kernel can ensure that the capability authorizing a write to userspace (e.g., for a `copy_to_user` operation) lacks the SL-permission. Even if a kernel bug causes it to accidentally attempt to copy sensitive, local kernel capabilities into the user's buffer, the hardware will invalidate those capabilities. This prevents unintentional leaks of usable capabilities to a less-privileged domain.

13.7.2. Enforcing Shallow vs. Deep No-Capture Guarantees

When passing a capability to a less-trusted function, a caller often wants to enforce "no-capture" guarantees, ensuring the callee cannot hold onto the capability after returning. Zylg provides two levels of protection against capability capture.

- **Shallow no-capture (via GL flag and SL-permission):**

  The caller makes the argument capability local by clearing its GL flag. Because the callee's global variables and persistent data structures are accessed through capabilities that lack the SL-permission, the callee cannot store the local argument capability into them. This enforces a shallow guarantee: the callee cannot capture the argument itself, but it might still load a global capability *through* the argument and capture that.

- **Deep no-capture (via LG-permission):**

  To solve the shallow limitation, the caller also removes the LG-permission from the argument capability. If the callee loads any capability through this argument, the hardware automatically removes the loaded capability's GL flag and LG-permission. This enforces a deep no-capture guarantee: the callee is prevented from capturing the argument *and* anything reachable from it.

Chapter 14. "Zyhybrid" Extension for CHERI Pointer Modes

Zyhybrid is an optional extension to RVY which adds the ability to execute standard RISC-V (RV32[IE]/RV64[IE]) code on an RVY hart, while still allowing access to RVY instructions.

The ability to choose between these two behaviors is referred to as switching between *CHERI Pointer Modes*. This extension introduces a new bit in the capability metadata, the P-bit, to control these modes. When installed in pc, the P-bit determines the current execution mode.



The P-bit determines whether pointers are interpreted as standard RISC-V Integer Addresses or as CHERI Capabilities.

Zyhybrid adds the instructions in Table 48 to query and update the current mode, and switch modes.

Table 48. Zyhybrid instruction extension

Mnemonic	Function
YMODEW	Set capability execution mode
YMODER	Read capability mode
YMODESWY	Switch execution to *Capability Pointer Mode*
YMODESWI	Switch execution to *Integral Pointer Mode*

This extension also adds the default data capability (ddc), an unprivileged YLEN-bit CSR used to authorize data memory accesses when in *Integral Pointer Mode*.



Together with pc, ddc enables confining execution to a compartment (or sandbox) where all memory accesses are bounded. These compartments provide binary compatibility for existing RV32[IE]/RV64[IE] code. Provided that the execution environment has set up ddc and pc appropriately, non-CHERI-aware programs will execute unmodified within these bounds.

For detailed examples of how this extension facilitates co-existence and sandboxing, see Appendix B.

RVY implementations that support Zyhybrid are typically referred to as CHERI Hybrid. Implementations which do not support Zyhybrid are referred to as CHERI Purecap and always operate in *Capability Pointer Mode*.

Because the base architecture remains RVY, the custom encoding space re-allocated to CHERI remains re-allocated.



If an implementation needs to be able to access custom encodings in spaces which have been reallocated, then privileged software should disable CHERI entirely.

14.1. CHERI Pointer Modes

The two CHERI Pointer Modes are:

Capability Pointer Mode

All pointer operands to instructions are interpreted as capabilities. Instruction semantics are as defined in the RVY base ISA.

Integral Pointer Mode

All pointer operands to instructions are interpreted as for the RV32[IE]/RV64[IE] base ISA, i.e., as plain memory addresses without metadata. While the base ISA remains RVY this mode provides backwards

compatibility with RV32[IE]/RV64[IE]. Operands are still checked against a capability, but instead of using the metadata from the full YLEN-bit register, the bounds and permissions are obtained from ddc (for data accesses) or pc (for instruction accesses).

ARC-NOTE: Please confirm the names of the two CHERI Pointer Modes. The names Integral Pointer Mode and Capability Pointer Mode distinguish interpreting pointer operands as plain integer addresses vs. full capabilities with metadata, and match compiler terminology (integral vs. non-integral pointers).

If RVC encodings are supported, load/store encodings will revert to their *non-CHERI encodings*, such as C.LY reverting to C.FLD for RV64/RV64Y. This behavior is summarized in Table 33 and Table 34.

Instructions which are *modified* on an RVY architecture (see Table 22) *revert to their standard RV32[IE]/RV64[IE] behavior.*

All RVY instructions and CSRs are available in addition to all standard RV32[IE]/RV64[IE] instructions and CSRs.

Non-standard extensions with encodings in remapped custom space are not supported.

The authorizing capability for data access is ddc (as opposed to `rs1`). That is, all data memory accesses are implicitly authorized by ddc with only the memory address sourced from `rs1`.

ddc is also used to authorize memory instructions added by RVY (such as LY and SY).

The restriction of not allowing `rs1=x0` is removed and *all* data memory accesses are permitted to use `x0` as the base address.

Capability memory access instructions are also permitted to have `rs1=x0`.

All instruction fetches are authorized by pc.

Prefetches are also checked against the appropriate CSR (ddc for PREFETCH.W (RVY) and PREFETCH.R (RVY), pc for PREFETCH.I (RVY)) and execute as nops if authorization fails.

Authorization of prefetches is useful to mitigate against side channels, e.g., preventing userspace from fetching kernel memory into a cache.

Accesses to YLEN-bit CSRs added by RVY or its extensions (such as ddc), remain YLEN-bit (see Zicsr (RVY modified behavior)). Accesses to all other CSRs are XLEN-bit and writes use the semantics of the YADDRW instruction.

The *CHERI Pointer Mode* is key in providing backwards compatibility with the base RV32[IE]/RV64[IE] ISAs.

The CHERI Pointer Mode is always Capability Pointer Mode on implementations that support RVY, but not Zyhybrid.

Software is Purecap if it uses CHERI capabilities for all memory accesses (including loads, stores, and instruction fetches). Purecap software requires RVY. Software is Hybrid if it uses both integer addresses and CHERI capabilities. Hybrid software requires both RVY and Zyhybrid.

14.1.1. CHERI Pointer Mode Encoding

The current CHERI Pointer Mode is controlled by the P-bit in the capability AP-field of pc. The P-bit (P) is only architecturally relevant for capabilities granting X-permission, as it only affects execution state when installed into pc.



While the P-bit can be read or written explicitly using YMODER and YMODEW, these instructions do not change the current execution mode. The mode changes when a capability is installed into pc (e.g., via an indirect jump, see Section 14.1.2).

The P-bit is only valid in capabilities that grant X-permission (see Section 14.1.3). Capabilities that do not grant X-permission therefore do not carry a meaningful P-bit, so YMODEW cannot modify their P-bit. Zyhybrid requires a capability-encoding format that supports transport of the P-bit (e.g., RV64LYA).

- CHERI Pointer Mode (P)=0 indicates *Capability Pointer Mode*.
- CHERI Pointer Mode (P)=1 indicates *Integral Pointer Mode*.

When executing YPERMC, if X-permission is removed while the P-bit is set to 1, and the capability-encoding format still permits representing the P-bit, then the P-bit must be set to 0.

14.1.2. Changing CHERI Pointer Mode

The P-bit of pc can be updated by the instructions listed in Table 49:

Table 49. Zyhybrid instructions that can perform mode changes

Mnemonic	From mode	Description
JALR (RVY)	*Capability Pointer Mode*	Jump to capability register, and link to capability register
YMODESWI	*Capability Pointer Mode*	Switch execution to *Integral Pointer Mode*
YMODESWY	*Integral Pointer Mode*	Switch execution to *Capability Pointer Mode*



When JALR (RVY) copies `rs1` into pc it includes copying the P-bit and so setting the CHERI Pointer Mode of the target instruction.

The P-bit of a X-permission-granting capability can be read and written by the instructions listed in Table 50:

Table 50. Zyhybrid instructions to observe and update the mode in a capability

Mnemonic	Description
YMODEW	Set capability execution mode
YMODER	Read capability mode



In addition to the mode switching instructions, the current mode can also be updated by setting the P-bit of a target capability using YMODEW followed by a JALR (RVY).

14.1.3. Additional YPERMC Permission Rule

The following YPERMC permission rule is added:

Table 51. YPERMC permission rules for Zyhybrid.

YPERMC Rule	Permission	Valid only if
Zyhybrid-1	P-bit	X-permission

14.1.4. Observing the CHERI Pointer Mode

The effective CHERI Pointer Mode cannot be determined just by reading the P-bit from the pc since it also depends on the execution environment. The following code sequence demonstrates how a program can observe the current, effective CHERI Pointer Mode. It will write, to `x1`, the value `1` for *Capability Pointer Mode* (where AUIPC (RVY) derives a valid capability from the pc) or `0` for *Integral Pointer Mode* (where `auipc` reverts to its standard behavior and writes an integer result):

```
auipc x1, 0
ytagr x1, x1
```

14.2. Added Instructions

14.2.1. YMODEW

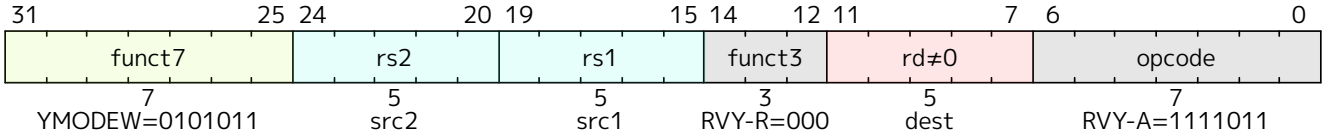
Synopsis

Set capability execution mode

Mnemonic

`YMODEW rd, rs1, rs2`

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Copy `rs1` to `rd`.

If `rs1` is sealed or if `rs1` fails any integrity checks, then set `rd.tag=0`.

Otherwise, if `rs1` grants X-permission then update the P-bit of `rd` to:

1. *Capability Pointer Mode* if the least significant bit of `rs2` is 0, or,
2. *Integral Pointer Mode* if the least significant bit of `rs2` is 1.

The value of `rs1.tag` *is not checked and is propagated unchanged to* `rd`.

Prerequisites

Zyhybrid

Operation

```
let cap = C(cs1);
let mode = execution_mode_encdec(X(rs2)[0 .. 0]);

let cap = clearTagIf(cap, capIsSealed(cap));
let hasMode = not(permsMalformed(cap)) & canX(cap);
let newCap = if hasMode then setCapMode(cap, mode) else cap;

C(cd) = newCap;
RETIRE_SUCCESS
```

14.2.2. YMODER

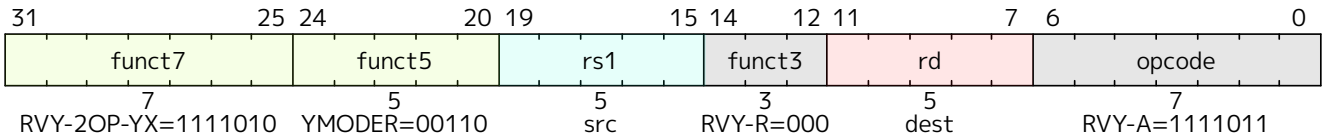
Synopsis

Read capability mode

Mnemonic

```
YMODER rd, rs1
```

Encoding





The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Decode the CHERI Pointer Mode from the capability in `rs1` and write the result to `rd`.

Set `rd` to 0 if `rs1` does not grant X-permission.

Set `rd` to 0 if any integrity checks fail.

Otherwise set `rd` according to `rs1` 's CHERI Pointer Mode (P-bit):

1. Set `rd` to 0 for *Capability Pointer Mode*, or,
2. Set `rd` to 1 for *Integral Pointer Mode*.



The value of `rs1.tag` *does not affect the result.*

Prerequisites

Zyhybrid

Operation

```
let capVal = C(cs1);
X(rd) = zero_extend(execution_mode_encdec(getCapMode(capVal)));
RETIRE_SUCCESS
```

14.2.3. YMODESWY, YMODESWI

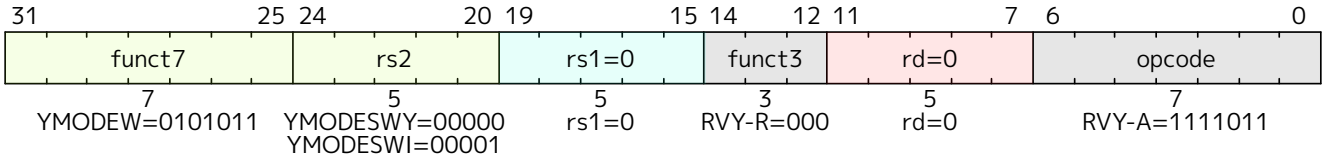
Synopsis

Switch execution mode to *Capability Pointer Mode* (YMODESWY), or *Integral Pointer Mode* (YMODESWI), 32-bit encodings

Mnemonics

`YMODESWY`
`YMODESWI`

Encoding



The instruction encoding is not final and is highly likely to change prior to v1.0

Description

Set the current CHERI Pointer Mode in pc.

- YMODESWY: If the current mode in pc is *Integral Pointer Mode* (1), then the P-bit in pc is set to *Capability Pointer Mode* (0). Otherwise no effect.
- YMODESWI: If the current mode in pc is *Capability Pointer Mode* (0), then the P-bit in pc is set to *Integral Pointer Mode* (1). Otherwise no effect.

Prerequisites

Zyhybrid

Operation

This listing is manually maintained rather than extracted from the Sail model.

```
let target : ExecutionMode = match op {
  MODESW_CAP => CapPtrMode,
  MODESW_INT => IntPtrMode,
};
set_next_pcc(setCapMode(PCC, target));
RETIRE_SUCCESS
```

14.3. Added State

Zyhybrid adds the YLEN-bit CSR shown in Table 52.

Table 52. Unprivileged YLEN-bit CSRs added in Zyhybrid

YLEN-bit CSR	Permissions	Description
ddc	URW	Default Data Capability

14.3.1. Default Data Capability CSR (ddc)

ddc is a read-write, U-mode accessible YLEN-bit CSR. It does not require ASR-permission in pc for writes or reads. Similarly to pc authorizing all control flow and instruction fetches, this capability register is implicitly checked to authorize all data memory accesses when the current CHERI Pointer Mode is *Integral Pointer Mode*.

The EEI initializes ddc such that the program can run successfully (e.g., by setting it to be a Root Data capability).

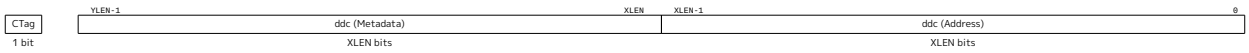


Figure 14. Unprivileged default data capability register

Chapter 15. Zicsr (RVY modified behavior)

Table 53. Zicsr (RVY modified behavior) instructions

Mnemonic	Function
CSRRW (RVY)	CSR write
CSRRS (RVY)	CSR set
CSRRC (RVY)	CSR clear
CSRRWI (RVY)	CSR write (immediate form)
CSRRSI (RVY)	CSR set (immediate form)
CSRRCI (RVY)	CSR clear (immediate form)

Zyhybrid makes *Integral Pointer Mode* available.

In addition to the rules in Section 2.6.4, when executing in *Integral Pointer Mode*, accesses to YLEN-bit CSRs are XLEN-bit for backwards compatibility, and the final written value is determined using the semantics of the YADDRW instruction. The exception is CSRs added by RVY or its extensions (such as utidc and ddc), which continue to be accessed as YLEN-bit.



Access to ddc remains YLEN-bit, as XLEN-bit access to ddc is not useful to software.

15.1. CSRRWI, CSRRS, CSRRSI, CSRRC, CSRRCI (RVY)

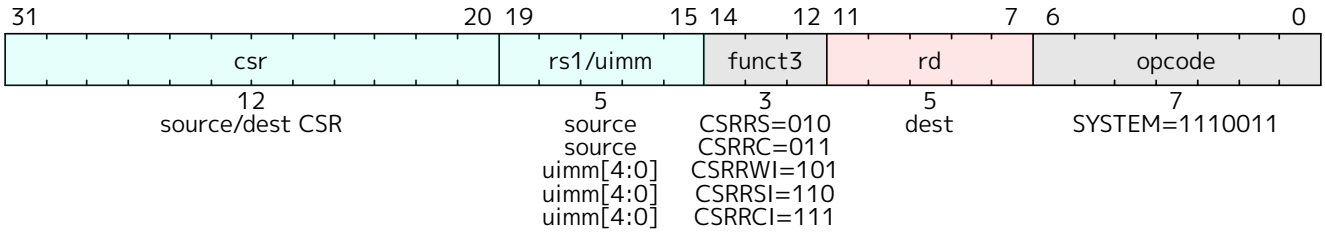
Synopsis

CSR access (CSRRWI, CSRRS, CSRRSI, CSRRC, CSRRCI) 32-bit encodings for RVY

Mnemonics

```
csrrs rd, csr, rs1
csrrc rd, csr, rs1
csrrwi rd, csr, imm
csrrsi rd, csr, imm
csrrci rd, csr, imm
```

Encoding



Description

These CSR instructions have extended functionality for accessing YLEN-bit CSRs.

Access to XLEN-bit CSRs is as defined in Zicsr.

Zicsr rules are followed when determining whether to read or write the CSR. Suppressed read or write actions have no side-effects on the CSR.

For the instructions on this page, all writes to YLEN-bit CSRs specify XLEN-bit write data only and use the semantics of the YADDRW instruction to determine the final write data.

Read data from YLEN-bit CSRs is YLEN bits.

However, if Zyhybrid is implemented and the current mode is *Integral Pointer Mode*, read data is XLEN bits unless the CSR was added by RVY or its extensions.

Permissions

Accessing CSRs may require ASR-permission.

Prerequisites

RVY, Zicsr, optional Zyhybrid

Operation

```
let isWrite = op == CSRRW | rs1 != zeros();
// CSRRS[I] and CSRRC[I] always read. CSRRW[I] reads if the destination
// register is not x0.
let isRead = op != CSRRW | rd != zeros();

if not(check_CSR(csr, cur_privilege(), isWrite))
then { handle_illegal(); RETIRE_FAIL }
```

```
else if not(ext_check_CSR(csr, cur_privilege(), isWrite))
then { ext_check_CSR_fail(); RETIRE_FAIL }
else handle_csr_op(csr, rs1, rd, is_imm, op, isWrite, isRead)
```

15.2. CSRRW (RVY)

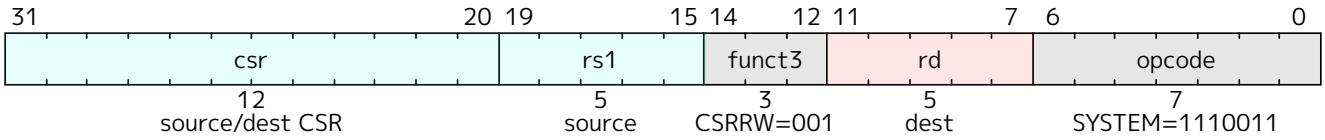
Synopsis

CSR access (CSRRW) 32-bit encodings for RVY

Mnemonic

```
csrrw rd, csr, rs1
```

Encoding



Description

CSRRW has extended functionality for accessing YLEN-bit CSRs.

Access to XLEN-bit CSRs is as defined in Zicsr.

CSRRW accesses to YLEN-bit CSRs read YLEN bits into `rd` and write YLEN bits of `rs1` into the CSR.

However, if Zyhybrid is implemented and the current mode is *Integral Pointer Mode*, accesses are XLEN-bit and the final write data is determined using the semantics of the YADDRW instruction, unless the CSR was added by RVY or its extensions.

When writing YLEN bits of `rs1`, if any integrity checks fail then invalidate the capability before writing to the CSR.



Any YLEN-bit CSRs may have additional rules defined to determine the final written value of the metadata and/or to write zero to the capability tag. These rules are shown in Table 43, Table 59, and Table 62.

Permissions

Accessing CSRs may require ASR-permission.

Prerequisites

RVY, Zicsr, optional Zyhybrid

Operation

```
let isWrite = op == CSRRW | rs1 != zeros();
// CSRRS[I] and CSRRC[I] always read. CSRRW[I] reads if the destination
// register is not x0.
let isRead = op != CSRRW | rd != zeros();

if not(check_CSR(csr, cur_privilege(), isWrite))
then { handle_illegal(); RETIRE_FAIL }
else if not(ext_check_CSR(csr, cur_privilege(), isWrite))
then { ext_check_CSR_fail(); RETIRE_FAIL }
else handle_csr_op(csr, rs1, rd, is_imm, op, isWrite, isRead)
```

Chapter 16. Placeholder References to the Unprivileged Spec



This chapter only exists for the standalone document to allow references to resolve.

RV32I

See Chapter *RV32I Base Integer Instruction Set* in (RISC-V, 2023).

General-purpose registers

See Chapter *RV32I Base Integer Instruction Set* in (RISC-V, 2023).

Load and Store Instructions

See Chapter *RV32I Base Integer Instruction Set* in (RISC-V, 2023).

Zcmt

See Chapter *"Zc*" Extension for Code Size Reduction* in (RISC-V, 2023).

Zcmp

See Chapter *"Zc*" Extension for Code Size Reduction* in (RISC-V, 2023).

Chapters for the Privileged Specification

Chapter 17. "Smy" Extension for Machine-Level Architecture, Version 1.0

This chapter describes integration of RVY with the RISC-V privileged architecture. Smy is mandatory for all RVY implementations.

17.1. Machine-Level CSRs widened by RVY

RVY widens some M-mode CSRs so that they can hold capabilities. Additionally, RVY allocates some previously reserved bits in existing CSRs.



`mconfigptr` *is not widened, despite representing an address, as it holds a read-only address solely for use by low-level system software and is not interpreted by hardware. Such software can be expected to hold suitable capabilities from which it can derive a capability to access the data structures at the target address.*

17.1.1. Machine Trap Vector Base Address Capability Register (mtvec)

The WARL `mtvec` register is widened to a capability. Its reset value is nominally a Root Executable capability.



mtvec (RVY) exists in all CHERI implementations, and so may be used as a source of a Root Executable capability after reset.

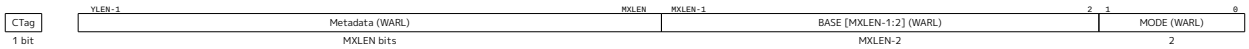


Figure 15. Machine trap-vector base-capability register



Implementations may use the WARL behavior to restrict the metadata to only values valid for code pointers.

When a trap is taken into M-mode, the capability in mtvec (RVY) is installed in the pc, with the address determined by the standard `mtvec` behavior.

As in the standard `mtvec` definition, the address observed through mtvec (RVY) is not simply the stored base address:

1. `mtvec.address[1:0]` holds the MODE field, which selects the trap-vector mode and does not form part of the trap-vector base address.
2. When MODE=Vectored, the effective trap-vector address is the base address plus four times the interrupt cause.
3. An explicit CSR read returns the base address with the MODE field in `mtvec.address[1:0]`.

`HICAUSE` is defined to be the largest interrupt cause value that the implementation can write to `xcause` when an interrupt is taken. The set of addresses that can therefore be observed through mtvec (RVY) ranges from a minimum of `mtvec.address & ~3` to a maximum of `(mtvec.address & ~3) + 4 x HICAUSE`.

Every read of mtvec (RVY), whether implicit (taking a trap) or explicit (a CSR read), forms the observed address using the semantics of the YADDRW instruction, either inserting the MODE bits or, in Vectored mode, adding the cause offset. This has two consequences:

- All of the observable addresses must be represented exactly. Software must ensure this property holds when writing mtvec (RVY), and the hardware sets the capability tag to zero if any observable address cannot be represented exactly.
- A sealed capability written to mtvec (RVY) is always read back with its capability tag cleared, because the YADDRW address update invalidates a sealed capability even when the address value is unchanged.

> *The capability in mtvec (RVY) is not unsealed when it is written to pc, unlike other CSRs such as mepc (RVY).*

Following the standard `mtvec` rules, mtvec (RVY) need not be able to hold all possible invalid addresses.

17.1.2. Machine Scratch Capability Register (mscratch)

The `mscratch` register is widened to a capability.

The reset value of the capability tag of this CSR is zero, the reset values of the metadata and address fields are UNSPECIFIED.

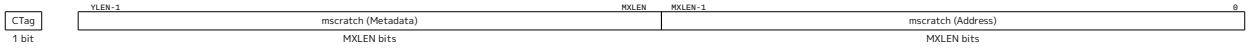


Figure 16. Machine scratch capability register

17.1.3. Machine Exception Program Counter Capability (mepc)

The WARL `mepc` register is widened to a capability. Its reset value is nominally a Root Executable capability.

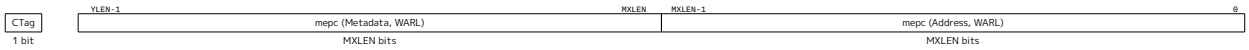


Figure 17. Machine exception program counter capability register

`mepc.address` is the `mepc` CSR, and so follows the standard rules, meaning that:

1. `mepc.address[0]=0`, and
2. `mepc.address[1]=0` when IALIGN is fixed to 32
3. `mepc.address[1]` reads as zero when IALIGN is programmable and is set to 32

> *Implementations may use the WARL behavior to restrict the metadata to values only valid for code pointers.*

> *JALR (RVY) may jump to a target pc value with invalid metadata. The exception is raised at the target pc. The WARL behavior means that the metadata reported can be restricted to valid pc values. This allows invalid metadata fields not to need to be maintained within pc values within the microarchitecture.*
> *As xepc CSRs are WARL, implementations should take care not to fix too many fields so that debug becomes difficult. For example, if the X-permission field is hardwired to one, then it may be hard to diagnose a fault caused by jumping to a pc without X-permission, as the bit will read as one.*

As listed above for mtvec (RVY), this means that `mepc.address` can represent multiple different values. Therefore, software must ensure that all possible values are represented exactly on writing, otherwise the hardware sets the written capability tag to zero.

Sealed capabilities may be written to mepc (RVY). Invalidate the capability on writing if:

1. `mepc.address[0]=1`, or
2. `mepc.address[1]=1` when IALIGN=32

In the following case the value of the capability tag observable in the CSR depends on the value of IALIGN:

1. `mepc.ct≠0 && mepc.tag=1`, and
2. `mepc.address[1]=1` and IALIGN=16 when writing the CSR

The observable capability tag is zero when IALIGN=32 and one when IALIGN=16.

When a trap is taken into M-mode, the pc (including its capability tag and metadata) is written to mepc (RVY), with `mepc.address` set to the `pc` of the interrupted or excepting instruction following the standard behavior.

On execution of an MRET (RVY) instruction, the capability value from mepc (RVY) is unsealed if it is a Sealed entry point capability and written to pc.

mepc (RVY) follows the rule from `mepc` about not needing to be able to hold all possible invalid addresses.

17.2. Machine-Level CSRs added by RVY

17.2.1. Machine Thread Identifier Capability (mtidc)

The mtidc register is used to identify the current software thread in M-mode, using the method defined in the section for the unprivileged utidc CSR.

The reset value of the capability tag of this CSR is zero, the reset values of the metadata and address fields are UNSPECIFIED.

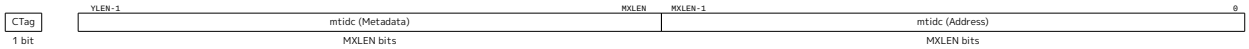


Figure 18. Machine thread identifier capability register

17.3. Machine-Level CSRs modified by RVY

17.3.1. Machine Environment Configuration (`menvcfg`) Register

Smyhybrid adds the Y bit to **menvcfg**. The Y bit can be used to disable or enable RVY for privilege modes below M-mode, as described in Chapter 20, *"Smyhybrid and Ssyhybrid" Extensions, Version 1.0*.

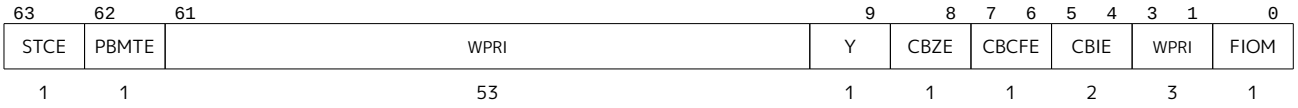


*Figure 19. Machine environment configuration register (**menvcfg**)*

17.3.2. Machine ISA (`misa`) Register

When the base ISA is RVY, the Y bit is one.

`misa.Y` is a WARL field. Zero is only a legal value for misa.Y if the capability metadata of pc, `xtvec`, and `xepc` are currently configured as their respective reset values.

The list also includes ddc if Zyhybrid is implemented.

This restriction ensures M-mode software cannot drop CHERI checks by modifying misa once they have been configured. A simpler conforming implementation could prevent setting misa.Y to zero if the capability metadata of any of these registers has ever been written to, rather than dynamically checking the current metadata.

Either misa.I or misa.E is *also* set to indicate whether 32 or 16 general-purpose registers are present.

Implementations which allow misa.C to be writable need to legalize `xepc` on reading if the misa.C value has changed since the value was written as this can cause the read value of bit [1] to change state.

17.3.3. Machine Status Registers (mstatus)

`mstatus` includes the Userspace Capability Read Generation (UYRG) bit, the Supervisor version (SYRG) and the enable bit (YRGE). These fields reside in `mstatus`; the other CSRs provide the standard restricted views of them.

The definitions of the following fields are furnished by the Svyrg extension:

- Userspace Capability Read Generation (UYRG)
- Supervisor Capability Read Generation (SYRG)
- The Capability Read Generation Enable bit (YRGE).

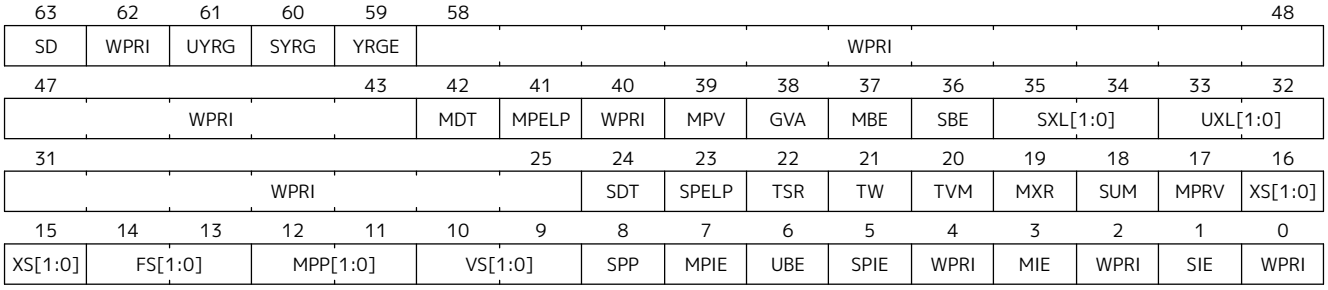


*Figure 20. Machine status (**mstatus**) register for RV64Y*

mstatus for RVY has additional restrictions on XLEN and endianness:

- The MXL field, and the SXL and UXL fields that control the value of XLEN for S-mode and U-mode, must be read-only when the base architecture is RVY, with SXL and UXL equal to MXL. Only 1 and 2 are supported values.
- The MBE, SBE, and UBE fields that control the memory system endianness for M-mode, S-mode, and U-mode must be read-only when the base architecture is RVY. SBE and UBE must be read-only and equal to MBE, if S-mode or U-mode, respectively, is implemented, or read-only zero otherwise.

Changing XLEN or endianness would change the interpretation of all in-memory capabilities, so allowing these fields to change at runtime is prohibited.

These restrictions may be relaxed by a future extension. Such an extension is likely to enforce the constraint that any privilege level with XLEN less than MXLEN has CHERI disabled.

MXR has no effect on the CHERI permission checking.

CHERI does not need to make use of execute-only memory for security reasons, and so MXR has no relevance.

17.3.4. Machine Cause Register (mcause)

RVY adds new exception codes for CHERI exceptions that **mcause** must be able to represent. The new exception codes are listed in Table 54 and the priorities are listed in Table 55.

Table 54. Synchronous exception causes for RVY.

Exception code	Description
32	CHERI Instruction Access Fault
33	CHERI Load Access Fault
34	CHERI Store/AMO Access Fault
35	CHERI Load Capability Fault[1]
36	CHERI Store/AMO Page Fault[1]

[1] These exceptions are defined by the Svyrg extension which is available for RV64Y only.

Table 55. Synchronous exception priority in decreasing priority order for RVY.

Priority	Exc.Code	Description
Highest	3	Instruction address breakpoint
	32	Prior to instruction address translation: CHERI Instruction Access Fault due to pc checks (capability tag, sealed, execute permission, bounds[1])
	12, 1	During instruction address translation: First encountered page fault or access fault
	1	With physical address for instruction: Instruction access fault
	2 0 8,9,11 3 3	Illegal instruction Instruction address misaligned Environment call Environment break Load/store/AMO address breakpoint
	33,34	Prior to address translation for an explicit memory access: CHERI Load Access Fault, CHERI Store/AMO Access Fault due to capability checks (capability tag, sealed, permissions, bounds)
	5,7	Load/store/AMO capability address misaligned[3]
	4,6	Optionally: Load/store/AMO address misaligned
	36, 13, 15, 5, 7	During address translation for an explicit memory access: First encountered CHERI Store/AMO Page Fault, page fault or access fault
	5,7	With physical address for an explicit memory access: Load/store/AMO access fault
	4,6	If not higher priority: Load/store/AMO address misaligned
Lowest	35	CHERI Load Capability Fault[2]

[1] pc bounds are checked against all bytes of fetched instructions. If the instructions could not be decoded to determine the length, then the pc bounds check is made against the minimum-sized instruction supported by the implementation which can be executed, when prioritizing against Instruction Access Faults.

[2] CHERI Load Capability Fault is the lowest priority as determining whether to raise the exception may

include checking the loaded capability tag.

[3] Misaligned capability accesses raise access faults instead of misaligned faults since they cannot be emulated in software.

Full details of CHERI Instruction Access Fault, CHERI Load Access Fault and CHERI Store/AMO Access Fault are in Table 56.

17.3.5. Machine Trap Delegation Register (medeleg)

Bits 32,33,34,35,36 of **medeleg** refer to valid CHERI exceptions and so can be used to delegate CHERI exceptions to S-mode.

17.3.6. "Smstateen" Integration

The TIDC bit in `mstateen0` controls access to the stidc, and utidc CSRs.

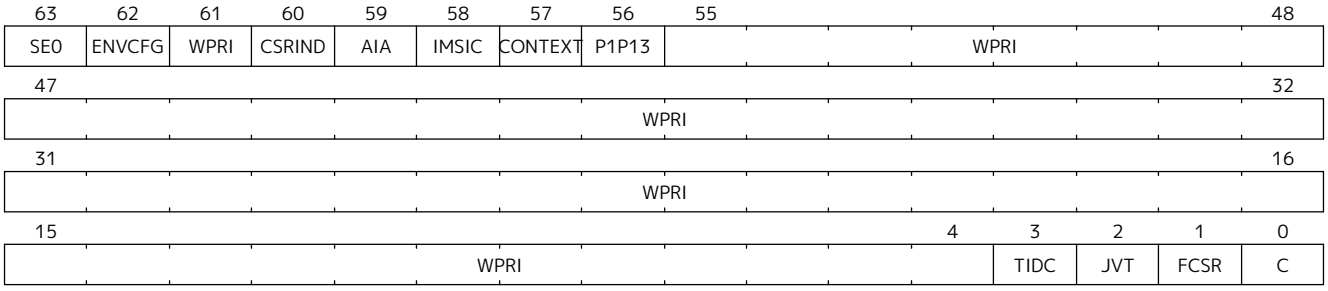


Figure 21. Machine State Enable 0 Register (`mstateen0`)

17.4. CHERI Exception Handling

CHERI faults are typically higher priority than standard RISC-V faults. E.g., CHERI faults on the pc are higher priority than any other fault affecting the program counter such as Instruction Access Fault.

The overall priority is shown in Table 55.

Table 56 shows the detailed exception checks added by CHERI.

In the table below `auth_cap` is `rs1`, unless Zyhybrid is implemented and executing in Integral Pointer Mode where it is ddc.

Table 56. Valid CHERI exception combination description

Instructions	Xcause	Description	Check
All instructions have these exception checks first			
All	32	CHERI Instruction Access Fault	pc capability tag is zero
All	32	CHERI Instruction Access Fault	pc is sealed
All	32	CHERI Instruction Access Fault	pc does not grant X-permission
All	32	CHERI Instruction Access Fault	Any byte of the current instruction is out of pc bounds[1]
All	32	CHERI Instruction Access Fault	pc fails any integrity checks.
CSR/Xret additional exception check			

Instructions	Xcause	Description	Check
CSR*, MRET (RVY), SRET (RVY), CBO.INVAL (RVY)	2	Illegal instruction	pc does not grant ASR-permission when required for CSR access or execution of MRET (RVY), SRET (RVY), or CBO.INVAL (RVY)
Load additional exception checks			
All loads	33	CHERI Load Access Fault	`auth_cap` capability tag is zero
All loads	33	CHERI Load Access Fault	`auth_cap` is sealed
All loads	33	CHERI Load Access Fault	`auth_cap` does not grant R-permission
All loads	33	CHERI Load Access Fault	Any byte of the load access is out of `auth_cap` bounds[2]
All loads	33	CHERI Load Access Fault	`auth_cap` fails any integrity checks.
Capability loads	5[3]	Load access fault	Misaligned capability load
Store/atomic/cache-block-operation additional exception checks			
All stores, all atomics, all CBOs	34	CHERI Store/AMO Access Fault	`auth_cap` capability tag is zero
All stores, all atomics, all CBOs	34	CHERI Store/AMO Access Fault	`auth_cap` is sealed
All stores, CBO.ZERO	34	CHERI Store/AMO Access Fault	`auth_cap` does not grant W-permission
All atomics, CBO.CLEAN, CBO.FLUSH, CBO.INVAL	34	CHERI Store/AMO Access Fault	`auth_cap` does not grant both R-permission and W-permission
All stores, all atomics	34	CHERI Store/AMO Access Fault	Any byte of the access is out of `auth_cap` bounds[2]
CBO.ZERO, CBO.INVAL	34	CHERI Store/AMO Access Fault	Any byte of the cache block is out of `auth_cap` bounds[2]
CBO.CLEAN, CBO.FLUSH	34	CHERI Store/AMO Access Fault	All bytes of the cache block are out of `auth_cap` bounds[2]
All stores, all atomics, all CBOs	34	CHERI Store/AMO Access Fault	`auth_cap` fails any integrity checks.
Capability stores	7[3]	Store access fault	Misaligned capability store

[1] The pc bounds checks includes the case where the bounds could not be decoded and so a minimum-sized instruction is not fully in bounds.

[2] The load/store bounds check includes the case where the bounds could not be decoded as they are malformed.

[3] Misaligned capability accesses raise access faults instead of misaligned faults since they cannot be emulated in software.

17.5. CHERI Exceptions and Speculative Execution

ARC-NOTE: Please advise whether this section should remain in this chapter, or whether it should be expanded and placed in a supporting document.

CHERI adds architectural guarantees that can prove to be micro-architecturally useful for reducing side-channels caused by speculative-execution.

> *Programmers should be able to rely on instructions that fail CHERI permission checks not to update microarchitectural state, e.g., not updating cache state for a load which fails a bound check.*
> *When implementing an RVY hart, micro-architects need to carefully consider the interaction of raising exceptions late in the pipeline, speculation windows, and the resulting side-channel attacks.*

17.6. Physical Memory Attributes (PMA)

Typically, only parts of the entire memory space need to support CHERI capability tags. Therefore, Physical Memory Attributes are included indicating whether a memory region allows storing CHERI capability tags and to define the behavior when attempting to access them.

There are three levels of support:

Table 57. CHERI PMAs

PMA	Load Behavior	Store Behavior	Comment
CHERI Capability Tag	Load capability tag	Store capability tag	Tagged memory supporting capability tags
CHERI Capability Tag Strip	Load zero capability tag	Ignore stored capability tag	No support for capability tags, ignore them
CHERI Capability Tag Store Fault	Load zero capability tag	Store/AMO Access Fault on capability tag[1]	No support for capability tags, trap on storing one

[1] The access fault is triggered on all capability stores or atomics such as SY or AMOSWAP.Y when C-permission and W-permission are granted and the to-be-stored capability is valid. For SC.Y, the access fault is raised if the to-be-stored capability in `rs2` is valid, independent of the reservation outcome. This matches the semantics of SC.* with regard to *pte.d* (see also Capability Dirty Tracking).

Only memory regions that have the *CHERI Capability Tag* PMA require storage for capability tags.

17.7. Modified MRET Behavior

MRET (RVY) reads the full YLEN bits of the mepc (RVY) register and unseals it prior to exception return if it is sealed as a Sealed entry point capability.

17.8. M-mode YLEN-bit CSR Summary

Table 58 shows all YLEN-bit M-mode CSRs added or modified by RVY or its extensions.

Table 58. All YLEN-bit M-mode CSRs.

RVY CSR	Extension	Address	Permissions	Reset Value	Description
mscratch (RVY)	Smy	0x340	MRW, ASR-permission	tag=0, otherwise specified by the platform	Machine Scratch Capability
mtvec (RVY)	Smy	0x305	MRW, ASR-permission	Root Executable	Machine Trap-Vector Base-Address Capability
mepc (RVY)	Smy	0x341	MRW, ASR-permission	Root Executable	Machine Exception Program Counter Capability

RVY CSR	Extension	Address	Permissions	Reset Value	Description
mtidc	Smy	0x37f	Read: M, Write: M, ASR-permission	tag=0, otherwise specified by the platform	Machine Thread ID



Where reset values are specified as a Root Executable or Root Data they are the maximum possible values. The platform may reset those CSRs to a smaller memory range, or to grant fewer permissions.

Table 59. Action taken on writing to YLEN-bit M-mode CSRs

RVY CSR	Extension	Action on XLEN write	Action on YLEN write
mscratch (RVY)	Smy	Update the CSR using YADDRW.	Direct write
mtvec (RVY)	Smy	Apply Invalid address conversion. Always update the CSR with YADDRW even if the address did not change, including the MODE field in the address for simplicity. Vector range check if vectored mode is programmed.	Apply Invalid address conversion. Always update the CSR with YADDRW even if the address did not change, including the MODE field in the address for simplicity. Vector range check if vectored mode is programmed.
mepc (RVY)	Smy	Apply Invalid address conversion. Always update the CSR with YADDRW even if the address did not change.	Apply Invalid address conversion and update the CSR with the result if the address changed, direct write if address did not change
mtidc	Smy	Update the CSR using YADDRW.	Direct write

Code pointers and data pointers in Table 60 are WARL CSRs that do not need to store full 64-bit addresses on RV64, and so may be subject to Invalid address conversion on writing.

Table 60. YLEN-bit M-mode CSRs storing code pointers or data pointers

RVY CSR	Extension	Code Pointer	Data Pointer	Unseal On Execution
mtvec (RVY)	Smy	✔		
mepc (RVY)	Smy	✔		✔

Chapter 18. "Ssy" Extension for Supervisor-Level Architecture, Version 1.0

Ssy is mandatory for all RVY implementations that support S-mode.

18.1. Supervisor-Level CSRs widened by RVY

18.1.1. Supervisor Trap Vector Base Address Capability Register (stvec)

The WARL `stvec` register is widened to a capability.

When the S-mode execution environment starts, the maximum value is the Root Executable capability.



Figure 22. Supervisor trap-vector base-capability register

The handling of stvec (RVY) is identical to mtvec (RVY), but in S-mode.

18.1.2. Supervisor Status Registers (sstatus)

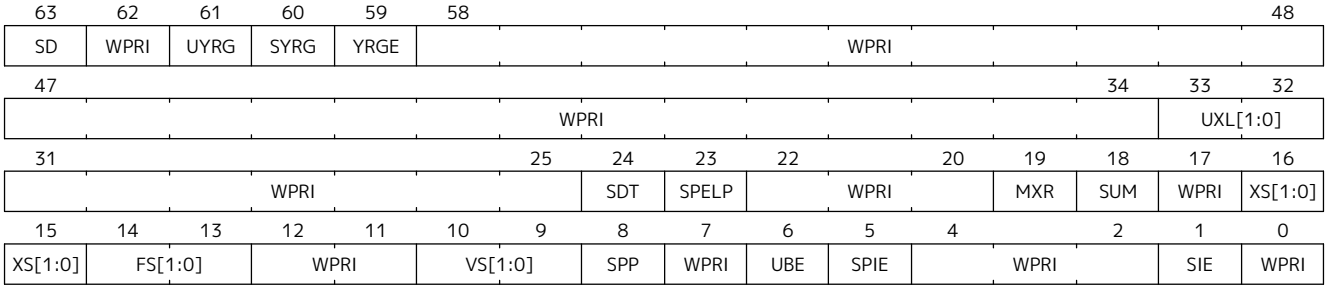


Figure 23. Supervisor status (sstatus) register when SXLEN=64

`sstatus` makes visible the UYRG, SYRG and YRGE fields added to mstatus (RVY).

18.1.3. Supervisor Scratch Capability Register (sscratch)

The `sscratch` register is widened to a capability.

At the start of the S-mode execution environment, the value of the capability tag of this CSR is zero and the values of the metadata and address fields are UNSPECIFIED.

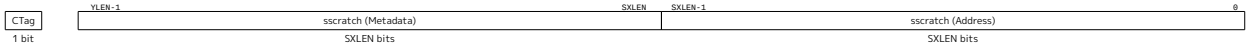


Figure 24. Supervisor scratch capability register

18.1.4. Supervisor Exception Program Counter Capability (sepc)

The WARL `sepc` register is widened to a capability.

When the S-mode execution environment starts, the maximum value is the Root Executable capability.

The handling of sepc (RVY) is identical to mepc (RVY), but in S-mode (i.e. applying to SRET (RVY)).

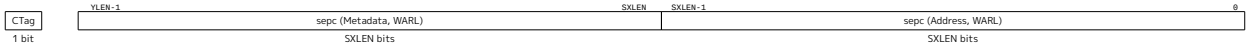


Figure 25. Supervisor exception program counter capability register

18.2. Supervisor-Level CSRs added by RVY

18.2.1. Supervisor Thread Identifier Capability (stidc)

The stidc register is used to identify the current software thread in S-mode, using the method defined in the section for the unprivileged utidc CSR.

At the start of the S-mode execution environment, the value of the capability tag of this CSR is zero and the values of the metadata and address fields are UNSPECIFIED.

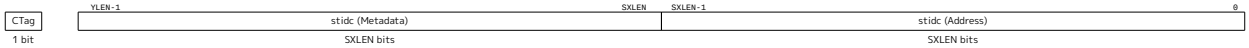


Figure 26. Supervisor thread identifier capability register

18.3. Supervisor-Level CSRs modified by RVY

18.3.1. Supervisor Environment Configuration Register (senvcfg)

Ssyhybrid adds the Y bit to **senvcfg**. The Y bit can be used to disable or enable RVY for U-mode, as described in Chapter 20, *"Smyhybrid and Ssyhybrid" Extensions, Version 1.0*.

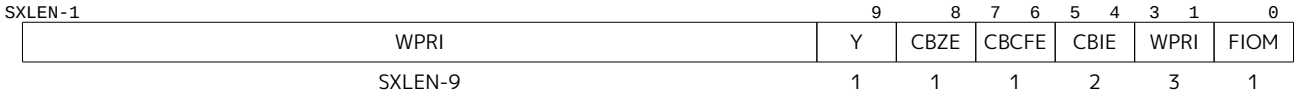


*Figure 27. Supervisor environment configuration register (**senvcfg**)*

18.3.2. Supervisor Cause Register (scause)

RVY adds new exception codes for CHERI exceptions as in mcause (RVY).

18.3.3. "Ssstateen" Integration

The TIDC bit in `sstateen0` controls access to the utidc CSR. See utidc for a description of the usage.

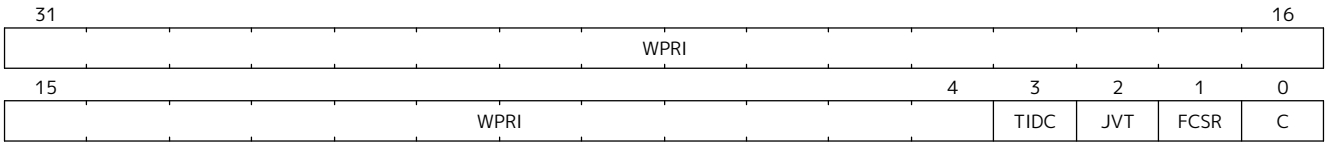


Figure 28. Supervisor State Enable 0 Register (`sstateen0`)

18.4. Modified SRET Behavior

SRET (RVY) reads the full YLEN bits of the sepc (RVY) register and unseals it prior to exception return if it is sealed as a Sealed entry point capability.

18.4.1. SRET (RVY)

See MRET (RVY).

18.4.2. MRET (RVY)

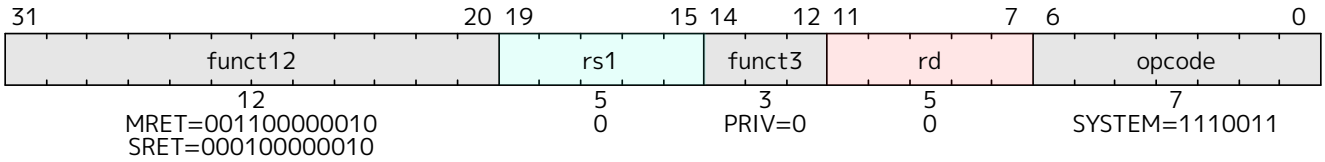
Synopsis

Trap Return (MRET, SRET)

Mnemonics

```
mret
sret
```

Encoding



Description

Return from machine mode (MRET) or supervisor mode (SRET) trap handler.

MRET unseals mepc (RVY) and writes the result into pc.

SRET unseals sepc (RVY) and writes the result into pc.

Exceptions

An illegal instruction exception is raised when pc does not grant ASR-permission.

ASR-permission is required as MRET and SRET require access to privileged CSRs.

Prerequisites (MRET)

Machine-Level ISA, Smy

Prerequisites (SRET)

Supervisor-Level ISA, Ssy

Operation for MRET

```
if   cur_privilege() != Machine
then { handle_illegal(); RETIRE_FAIL }
else if not(ext_check_xret_priv (Machine))
then { ext_fail_xret_priv(); RETIRE_FAIL }
else {
  set_next_pc(exception_handler(cur_privilege(), CTL_MRET(), PC));
  RETIRE_SUCCESS
}
```

Operation for SRET

```
let sret_illegal : bool = match cur_privilege() {
```

```
  User       => true,
  Supervisor => not(extensionEnabled(Ext_S)) | mstatus[TSR] == 0b1,
  Machine    => not(extensionEnabled(Ext_S))
};
if   sret_illegal
then { handle_illegal(); RETIRE_FAIL }
else if not(ext_check_xret_priv (Supervisor))
then { ext_fail_xret_priv(); RETIRE_FAIL }
else {
  set_next_pc(exception_handler(cur_privilege(), CTL_SRET(), PC));
  RETIRE_SUCCESS
}
```

18.5. S-mode YLEN-bit CSR Summary

Table 61 shows all YLEN-bit S-mode CSRs added or modified by RVY or its extensions.

Table 61. All YLEN-bit S-mode CSRs.

RVY CSR	Extension	Address	Permissions	Reset Value	Description
sscratch (RVY)	Ssy	0x140	SRW, ASR-permission	tag=0, otherwise specified by the platform	Supervisor Scratch Capability
stvec (RVY)	Ssy	0x105	SRW, ASR-permission	Root Executable	Supervisor Trap-Vector Base-Address Capability
sepc (RVY)	Ssy	0x141	SRW, ASR-permission	Root Executable	Supervisor Exception Program Counter Capability
stidc	Ssy	0x17f	Read: S, Write: S, ASR-permission	tag=0, otherwise specified by the platform	Supervisor Thread ID

Table 62. Action taken on writing to YLEN-bit S-mode CSRs

RVY CSR	Extension	Action on XLEN write	Action on YLEN write
sscratch (RVY)	Ssy	Update the CSR using YADDRW.	Direct write
stvec (RVY)	Ssy	Apply Invalid address conversion. Always update the CSR with YADDRW even if the address did not change, including the MODE field in the address for simplicity. Vector range check if vectored mode is programmed.	Apply Invalid address conversion. Always update the CSR with YADDRW even if the address did not change, including the MODE field in the address for simplicity. Vector range check if vectored mode is programmed.
sepc (RVY)	Ssy	Apply Invalid address conversion. Always update the CSR with YADDRW even if the address did not change.	Apply Invalid address conversion and update the CSR with the result if the address changed, direct write if address did not change
stidc	Ssy	Update the CSR using YADDRW.	Direct write

Code pointers and data pointers in Table 63 are WARL CSRs that do not need to store full 64-bit addresses on RV64, and so may be subject to Invalid address conversion on writing.

Table 63. YLEN-bit S-mode CSRs storing code pointers or data pointers

RVY CSR	Extension	Code Pointer	Data Pointer	Unseal On Execution
stvec (RVY)	Ssy	✔		
sepc (RVY)	Ssy	✔		✔

Chapter 19. "Sv39/Sv48/Sv57 (RVY)" Page-Based Virtual-Memory Extensions, Version 1.0

When the base architecture is RV64Y, the Sv39, Sv48, and Sv57 page-based virtual-memory extensions allocate the `RVY` PTE field to control capability stores.

CHERI checks are made on the effective address according to the current translation scheme. I.e., on the virtual address if translation is enabled or the physical address if translation is disabled.

Implicit memory accesses made by the page table walker are not subject to CHERI checks.

A future extension may add CHERI checks to the page table walker.

RVY does not modify the behavior of SFENCE., or SINVAL.*. Address operands passed to them are received as XLEN-bit values, and so the metadata and capability tag are ignored. A future extension may add CHERI checks to these instructions.*

19.1. Limiting Capability Propagation

Page table enforcement can allow the operating system to limit the flow of capabilities between processes. It is highly desirable that a process should only possess capabilities that have been issued for that address space by the operating system. Unix processes may share memory for efficient communication, but capability pointers must not be shared across these channels into a foreign address space. An operating system might defend against this by only issuing a capability to the shared region that does not grant the load/store capability permission. However, there are circumstances where portions of general-purpose, mmapped memory become shared, and the operating system must prevent future capability communication through those pages. This is not possible without restructuring software, as the capability for the original allocation, which spans both shared memory and private memory, would need to be deleted and replaced with a list of distinct capabilities with appropriate permissions for each range. Such a change would not be transparent to the program. Such sharing through virtual memory is on the page granularity, so preventing capability writes with a PTE permission is a natural solution.*

** allocated using mmap*

19.2. The *pte.rvy* Field for Capability Flow Control

As RV32Y has no field allocated in the PTE, capability loads and stores operate as normal. Therefore, the RVY field is reserved in the RV64 PTE formats only (e.g., Figure 29) for use by RV64Y extensions. The principle is that extensions, such as Svyrg may redefine the usage of the RVY field by adding an enable bit into `Xstatus`.

The definition of the RVY field is as follows:

Table 64. Definition of the pte.rvy field

Bit	Name	Comment
pte.rvy[3]	*pte.y*	Enable capability (Y) access
pte.rvy[2]	*reserved*	*reserved*
pte.rvy[1]	*reserved*	*reserved*
pte.rvy[0]	*reserved*	*reserved*

If *pte.y*=0 then:

- All capability loads set the loaded capability tag to zero
- All capability stores of valid capabilities raise a CHERI Store/AMO Page Fault.

If *pte.y*=1 then:

- All capability loads and capability stores operate as normal.



The reserved bits pte.rvy[2:0] follow the standard reserved behavior for PTE bits. These bits are allocated by Svyrg, which redefines the entire pte.rvy field when enabled.

See Section 21.3 for the effect on the Virtual Address Translation Process.

19.3. Invalid Virtual Address Handling

When address translation is in effect for RV64Y, the upper bits of virtual memory addresses must match for the address to be valid.

The CSRs shown in Table 60 and Table 63, as well as the pc, need not hold all possible invalid addresses.

However, the bounds encoding of capabilities depends on the address value if the bounds are not infinite.

Therefore, if the bounds are not infinite, implementations must not convert an invalid address to a different invalid address, except as permitted by the rules in the following subsections.

If the bounds could not be decoded due to the address being invalid, then a *CHERI Instruction Access Fault*, *CHERI Load Access Fault*, or *CHERI Store/AMO Access Fault* exception is raised as appropriate.



Not requiring the implementation to decode the bounds for invalid addresses reduces the size of bounds comparators from 64 bits to the supported virtual address width.

19.3.1. Updating CSRs

A CSR may be updated to hold a capability with an invalid address, due to:

- executing instructions, such as CSRRW (RVY)
- hardware updates to CSRs such as storing the pc into mepc (RVY)/sepc (RVY) etc. when taking an exception.

To ensure that the bounds of a valid capability cannot be corrupted:

- If the new address is invalid and the capability bounds are not infinite then invalidate the capability before writing to the CSR.



When the new address is invalid and happens to match an invalid address which the CSR can hold, then it is implementation-defined whether to invalidate the capability.

19.3.2. Branches and Jumps

If the effective target address of the jump or branch is invalid, and the authorizing capability's bounds are not infinite, then set the capability tag of the target pc to zero.

A CHERI Instruction Access Fault exception will be raised at the target.

RISC-V harts that do not support RVY normally raise an instruction access fault or page fault after jumping or branching to an invalid address. Therefore, RVY preserves that behavior to ensure that harts supporting RVY and Zyhybrid are fully compatible with RISC-V harts provided that pc and ddc are set to Root Executable and Root Data capabilities, respectively.

19.3.3. Memory Accesses

If the effective address of the memory access is invalid, and the authorizing capability's bounds are not infinite, then raise a CHERI Load Access Fault or CHERI Store/AMO Access Fault exception because the bounds cannot be reliably decoded.

Chapter 20. "Smyhybrid and Ssyhybrid" Extensions, Version 1.0

When using a system with Zyhybrid, it may be desirable to configure privilege levels below M-mode such that they operate as a standard RV32[IE]/RV64[IE] base architecture.

Smyhybrid is mandatory for all Zyhybrid implementations. Smyhybrid adds the ability to disable CHERI in lower privilege modes by making the `Y` bit of menvcfg (RVY) and `misa.Y` writable, and adds ddc to the registers that must have the capability metadata set to the reset value for `misa.Y` to be programmed to zero.

Ssyhybrid is mandatory for all Zyhybrid implementations that support S-mode, and adds the ability to disable CHERI in U-mode by making the `Y` bit of senvcfg (RVY) writable.

The effective CHERI-enable for the current privilege is:

- M-mode: always enabled
- S-mode: `menvcfg.Y`
- U-mode: `menvcfg.Y & senvcfg.Y` (or `menvcfg.Y` if S-mode is not implemented)

To allow standard RVI/RVE code to run at privilege levels below M-mode the registers are initialized as follows:

- `menvcfg.Y=0`
- `senvcfg.Y=0`
- pc.P-bit=1 (for *Integral Pointer Mode*).
- ddc is at most the Root Data capability.
- pc, `xtvec`, `xepc` are at most Root Executable capabilities.

The following occurs when executing code in a privilege mode that has CHERI disabled:

- Custom encoding spaces reallocated as standard encoding spaces for RVY revert to being custom encoding spaces
  - Therefore, RVY instructions are no longer available.
- No CSRs added by RVY or its extensions are available.
- All CSR accesses are XLEN-bit, as in *Integral Pointer Mode*.

Disabling CHERI has no effect on implicit accesses or security checks.

- The last capability written to pc and ddc before disabling CHERI is used to authorize instruction execution and data memory accesses respectively.
- Exceptions and interrupts use the full capability held in `xtvec` and `xepc`.



Disabling CHERI prevents low-privileged Integral Pointer Mode software from interfering with the correct operation of higher-privileged Integral Pointer Mode software that does not perform ddc switches on trap entry and return.

Table 65 summarizes the behavior of a hart for the different CHERI modes.

Table 65. Hart's behavior depending on the effective CHERI enable and CHERI Pointer Mode

	Y[1]=0	Y=1, P-bit=1	Y=1, P-bit=0
Authorizing capability for data memory accesses	ddc	ddc	capability in `rs1`
CSRs added by RVY or its extensions available	✘	✔	✔
Access width of other YLEN-bit CSRs	XLEN	XLEN	YLEN
16-bit RVY instructions available	✘	✘	✔
Wider RVY instructions available	✘	✔	✔
custom-1/2/3 available for custom use	✔	✘	✘
Summary	*Fully RV32[IE]/RV64[IE] compatible*[2]	*Integral Pointer Mode*	*Capability Pointer Mode*

[1] *Y* represents the effective CHERI enable for the current privilege mode.

[2] The hart is fully compatible with standard RISC-V when Y=0 provided that pc, `xtvec`, `xepc` and ddc have not been changed from their reset state.

Chapter 21. "Svyrg" Extension, Version 1.0 for RV64Y

This extension specifies use of the RVY field in the Sv39, Sv48, and Sv57 PTE formats (e.g., see Figure 29).

Page-Based 39-bit Virtual-Memory System (Sv39)

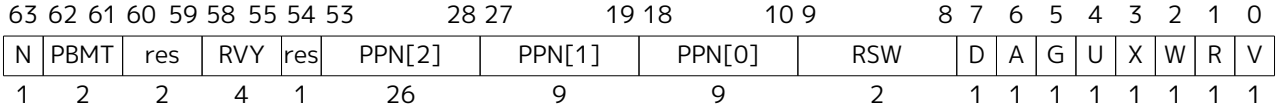


Figure 29. Sv39 page table entry.

Diagram included in the standalone specification for reference only.

Svyrg implies Sv39.

The Svyrg extension is enabled when the `sstatus.YRGE` bit is set to 1.

When `sstatus.YRGE`=0, only *pte.rvy[3]* (*pte.y*) has meaning: it completely enables or disables storing capabilities to the page, as defined in *pte.rvy* field. When `sstatus.YRGE`=1, the entire *pte.rvy* field is redefined as described in this chapter.

The extension adds the ability for S-mode software to quickly enable trapping loads of capabilities from all pages of memory, incrementally allow loads of capabilities from such pages, and track stores of capabilities to all pages of memory. Applied to userspace pages, this has been shown to allow the operating system to implement capability revocation schemes, which allow userspace memory allocators to deterministically guard against use-after-reallocation (Filardo et al., 2024). Applied to kernel pages, they can similarly be used against use-after-reallocation within the kernel itself.

The 4-bit *pte.rvy* field is subdivided into four 1-bit fields, two controlling capability loads (including the capability loaded during an AMO) and two controlling capability stores (again, including the capability stored as part of an AMO).

Table 66. Breakdown of the 4-bit pte.rvy field.

Bit	Name	Comment
Capability load/AMO fields		
pte.rvy[0]	*pte.yr*	Capability readable
pte.rvy[1]	*pte.yrg*	Capability read generation
Capability store/AMO fields		
pte.rvy[2]	*pte.yw*	Capability writable
pte.rvy[3]	*pte.yd*	Capability dirty

After changing `sstatus.YRGE`, executing an SFENCE.VMA instruction with *rs1*=`x0` and *rs2*=`x0` suffices to synchronize address-translation caches with respect to the altered interpretation of *pte.rvy* fields.

Setting `sstatus.YRGE` *is only required once per OS instance.*

If any applications running under the OS require capability access without revocation, then the setting of `sstatus.YRGE=0` *and pte.rvy=1 can be replicated with:*

- *pte.yw = 1, and*
- *pte.yd = 1*

And either:

- *pte.yr = 0, and*
- *pte.yrg = 1, and*

Or:

- *pte.yr = 1, and*
- *pte.yrg =* `sstatus.xYRG`

Therefore `sstatus.YRGE` *is not required to change dynamically and so requiring an SFENCE.VMA on a change should not cause a performance problem.*

When all of the following are true, Svyrg adds two related architectural features, CHERI Load Capability Faults and Capability Dirty Tracking:

1. `sstatus.YRGE` is set to 1.
2. The authorizing capability grants C-permission.
3. The PMA is set to *CHERI Capability Tag*.



It may make sense to subdivide this into two extensions as Capability Dirty Tracking is more generally useful than CHERI Load Capability Faults.

21.1. CHERI Load Capability Faults

pte.yr and *pte.yrg* control whether capability loads or AMOs write the loaded capability tag to `rd`.

When *pte.yr* is set to 0, *pte.yrg* selects the behavior of capability loads: if *pte.yrg* is set to 0, the loaded capability tag is written to `rd` as zero; if *pte.yrg* is set to 1, the load operates as normal.

When *pte.yr* is set to 1, *pte.yrg* can be used to trap on capability loads or AMOs when it does not match the Capability Read Generation value that is represented by the value of `sstatus.UYRG` for userspace pages (*pte.u=1*) or `sstatus.SYRG` for kernel pages (*pte.u=0*).

The implementation raises a CHERI Load Capability Fault when, in addition to the rules above, all of the following are true:

1. A capability load or AMO is executed.
2. *pte.yr* is set to 1.
3. if *pte.u=1*, *pte.yrg* does not equal `sstatus.UYRG`.
4. if *pte.u=0*, *pte.yrg* does not equal `sstatus.SYRG`.
5. The loaded capability is valid, unless the implementation traps conservatively[1].
6. Any other platform-specific rules have not forced the loaded capability tag to be set to 0.

[1] Implementations that trap conservatively may raise the fault regardless of the value of the loaded capability tag, as shown in Table 67.



Checking the value of the capability tag requires taking data-dependent exceptions on loaded capabilities for loads or AMOs. Ideally all implementations would trap precisely (taking the capability tag into account in all cases) rather than conservatively (trapping more often, potentially on every loaded capability). However, the loaded capability tag may not be available

in all implementations when determining whether to raise the exception, and therefore flexibility is permitted. As a result, the software is required to be tolerant of raising the trap when the capability tag is not set to 1, potentially resulting in spurious traps from pages that have pte.yr=1, and so are likely to store valid capabilities.

Implementations that already take synchronous traps on loaded data, such as ECC faults, are recommended to check the loaded capability tag when determining whether to raise the fault.

Table 67. Summary of capability load pte.yr and pte.yrg behavior in the PTEs

pte.yr	*pte.yrg*	tag[1]	Load Capability Behavior
0	0	X	Clear loaded capability tag
0	1	X	Normal operation
1	≠ `sstatus.xYRG`[2]	0	Implementation-defined choice of CHERI Load Capability Fault or normal operation
1	≠ `sstatus.xYRG`[2]	1	CHERI Load Capability Fault
1	= `sstatus.xYRG`[2]	X	Normal operation

[1] The loaded capability tag.

[2] if *pte.u=1*, `sstatus.UYRG`, else `sstatus.SYRG`.



CHERI Load Capability Faults may be used to implement the load-barrier primitive from (Filardo et al., 2024).



In the case of AMOs that could trigger both a CHERI Store/AMO Page Fault, due to storing a valid capability tag, and a CHERI Load Capability Fault, the CHERI Store/AMO Page Fault takes priority as shown in Table 55.

21.2. Capability Dirty Tracking

When *pte.yw* is set to 0, capability stores or AMOs where the to-be-stored capability is valid will raise a CHERI Store/AMO Page Fault.

When *pte.yw* is set to 1, capability stores to the virtual page are permitted. In addition, the *pte.yd* bit indicates that a capability was stored to the virtual page since the last time the *pte.yd* bit was cleared to zero.



This is equivalent to the behavior of the pte.d bit except that pte.yd only tracks stores of valid capabilities instead of all stores.

Capability dirty tracking behavior is enabled when, in addition to the rules above, all of the following are true:

1. A capability store or AMO instruction is executed.
2. The to-be-stored capability is valid.
3. *pte.yw* is set to 1.
4. *pte.yd* is set to 0.

Two schemes for capability dirty tracking are permitted, and the scheme in use is determined by whether the *Svade* or *Svadu* extensions are enabled.

- For *Svade*, take a CHERI Store/AMO Page Fault.

- For *Svadu*, do a hardware update that sets *pte.yd=1*, following the same rules as setting *pte.d*.
  - When setting *pte.yd*, the hardware update also necessarily sets (or leaves set) *pte.a* and *pte.d*.

Table 68. Summary of capability store pte.yw and pte.yd behavior in the PTEs

pte.yw	*pte.yd*	tag[1]	Store Capability Behavior
0	X	0	Normal operation
0	X	1	CHERI Store/AMO Page Fault
1	0	0	Normal operation
1	0	1	CHERI Store/AMO Page Fault (*Svade*) or hardware *pte.yd* update (*Svadu*)
1	1	X	Normal operation

[1] The to-be-stored capability tag.

The capability dirty tracking is resolved during memory translation, but there are cases where it is not known if there will be a capability tag stored to memory or not at this point. Capability dirty tracking *must* be triggered when *pte.yw*=1 and *pte.yd*=0 and the to-be-stored capability is valid, and *may* be triggered in the following case where it is not known if there will be a stored capability tag during translation:

- SC.Y triggers capability dirty tracking if `rs2` is a valid capability, even if the store fails. This matches the semantics of SC.* with regard to *pte.d*.

> *pte.yd must always be set to 1 when a capability with a valid capability tag is written to a virtual page so that software knows which pages have had capabilities stored to them. It may be set to 1 too often, which may cause software to examine the page to check for capabilities when none are present. This is a situation software is required to handle anyway, as it is always possible for all capabilities in a page to be overwritten by non-capability data. In this case, the pte.yd bit would still be set to 1.*
>
> *Future AMOs fall into this category:*
>
> - *For future AMOCAS.Y, it is not known whether the store will happen until the load has executed, and the compare has been done. Therefore, AMOCAS.Y is likely to trigger capability dirty tracking if `rs2` is a valid capability.*
> - *For future AMOADD.Y, the stored capability tag depends upon the loaded capability tag which is not known during translation and on the execution of the YADD. Therefore, AMOADD.Y is likely to always trigger capability dirty tracking.*
>
> *Checking the stored capability tag is less of a burden to the implementation than checking the loaded capability tag for CHERI Load Capability Fault, which is why checking the loaded capability tag is optional behavior. However, a future extension may reduce the burden further by removing the check on the to-be-stored capability tag.*

> *Capability dirty tracking may be used to implement the store-barrier primitive from (Filardo et al., 2024).*

> *The minimum level of PTE support is to set pte.yr=1, pte.yd=1, pte.yw=1, and pte.yrg=0 in all PTEs intended for storing capabilities (e.g., private anonymous mappings) and set `sstatus.UYRG`=0 and `sstatus.SYRG`=0 on all harts, which will enable capabilities to be loaded and stored successfully.*

21.3. Virtual Address Translation Process



This section is copied and modified from the standard translation process. Step 7 is added to support processing of the RVY field and step 10 is modified to include capability dirty tracking (pte.yd).
The intention is to merge these into the main text in the privileged manual, not to maintain a standalone copy like this. The added text is in **bold** *to make it easy to identify.*

A virtual address *va* is translated into a physical address *pa* as follows:

1. Let *a* be `satp`.*ppn*×PAGESIZE, and let *i*=LEVELS-1. (For Sv32, PAGESIZE=2^{12} and LEVELS=2.) The `satp` register must be *active*, i.e., the effective privilege mode must be S-mode or U-mode.
2. Let *pte* be the value of the PTE at address *a*+*va.vpn*[*i*]×PTESIZE. (For Sv32, PTESIZE=4.) If accessing *pte* violates a PMA or PMP check, raise an access-fault exception corresponding to the original access type.
3. If *pte.v*=0, or if *pte.r*=0 and *pte.w*=1, or if any bits or encodings that are reserved for future standard use are set within *pte*, stop and raise a page-fault exception corresponding to the original access type.
4. Otherwise, the PTE is valid. If *pte.r*=1 or *pte.x*=1, go to step 5. Otherwise, this PTE is a pointer to the next level of the page table. Let *i*=*i*-1. If *i*<0, stop and raise a page-fault exception corresponding to the original access type. Otherwise, let *a*=*pte.ppn*×PAGESIZE and go to step 2.
5. A leaf PTE has been reached. If *i>0* and *pte.ppn*[*i*-1:0] ≠ 0, this is a misaligned superpage; stop and raise a page-fault exception corresponding to the original access type.
6. Determine if the requested memory access is allowed by the *pte.u* bit, given the current privilege mode and the value of the SUM and MXR fields of the **mstatus** register. If not, stop and raise a page-fault exception corresponding to the original access type.
7. **For RV64Y only: determine if the requested capability store is allowed by the *pte.rvy* field. If not, stop and raise a CHERI Store/AMO Page Fault exception.**
8. Determine if the requested memory access is allowed by the *pte.r*, *pte.w*, and *pte.x* bits, given the Shadow Stack Memory Protection rules. If not, stop and raise an access-fault exception.
9. Determine if the requested memory access is allowed by the *pte.r*, *pte.w*, and *pte.x* bits. If not, stop and raise a page-fault exception corresponding to the original access type.
10. If *pte.a*=0, or if the original memory access is a store and *pte.d*=0, **or if the original memory access is a capability store with the to-be-stored capability tag set to 1 and *pte.yd*=0**[1]:
    - If the Svade extension is implemented, stop and raise a page-fault exception corresponding to the original access type.
    - If a store to the PTE at address *a*+*va.vpn*[*i*]×PTESIZE would violate a PMA or PMP check, raise an access-fault exception corresponding to the original access type.
    - Perform the following steps atomically:
      - Compare *pte* to the value of the PTE at address *a*+*va.vpn*[*i*]×PTESIZE.
      - If the values match, set *pte.a* to 1 and, if the original memory access is a store, also set *pte.d* to 1, **and if the original memory access is a capability store with the to-be-stored capability tag set to 1, also set *pte.yd* to 1**. Then store *pte* to the PTE at address *a*+*va.vpn*[*i*]×PTESIZE.
      - If the comparison fails, return to step 2.
11. The translation is successful. The translated physical address is given as follows:
    - *pa.pgoff* = *va.pgoff*.
    - If *i*>0, then this is a superpage translation and *pa.ppn*[*i*-1:0] = *va.vpn*[*i*-1:0].
    - *pa.ppn*[LEVELS-1:*i*] = *pte.ppn*[LEVELS-1:*i*].

[1] If Svyrg is enabled (RV64Y only), then this condition causes capability dirty tracking.

Appendix A: RVY Instruction Encoding Overview

ARC-NOTE: The instruction encoding listings below are presented in three different formats (bytefield, wavedrom, and AsciiDoc tables). Please choose which of these formats you would like to see in the final specification.

A.1. Bytefield Encoding Overview

A.1.1. 3OP Instruction Encodings

31 30 29 28 27 26 25	24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0	
0000001	rs2	rs1	000	rd	RVY-A=1111011	YHIW
0000011	rs2≠0	rs1	000	rd	RVY-A=1111011	YADD
0000011	00000	rs1	000	rd	RVY-A=1111011	YMV
0001011	rs2	rs1	000	rd	RVY-A=1111011	YADDRW
0010011	rs2	rs1	000	rd	RVY-A=1111011	YPERMC
0011011	rs2	rs1	000	rd	RVY-A=1111011	YBNDSW
0100011	rs2	rs1	000	rd	RVY-A=1111011	YBNDSRW
0101011	rs2	rs1	000	rd≠0	RVY-A=1111011	YMODEW
0101011	00000	rs1=0	000	rd=0	RVY-A=1111011	YMODESWY
0101011	00001	rs1=0	000	rd=0	RVY-A=1111011	YMODESWI
0000101	rs2	rs1	000	rd	RVY-A=1111011	YSH1ADD
0001101	rs2	rs1	000	rd	RVY-A=1111011	YSH2ADD
0010101	rs2	rs1	000	rd	RVY-A=1111011	YSH3ADD
0011101	rs2	rs1	000	rd	RVY-A=1111011	YSH4ADD
0100101	rs2	rs1	000	rd	RVY-A=1111011	YSH1ADD.UW
0101101	rs2	rs1	000	rd	RVY-A=1111011	YSH2ADD.UW
0110101	rs2	rs1	000	rd	RVY-A=1111011	YSH3ADD.UW
0111101	rs2	rs1	000	rd	RVY-A=1111011	YSH4ADD.UW
0000110	rs2	rs1	000	rd	RVY-A=1111011	YEQ
0001110	rs2	rs1	000	rd	RVY-A=1111011	YSS
0000111	rs2	rs1	000	rd	RVY-A=1111011	YUNSEALS
0001111	rs2	rs1	000	rd	RVY-A=1111011	YBLD
0010111	rs2	rs1=0	000	rd	RVY-A=1111011	YSEALE

A.1.2. 2OP Instruction Encodings

31 30 29 28 27 26 25	24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0	
1111000	00000	rs1	000	rd	RVY-A=1111011	YAMASK
1111010	00000	rs1	000	rd	RVY-A=1111011	YBASER
1111010	00001	rs1	000	rd	RVY-A=1111011	YPERMR
1111010	00010	rs1	000	rd	RVY-A=1111011	YTOPR
1111010	00011	rs1	000	rd	RVY-A=1111011	YLENR
1111010	00100	rs1	000	rd	RVY-A=1111011	YTAGR
1111010	00101	rs1	000	rd	RVY-A=1111011	YTYPER
1111010	00110	rs1	000	rd	RVY-A=1111011	YMODER
1111010	00111	rs1	000	rd	RVY-A=1111011	YHIR

A.1.3. YADDI/LY/SY Instruction Encodings

31 30 29 28 27 26 25 24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0	
imm[11:0]	rs1	100	rd	RVY-A=1111011	YADDI
imm[11:0]	rs1≠0	001	rd	RVY-A=1111011	LY
offset[11:5] rs2	rs1≠0	010	offset[4:0]	RVY-A=1111011	SY

A.1.4. AMO Instruction Encodings

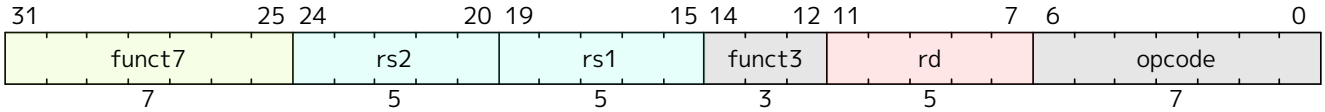
31 30 29 28 27	26	25	24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0	
00010	aq	rl	00000	rs1≠0	011	rd	RVY-A=1111011	LR.Y
00001	aq	rl	rs2	rs1≠0	011	rd	RVY-A=1111011	AMOSWAP.Y
00011	aq	rl	rs2	rs1≠0	011	rd	RVY-A=1111011	SC.Y

A.1.5. MISC Instruction Encodings

31 30 29	28 27 26 25 24 23 22 21 20	19 18 17 16 15	14 13 12	11 10 9 8 7	6 5 4 3 2 1 0	
111	ybndswimm[8:0]	rs1	101	rd	RVY-A=1111011	YBNDSWI

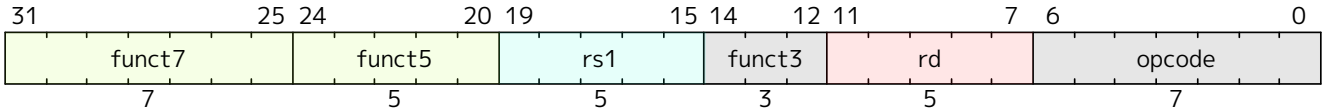
A.2. Wavedrom Encoding Overview

A.2.1. 3OP Instruction Encodings



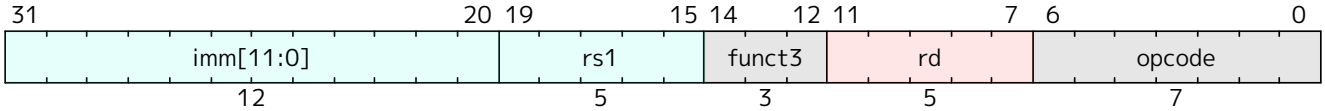
7	5	5	3	5	7
YHIW=0000001	src2	src1	RVY-R=000	dest	RVY-A=1111011
YADD=0000011	increment	src	RVY-R=000	dest	RVY-A=1111011
YADD=0000011	YMV: rs2=0	src	RVY-R=000	dest	RVY-A=1111011
YADDRW=0001011	address	src	RVY-R=000	dest	RVY-A=1111011
YPERMC=0010011	mask	src	RVY-R=000	dest	RVY-A=1111011
YBNDSW=0011011	src2	src1	RVY-R=000	dest	RVY-A=1111011
YBNDSRW=0100011	src2	src1	RVY-R=000	dest	RVY-A=1111011
YMODEW=0101011	src2	src1	RVY-R=000	dest	RVY-A=1111011
YMODEW=0101011	YMODESWY=00000	rs1=0	RVY-R=000	rd=0	RVY-A=1111011
YMODEW=0101011	YMODESWI=00001	rs1=0	RVY-R=000	rd=0	RVY-A=1111011
YSH1ADD=0000101	src2	src1	RVY-R=000	dest	RVY-A=1111011
YSH2ADD=0001101	src2	src1	RVY-R=000	dest	RVY-A=1111011
YSH3ADD=0010101	src2	src1	RVY-R=000	dest	RVY-A=1111011
YSH4ADD=0011101	src2	src1	RVY-R=000	dest	RVY-A=1111011
YSH1ADD.UW=0100101	src2	src1	RVY-R=000	dest	RVY-A=1111011
YSH2ADD.UW=0101101	src2	src1	RVY-R=000	dest	RVY-A=1111011
YSH3ADD.UW=0110101	src2	src1	RVY-R=000	dest	RVY-A=1111011
YSH4ADD.UW=0111101	src2	src1	RVY-R=000	dest	RVY-A=1111011
YEQ=0000110	src2	src1	RVY-R=000	dest	RVY-A=1111011
YSS=0001110	src2	src1	RVY-R=000	dest	RVY-A=1111011
YUNSEALS=0000111	src2	src1	RVY-R=000	dest	RVY-A=1111011
YBLD=0001111	src2	src1	RVY-R=000	dest	RVY-A=1111011
YSEALE=0010111	src2	src1=0	RVY-R=000	dest	RVY-A=1111011

A.2.2. 2OP Instruction Encodings

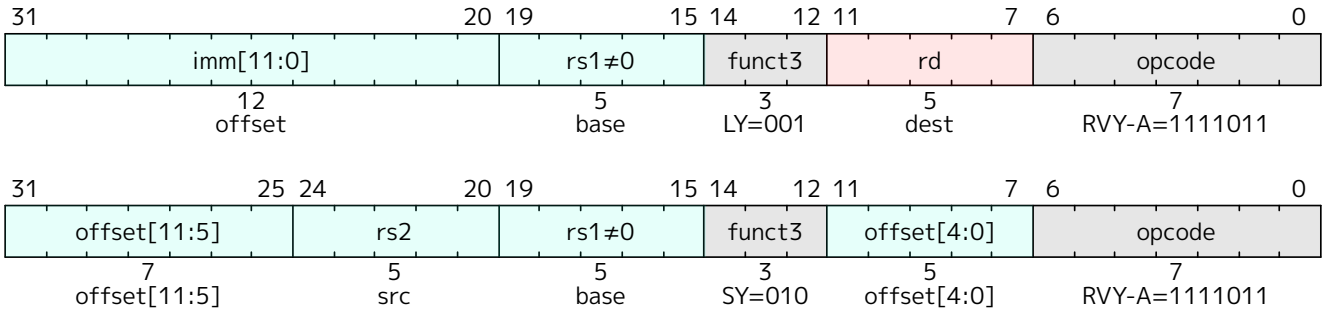


7	5	5	3	5	7
RVY-2OP-XX=1111000	YAMASK=00000	src	RVY-R=000	dest	RVY-A=1111011
RVY-2OP-YX=1111010	YBASER=00000	src	RVY-R=000	dest	RVY-A=1111011
RVY-2OP-YX=1111010	YPERMR=00001	src	RVY-R=000	dest	RVY-A=1111011
RVY-2OP-YX=1111010	YTOPR=00010	src	RVY-R=000	dest	RVY-A=1111011
RVY-2OP-YX=1111010	YLENR=00011	src	RVY-R=000	dest	RVY-A=1111011
RVY-2OP-YX=1111010	YTAGR=00100	src	RVY-R=000	dest	RVY-A=1111011
RVY-2OP-YX=1111010	YTYPER=00101	src	RVY-R=000	dest	RVY-A=1111011
RVY-2OP-YX=1111010	YMODER=00110	src	RVY-R=000	dest	RVY-A=1111011
RVY-2OP-YX=1111010	YHIR=00111	src	RVY-R=000	dest	RVY-A=1111011

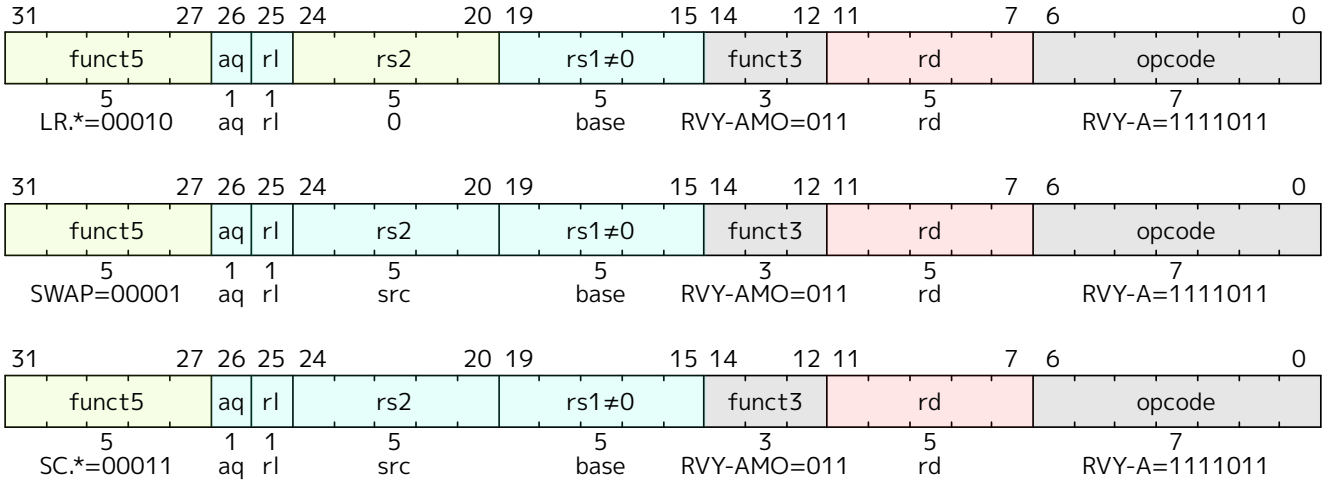
A.2.3. YADDI/LY/SY Instruction Encodings



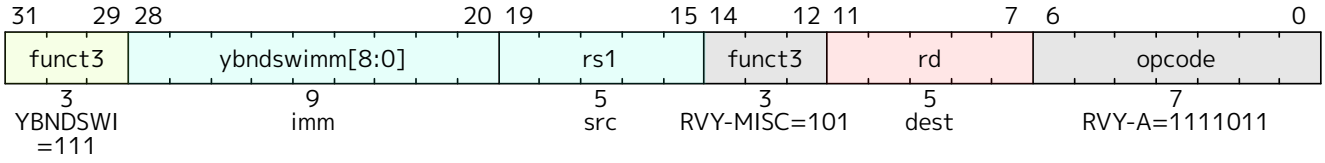
12	5	3	5	7
imm	src	YADDI=100	dest	RVY-A=1111011



A.2.4. AMO Instruction Encodings



A.2.5. MISC Instruction Encodings



A.3. AsciiDoc Table Encoding Overview

A.3.1. 3OP Instruction Encodings

3OP Instruction Encodings						
31:25	24:20	19:15	14:12	11:7	6:0	Inst
0000001	rs2	rs1	000	rd	RVY-A=1111011	YHIW
0000011	rs2≠0	rs1	000	rd	RVY-A=1111011	YADD
0000011	00000	rs1	000	rd	RVY-A=1111011	YMV
0001011	rs2	rs1	000	rd	RVY-A=1111011	YADDRW
0010011	rs2	rs1	000	rd	RVY-A=1111011	YPERMC
0011011	rs2	rs1	000	rd	RVY-A=1111011	YBNDSW
0100011	rs2	rs1	000	rd	RVY-A=1111011	YBNDSRW
0101011	rs2	rs1	000	rd≠0	RVY-A=1111011	YMODEW

0101011	00000	rs1=0	000	rd=0	RVY-A=1111011	YMODESWY
0101011	00001	rs1=0	000	rd=0	RVY-A=1111011	YMODESWI
0000101	rs2	rs1	000	rd	RVY-A=1111011	YSH1ADD
0001101	rs2	rs1	000	rd	RVY-A=1111011	YSH2ADD
0010101	rs2	rs1	000	rd	RVY-A=1111011	YSH3ADD
0011101	rs2	rs1	000	rd	RVY-A=1111011	YSH4ADD
0100101	rs2	rs1	000	rd	RVY-A=1111011	YSH1ADD.UW
0101101	rs2	rs1	000	rd	RVY-A=1111011	YSH2ADD.UW
0110101	rs2	rs1	000	rd	RVY-A=1111011	YSH3ADD.UW
0111101	rs2	rs1	000	rd	RVY-A=1111011	YSH4ADD.UW
0000110	rs2	rs1	000	rd	RVY-A=1111011	YEQ
0001110	rs2	rs1	000	rd	RVY-A=1111011	YSS
0000111	rs2	rs1	000	rd	RVY-A=1111011	YUNSEALS
0001111	rs2	rs1	000	rd	RVY-A=1111011	YBLD
0010111	rs2	rs1=0	000	rd	RVY-A=1111011	YSEALE

A.3.2. 2OP Instruction Encodings

2OP Instruction Encodings						
31:25	24:20	19:15	14:12	11:7	6:0	Inst
1111000	00000	rs1	000	rd	RVY-A=1111011	YAMASK
1111010	00000	rs1	000	rd	RVY-A=1111011	YBASER
1111010	00001	rs1	000	rd	RVY-A=1111011	YPERMR
1111010	00010	rs1	000	rd	RVY-A=1111011	YTOPR
1111010	00011	rs1	000	rd	RVY-A=1111011	YLENR
1111010	00100	rs1	000	rd	RVY-A=1111011	YTAGR
1111010	00101	rs1	000	rd	RVY-A=1111011	YTYPER
1111010	00110	rs1	000	rd	RVY-A=1111011	YMODER
1111010	00111	rs1	000	rd	RVY-A=1111011	YHIR

A.3.3. YADDI/LY/SY Instruction Encodings

YADDI/LY/SY Instruction Encodings						
31:25	24:20	19:15	14:12	11:7	6:0	Inst
imm[11:0]		rs1	100	rd	RVY-A=1111011	YADDI
imm[11:0]		rs1≠0	001	rd	RVY-A=1111011	LY
offset[11:5]	rs2	rs1≠0	010	offset[4:0]	RVY-A=1111011	SY

A.3.4. AMO Instruction Encodings

AMO Instruction Encodings								
31:27	26	25	24:20	19:15	14:12	11:7	6:0	Inst
00010	aq	rl	00000	rs1≠0	011	rd	RVY-A=1111011	LR.Y
00001	aq	rl	rs2	rs1≠0	011	rd	RVY-A=1111011	AMOSWAP.Y
00011	aq	rl	rs2	rs1≠0	011	rd	RVY-A=1111011	SC.Y

A.3.5. MISC Instruction Encodings

MISC Instruction Encodings							
31:29	28:27	26:20	19:15	14:12	11:7	6:0	Inst
111	ybndswimm[8:0]		rs1	101	rd	RVY-A=1111011	YBNDSWI

A.4. RVY-A Instruction Allocation

A.4.1. Funct3 Allocations

funct3	Instruction(s)
000	R-Type (3-op and 1/2-op)
001	LY
010	SY
011	AMO
100	YADDI
101	MISC IMM (YBNDSWI)
110	Unallocated
111	Unallocated

A.4.2. R-Type 3-Operand (funct3=000)

funct7[6:3]	funct7[2:0]							
	000	001	010	011	100	101	110	111
0000		YHIW		YADD/YMV		YSH1ADD	YEQ	YUNSEALS
0001				YADDRW		YSH2ADD	YSS	YBLD
0010				YPERMC		YSH3ADD		YSEALE
0011				YBNDSW		YSH4ADD		
0100				YBNDSRW		YSH1ADD.UW		

0101				YMODEW/YMODESWY/YMODESWI		YSH2ADD.UW		
0110						YSH3ADD.UW		
0111						YSH4ADD.UW		
1000								
1001								
1010								
1011								
1100								
1101								
1110								
1111	1OP/2OP		1OP/2OP					

A.4.3. R-Type 1-Op/2-Op (funct3=000, funct7=1111000)

Note: The 5-bit rs2 field is split into columns rs2[4:3] and rows rs2[2:0].

rs2[2:0]	rs2[4:3]			
	00	01	10	11
000	YAMASK			
001				
010				
011				
100				
101				
110				
111				

A.4.4. R-Type 1-Op/2-Op (funct3=000, funct7=1111010)

Note: The 5-bit rs2 field is split into columns rs2[4:3] and rows rs2[2:0].

rs2[2:0]	rs2[4:3]			
	00	01	10	11
000	YBASER			
001	YPERMR			
010	YTOPR			
011	YLENR			
100	YTAGR			
101	YTYPER			

110	YMODER			
111	YHIR			

A.4.5. AMO Sub-opcode Allocations (funct3=011)

funct7[6:3]	funct7[2:0]							
	000	001	010	011	100	101	110	111
0000					AMOSWAP.Y	AMOSWAP.Y	AMOSWAP.Y	AMOSWAP.Y
0001	LR.Y	LR.Y	LR.Y	LR.Y	SC.Y	SC.Y	SC.Y	SC.Y
0010								
0011								
0100								
0101								
0110								
0111								
1000								
1001								
1010								
1011								
1100								
1101								
1110								
1111								

A.4.6. MISC Sub-opcode Allocations (funct3=101)

funct7[6:3]	funct7[2:0]							
	000	001	010	011	100	101	110	111
0000								
0001								
0010								
0011								
0100								
0101								
0110								
0111								
1000								
1001								
1010								
1011								

1100								
1101								
1110	YBNDSWI	YBNDSWI	YBNDSWI	YBNDSWI	YBNDSWI	YBNDSWI	YBNDSWI	YBNDSWI
1111	YBNDSWI	YBNDSWI	YBNDSWI	YBNDSWI	YBNDSWI	YBNDSWI	YBNDSWI	YBNDSWI

Appendix B: Hybrid Mode Usage Models

The `Zyhybrid` extension enables a hart to switch between *Capability Pointer Mode* and *Integral Pointer Mode*. This functionality supports various usage models that facilitate the co-existence of and transition between non-CHERI software and pure-capability CHERI software.

B.1. High-Level Overview of Compatibility

There are three operating modes on an RVY-enabled core:

RVI/RVE mode (`misa.Y`=0 or `xenvcfg.Y`=0)

The base ISA is RVI or RVE. There is no access to YLEN-bit registers. *custom*-1/2/3 major opcodes remain available for vendor extensions.

RVY pure-capability mode (`misa.Y`=1 and `xenvcfg.Y`=1, and pc.P-bit = 0)

The base ISA is RVY, and *custom*-1/2/3 major opcodes are used for RVY instructions. All loads and stores use a YLEN-bit base register for permissions and bounds checks.

RVY in RVI compatibility mode (`misa.Y`=1 and `xenvcfg.Y`=1, and pc.P-bit = 1)

The base ISA is RVY, and *custom*-1/2/3 major opcodes are used for RVY instructions. However, all loads and stores use an XLEN-bit base register and check ddc for permissions and bounds. This mode requires `Zyhybrid` support.

B.2. Example Usage Models

Purecap Kernel with 64-bit Plain RISC-V Programs

In this model, a CHERI-aware operating system (the kernel) executes in *Capability Pointer Mode*, benefiting from full memory safety for kernel operations. User applications may be standard 64-bit RISC-V programs executing in *Integral Pointer Mode*. The kernel can decide based on the ELF header whether to start the program with CHERI enabled and in pure-capability mode or completely disabled.

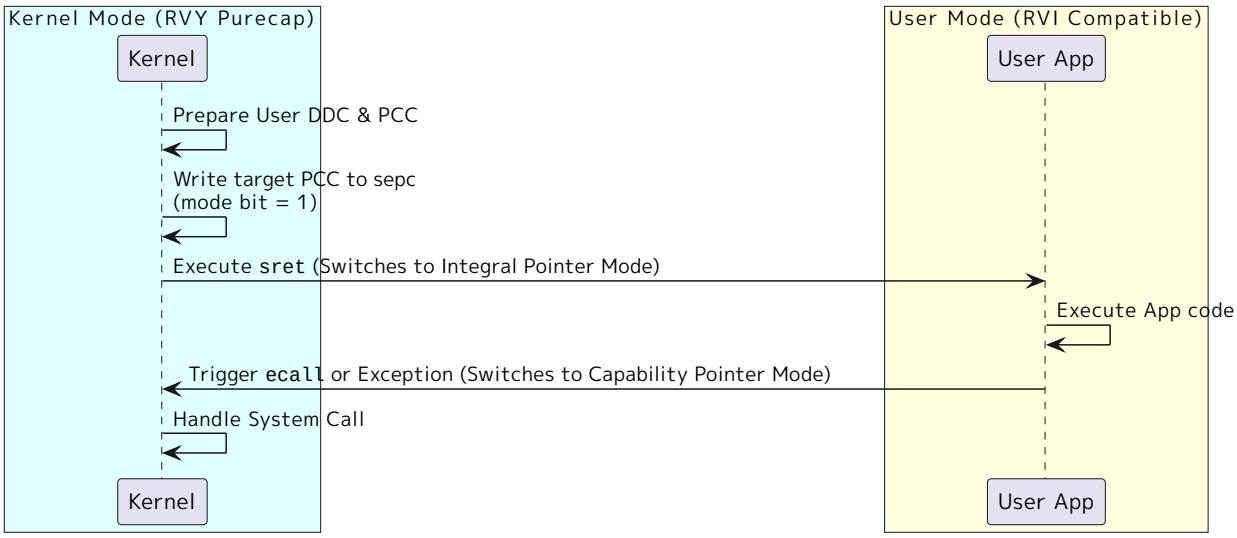
To set up this environment and handle transitions, the following sequence is typical:

1. **Initialize ddc for User Space:** The kernel creates a data capability with read and write permissions. Its bounds are restricted to the user program's data segment. This capability is written to the ddc CSR.
2. **Setup pc for User Space:** The kernel creates a capability for the user program entry point. It sets the P-bit to *Integral Pointer Mode* so the processor starts in *Integral Pointer Mode*. This capability is written to sepc.
3. **Enter User Mode:** The kernel executes the `sret` instruction. This installs the capability from sepc into pc, switching the mode to *Integral Pointer Mode*.
4. **Return to Kernel Mode:** When the user program makes a system call (via `ecall`) or an error occurs, the hardware saves the current pc into sepc. It then loads the kernel's exception vector capability from stvec into pc. Since stvec was set up with its P-bit set for pure-capability execution, this switches the execution mode back to *Capability Pointer Mode*.

   The kernel can disable CHERI for user mode by clearing the appropriate enable bits (e.g., bit in `senvcfg.Y`). This ensures that the user application cannot execute any RVY instructions. It also guarantees that the program has access to the *custom*-1/2/3 opcode space, which CHERI would otherwise use for its own instructions in *Integral Pointer Mode*.



To allow mixing plain RISC-V and pure-capability RISC-V user applications, the kernel must either context-switch the state of `senvcfg.Y`*, or run the plain RISC-V applications in RVI compatibility mode, in which case the custom opcode space reserved by RVY is not available to userspace.*

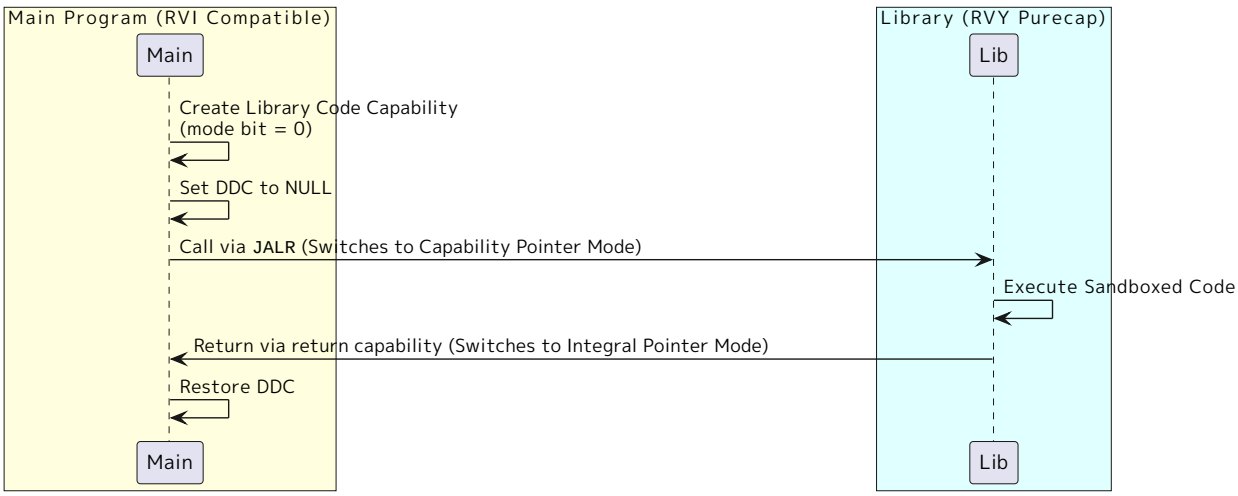


Plain RISC-V Program with a Purecap Library Sandbox

A large non-CHERI application (executing in *Integral Pointer Mode*) may wish to use a specific library that has been recompiled for CHERI (executing in *Capability Pointer Mode*) to process untrusted data (e.g., an image or video codec).

To set up this environment and handle transitions, the following sequence is typical:

1. **Prepare Library Capabilities:** A dynamic linker or runtime creates a capability for the library's entry point with the P-bit set to *Capability Pointer Mode*. This capability only covers the library's code and read-only data section.
2. **Setup Sandbox Environment:** Before running the library, the system sets ddc to NULL. This stops the library from accessing memory unless it uses specific capabilities given to it.
3. **Invoke the Library:** The main program (in *Integral Pointer Mode*) calls the library using JALR (RVY). This installs the capability into pc and switches the mode to *Capability Pointer Mode*. It also saves a return capability in the link register.
4. **Return from the Library:** When finished, the library returns by jumping to the return capability in the link register. This switches the mode back to *Integral Pointer Mode*.

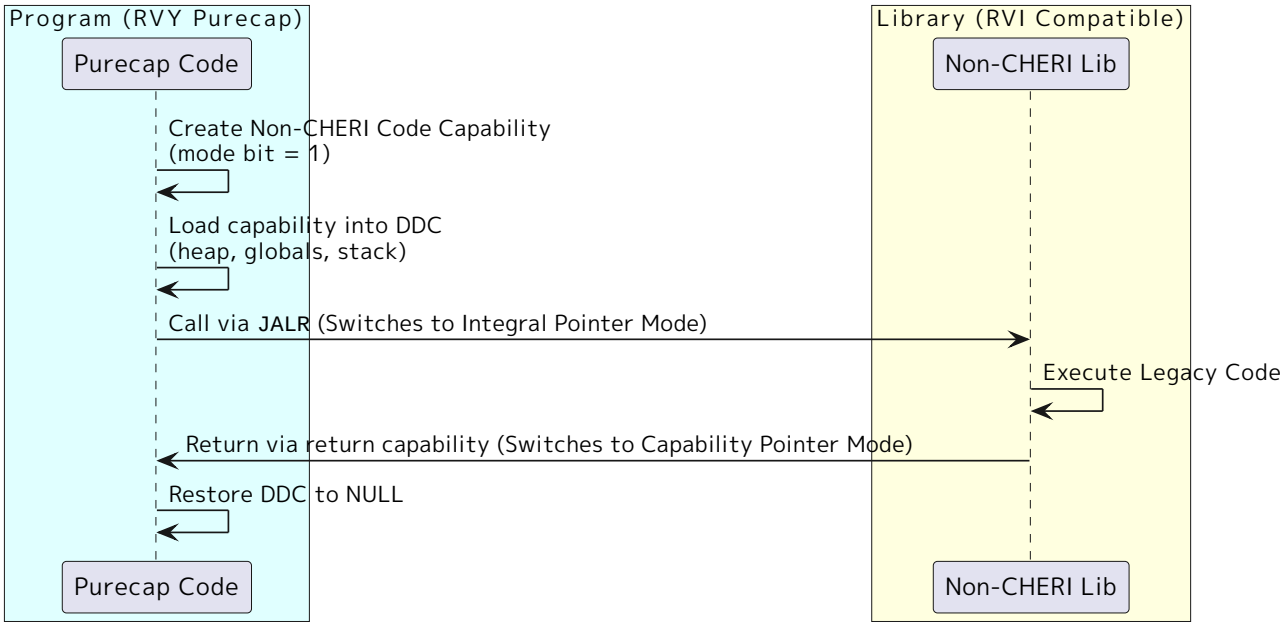


Pure-capability RISC-V Program Calling into a Non-CHERI Library

Conversely, a new pure-capability application may need to link against a non-CHERI binary-only library for which source code is unavailable.

To set up this environment and handle transitions, the following sequence is typical:

1. **Prepare Non-CHERI Code Capability:** The pure-capability program creates a capability for the non-CHERI library's functions with the P-bit set to *Integral Pointer Mode*. This capability only covers the non-CHERI library's code region.
2. **Setup ddc for Non-CHERI Code:** In pure-capability code, ddc is usually NULL. Before calling the non-CHERI library, the program must load a capability into ddc. This capability must cover all memory the non-CHERI library needs (heap, globals, and stack).
3. **Invoke Non-CHERI Code:** The pure-capability program calls the non-CHERI code using JALR (RVY). This switches the processor to *Integral Pointer Mode* and installs the restricted pc.
4. **Restore State on Return:** When the non-CHERI library returns (usually using a standard `JALR` instruction), the pure-capability program restores its state and sets ddc back to NULL.

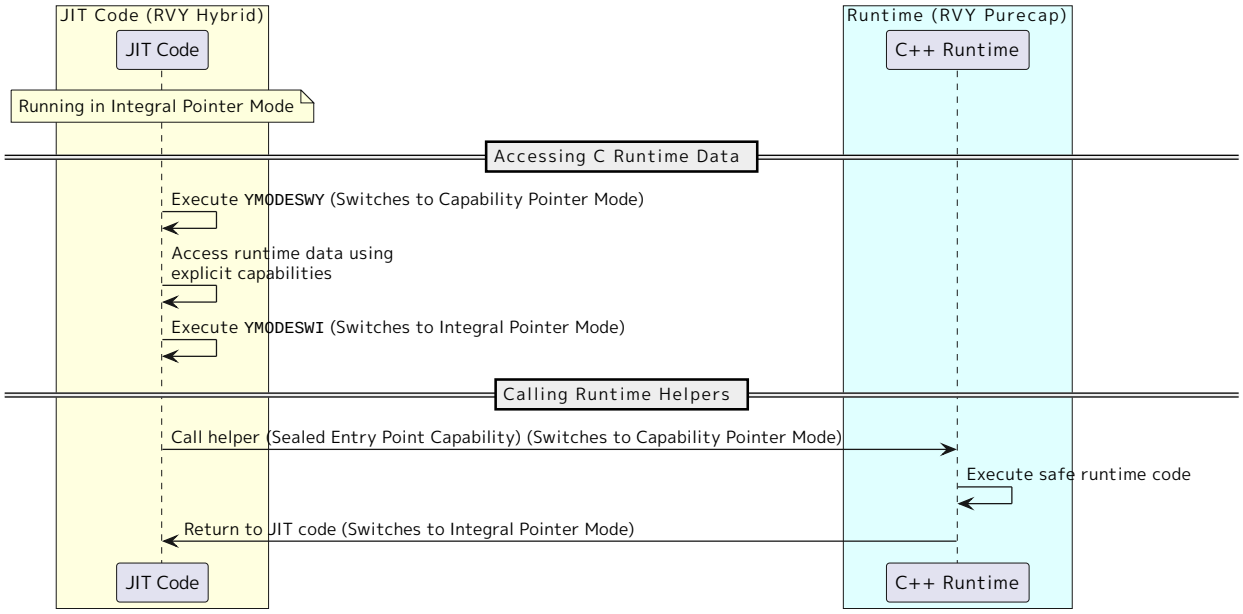


Hybrid Mode: RVI Code with Explicit Capabilities

For example, a Just-In-Time (JIT) compiler running in a pure-capability environment might generate code that uses standard integer addresses (or 32-bit compressed pointers) to avoid the overhead of larger capability pointers. While described here in the context of a JIT compiler and language runtime, this model is generalizable to other scenarios requiring efficient execution of code using smaller pointers within a larger pure-capability application.

This model sets up and switches modes just like the sandboxing models above. The main difference is that the generated code (running in *Integral Pointer Mode*) uses RVY instructions to talk to the pure-capability environment:

- **Accessing C Runtime Data:** The code can access data in the surrounding C++ runtime using explicit capability-based load/store operations. At the same time, it uses standard integer addresses for its own heap via ddc. This can be achieved by executing YMODESWY before the memory access and YMODESWI to return back to address interpretation.
- **Calling Runtime Helpers:** Runtime helper functions are given to the code as a sealed entry point capability. When the code calls these helpers, the processor automatically switches back to *Capability Pointer Mode*. This lets the runtime run with full safety before returning to the JIT code.



B.3. CSRs and Registers Involved in Mode Transitions

When transitioning between modes, certain capability registers and CSRs must be configured to ensure correct execution and authorization:

ddc (Default Data Capability)

This CSR must be written with a capability covering the data memory that the code in *Integral Pointer Mode* is authorized to access. Memory accesses in *Integral Pointer Mode* are implicitly authorized by ddc.

Exception Return Capability Registers (`sepc` or `mepc`)

When a privileged environment prepares to transition to a lower privilege mode via an exception return instruction, it must write the target executable capability to `sepc` or `mepc`. This capability must have its P-bit set to the desired execution mode (*Capability Pointer Mode* or *Integral Pointer Mode*). The exception return instruction installs this capability into pc, thereby setting the mode.

Trap Vector Registers (stvec or mtvec)

These registers hold the capability for the exception vector. When an exception or interrupt occurs, the hardware installs this capability into pc, thereby setting the mode for the trap handler. For example, if stvec was set up with its P-bit set to 0, the processor will switch to *Capability Pointer Mode* on entry to the trap handler.

B.4. CHERI Enable Bits in Setup (When Changing Privilege Levels)

In addition to setting up capabilities and mode bits, the execution environment must manage the CHERI enable bits provided by the privileged architecture:

Per-Privilege Enable Bits

The privileged architecture defines enable bits (often denoted as `Y`) in CSRs such as `misa`, `menvcfg`, and `senvcfg` to control CHERI availability in each privilege mode (see Chapter 20).

Reset State

On reset, CHERI is disabled for privilege levels below M-mode to ensure backwards compatibility with standard RISC-V software. This behavior applies to harts that support execution in both *Capability Pointer Mode* and *Integral Pointer Mode*.

Enabling CHERI

A higher privilege mode must set the appropriate enable bits to allow a lower privilege mode to use RVY instructions or switch between *Capability Pointer Mode* and *Integral Pointer Mode*.

Disabled CHERI with Bounds Enforcement

If CHERI is disabled for a specific privilege mode, that mode cannot execute RVY instructions or change modes. However, the hardware continues to enforce the bounds defined by the last installed pc and ddc. This allows a kernel to run non-CHERI-aware code in a strict integer environment while still confining it within capability bounds.

B.5. Transition Instructions and Frequency

Transitions between modes are performed using specific instructions:

YMODESWI and YMODESWY

These instructions are used to switch between modes without changing the control flow and are typically used when entering or leaving a section of code that requires a different mode.

JALR (RVY)

When jumping to a target capability with the P-bit set differently from the current mode, the jump performs a mode switch. This is commonly used for function calls between compartments or libraries in different modes.

The frequency of transitions depends on the model:

- **Kernel/User boundary:** Transitions occur on every system call, exception, and interrupt, which are relatively infrequent compared to instruction execution.
- **Library calls:** Transitions occur on every call to and return from the sandboxed library. If these calls are frequent, the overhead of mode switching (which may involve saving/restoring registers and clearing state) should be considered.

Appendix C: CHERI System Implications



ARC-NOTE: Please advise whether this appendix should remain in this specification or move to supporting documentation.

CHERI processors need memory systems that support the capability tags in memory.

CHERI systems range from tiny IoT devices up to server chips.

There are two types of bus connections used in SoCs which contain CHERI CPUs:

1. Tag-aware buses, where the bus protocol is extended to carry the capability tag along with the data. This is typically done using user-defined bits in the protocol.
    a. These buses will read capability tags from memory (if capability tags are present in the target memory) and return them to the requestor.
    b. These buses will write the capability tag to memory as an extension of the data write.
2. Non-capability tag-aware buses, i.e., current non-CHERI-aware buses.
    a. Reads of tagged memory will not read the capability tag.
    b. Writes to tagged memory will set the capability tag of any naturally aligned YLEN-bit memory location to zero where any byte is overwritten by the memory write.

The fundamental rule for any CHERI system is that the capability tag and data are always accessed atomically. For every naturally aligned YLEN-bit memory location, it must never be possible to:

1. Update any data bytes without also writing the capability tag.
    a. This implies setting the capability tag to zero if a non-CHERI-aware bus master overwrites a capability in memory.
2. Read a capability tag value with mismatched (stale or newer) data.
3. Set a capability tag to one without also atomically writing the associated data.



Clearing a capability tag may be performed without changing the associated data, provided stale tagged data can never be observed.

C.1. Small CHERI System Example

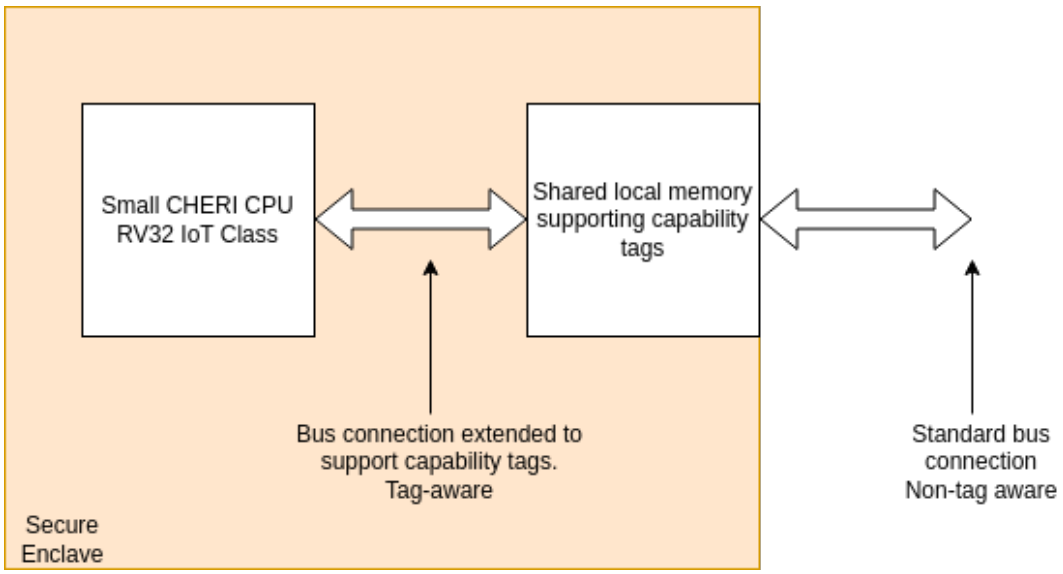


Figure 30. Example small CHERI system with local capability tag storage

This example shows a minimum-sized system where only the local memory is extended to support capability tags. The capability tag-aware region is highlighted. All capability tags are created by the CHERI CPU, and only stored locally. The memory is shared with the system, probably via a secure DMA, which is not capability tag-aware.

Therefore, the connection between CPU and memory is tag-aware, and the connection to the system is not capability tag-aware.

All writes from the system port to the memory must clear any memory capability tags to follow the rules from above.

C.2. Large CHERI System Example

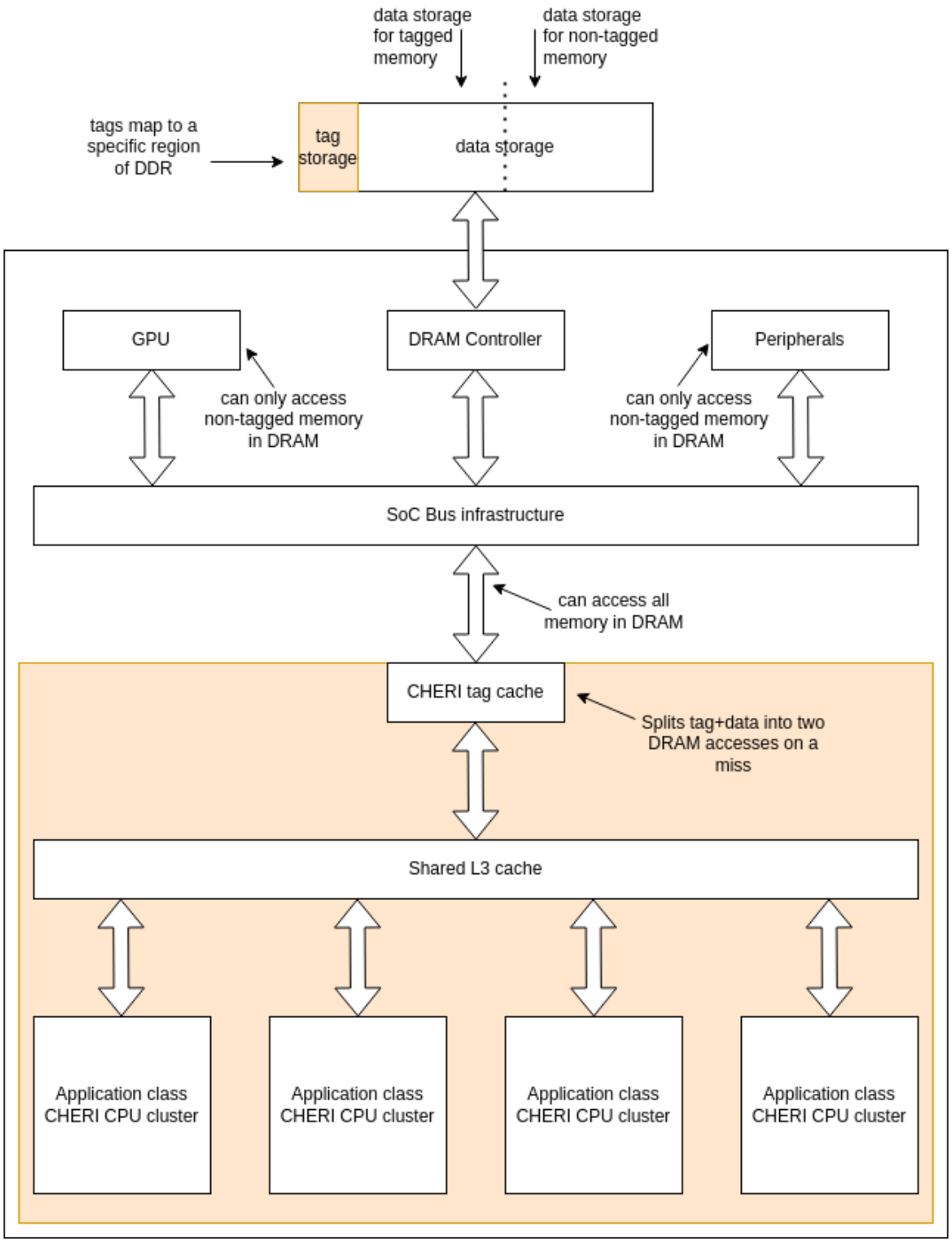


Figure 31. Example large CHERI system with capability tag cache

In the case of a large CHERI SoC with caches, all the cached memory visible to the CHERI CPUs must support capability tags. All memory is backed up by DRAM, and standard DRAM does not offer the extra bit required for CHERI capability tag storage and so a typical system will have a capability tag cache IP.

A region of DRAM is typically reserved for CHERI capability tag storage.

The capability tag cache sits on the boundary of the capability tag-aware and non-tag-aware memory domains, and it provides the bridge between the two. It stores capability tags locally in its cache, and if there is a miss, it will create an extra bus request to access the region of DRAM reserved for capability tag storage. Therefore, in the case of a miss a single access is split into two - one to access the data and one to access the capability tag.

The key property of the capability tag cache is to preserve the atomic access of data and capability tags in

the memory system so that all CPUs have a consistent view of capability tags and data.

The region of DRAM reserved for capability tag storage must only be written by the capability tag cache, therefore no bus initiators should be able to write to the DRAM without the transactions passing through the capability tag cache.

Therefore, the GPUs and peripherals cannot write to the capability tag storage in the DRAM or the capability tag-supporting memory data storage region. These constraints will be part of the design of the network-on-chip. It *is* possible for the GPU and peripherals to read the capability tag-supporting memory data storage region of the DRAM, if required.



It would be possible to allow a DMA to access the capability tagged memory region of the DRAM directly to allow swap to/from DRAM and external devices such as flash. This will require the highest level of security in the SoC, as the CHERI protection model relies on the integrity of the capability tags, and so the root-of-trust will need to authenticate and encrypt the transfer, with anti-rollback protection.

For further information on the capability tag cache see (Joannou et al., 2017).

C.3. Large CHERI Pure-Capability System Example

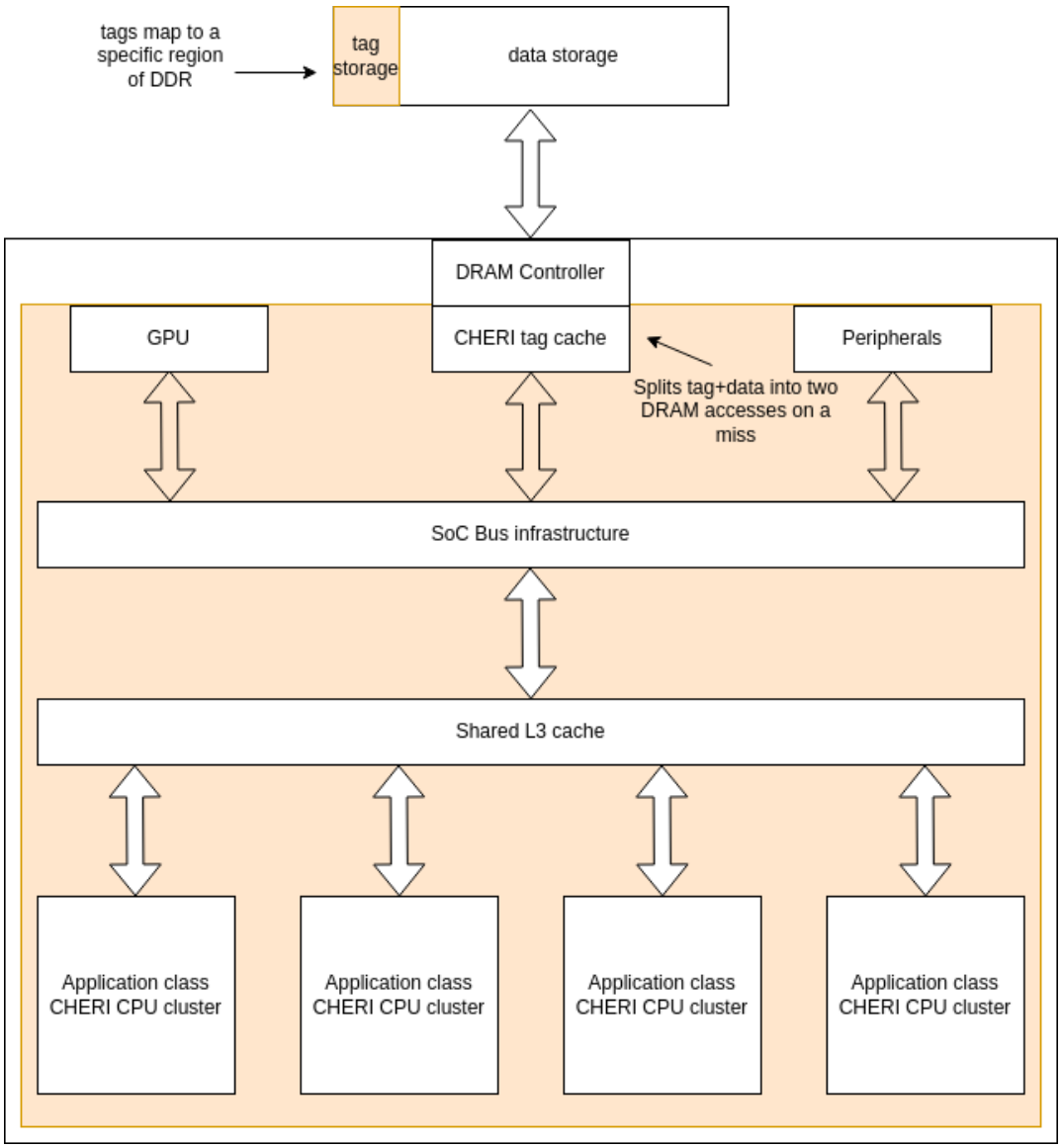


Figure 32. Example large CHERI system with only tag-aware bus masters

In this example every DRAM access passes through the capability tag cache, and so *all* bus masters are capability tag-aware and can access the capability tags associated with memory if permitted by the network-on-chip.

The system topology is simpler than in Figure 31.

There is likely to be a performance difference between the two systems. The main motivation for Figure 31 is to avoid the GPU DRAM traffic needing to look up every capability tag in the capability tag cache, potentially adding overhead to every transaction.

Bibliography

Filardo, N. W., Gutstein, B. F., Woodruff, J., Clarke, J., Rugg, P., Davis, B., Johnston, M., Norton, R., Chisnall, D., Moore, S. W., Neumann, P. G., & Watson, R. N. M. (2024). *Cornucopia Reloaded: Load Barriers for CHERI Heap Temporal Safety*. doi.org/10.1145/3620665.3640416

Joannou, A., Woodruff, J., Kovacsics, R., Moore, S. W., Bradbury, A., Xia, H., Watson, R. N. M., Chisnall, D., Roe, M., Davis, B., Napierala, E., Baldwin, J., Gudka, K., Neumann, P. G., Mazzinghi, A., Richardson, A., Son, S., & Markettos, A. T. (2017). *Efficient Tagged Memory*. www.cl.cam.ac.uk/research/security/ctsrd/pdfs/201711-iccd2017-efficient-tags.pdf

RISC-V. (2023). *RISC-V Unprivileged Specification*. github.com/riscv/riscv-isa-manual/releases/download/riscv-isa-release-056b6ff-2023-10-02/unpriv-isa-asciidoc.pdf

Watson, R. N. M., Neumann, P. G., Woodruff, J., Roe, M., Almatary, H., Anderson, J., Baldwin, J., Barnes, G., Chisnall, D., Clarke, J., Davis, B., Eisen, L., Filardo, N. W., Fuchs, F. A., Grisenthwaite, R., Joannou, A., Laurie, B., Markettos, A. T., Moore, S. W., … Xia, H. (2023). *Capability Hardware Enhanced RISC Instructions: CHERI Instruction-Set Architecture (Version 9)* (UCAM-CL-TR-987; Issue UCAM-CL-TR-987). University of Cambridge, Computer Laboratory. doi.org/10.48456/tr-987

Woodruff, J., Joannou, A., Xia, H., Fox, A., Norton, R. M., Chisnall, D., Davis, B., Gudka, K., Filardo, N. W., Markettos, A. T., & others. (2019). Cheri Concentrate: Practical compressed capabilities. *IEEE Transactions on Computers*, *68*(10), 1455–1469. doi.org/10.1109/TC.2019.2914037